Explore, Conjecture, Discover: Experimental Mathematics with Mathematica and Python

A Guide to Mathematical Research for High School Students

James H. Choi, Ph.D.assisted by LLMs

2026-04-15

Table of contents

Preface

To the Student

You are living through a strange moment in the history of mathematics.

Tools that once required years of training to use — symbolic algebra systems, high-precision numerical computation, visualization of complex structures — now fit in your pocket. Artificial intelligence can write code, solve equations, and explain concepts on demand. The barriers that once separated professional mathematicians from curious beginners have never been lower.

This should be cause for excitement. It is also cause for caution.

When a tool can do almost anything, the hardest question becomes: *what should I ask it to do?* A calculator is useless to someone who does not understand arithmetic. A powerful AI assistant is useless — or worse, misleading — to someone who does not understand what a valid mathematical argument looks like, what makes evidence convincing, or what it means to ask a precise question.

This book is about developing that understanding. Not by memorizing formulas, but by doing what mathematicians have always done: picking a problem, playing with it, noticing patterns, and asking why.

What This Book Is About

Every chapter in this book starts with a question simple enough to state in one sentence:

Does the Collatz sequence always reach 1?

Are the digits of π randomly distributed?

Can two simple tile shapes cover the infinite plane without ever repeating?

None of these questions has been fully answered. Some have resisted the efforts of the best mathematicians alive for decades or centuries. You will not solve them in a weekend — but you

can explore them, discover real patterns, formulate precise conjectures, and produce genuine mathematical insight.

The computer is your laboratory instrument. Wolfram Language, the computational environment used throughout this book, is designed precisely for this kind of experimental exploration: exact integer arithmetic, symbolic computation, seamless visualization, and access to real-world data all in one place. You will write code not to produce correct answers to known problems, but to generate data from which you will extract your own questions.

This is experimental mathematics — a style of research pioneered in the modern era by mathematicians like Jonathan Borwein and David Bailey, who argued that the computer deserved the same status in mathematics that the telescope holds in astronomy: not a replacement for theory, but an instrument that reveals phenomena theory has not yet reached.

On Artificial Intelligence

You will, inevitably, use AI tools while working through this book. That is fine. Use them to debug code, to look up syntax, to get a second explanation of a concept you found confusing.

But be careful about using AI to answer the research questions. Not because the AI will necessarily give you wrong answers — though it sometimes will — but because getting the answer is not the point. The point is the process: the moment when you notice something unexpected in a plot, form a hypothesis, design a test, and find out whether you were right. That process cannot be outsourced. It is the entire value of the exercise.

There is a deeper reason to be careful. The AI systems you are using were trained on mathematical text written by humans who did the hard work themselves. Their ability to appear mathematically fluent is built on a foundation of genuine human understanding. If you skip the foundation, you will eventually find yourself unable to evaluate whether the AI's output is correct, insightful, or subtly wrong in ways that matter.

The students who will thrive in the AI era are not those who are best at prompting AI systems. They are those who understand their subject deeply enough to direct, evaluate, and correct AI output — and to ask questions the AI has not been trained to answer because no one has answered them yet.

Mathematical research is one of the last domains where genuine novelty is accessible to a motivated beginner. The questions in this book are real open problems or close relatives of them. The OEIS contains sequences that no one has studied yet. The patterns you find in your computational experiments may not be recorded anywhere. That is not a flaw in the book's design. It is the design.

How to Use This Book

Each chapter has two parts: the main exposition and the research topics.

The **main exposition** introduces a mathematical area through a sequence of concrete computations. Follow along in Wolfram Language. Run the code. Look at the output. Try changing parameters. The understanding you build by *doing* is qualitatively different from the understanding you build by reading.

The **research topics** are open-ended investigations. They are designed to be hard enough that you cannot complete them in an hour, but tractable enough that a persistent student with the tools in this book can make real progress. They are not homework problems with known answers in the back. Some of them may have known answers in the mathematical literature — look them up after you have formed your own conjecture, not before.

The **appendices** contain full Wolfram Language implementations of every named function used in the chapters. When a chapter uses a function like `CollatzSequence` or `PisanoPeriod`, the appendix gives you the complete code with explanation. Read those implementations. Understanding *why* a function is written the way it is will teach you more about both Wolfram Language and the underlying mathematics than any tutorial.

A Note on Proof

This book does not ask you to write proofs. The research topics ask you to conjecture, test, and document — the experimental phase of mathematical work.

This is not because proofs are unimportant. They are the final standard of mathematical truth, and every conjecture in this book is only a conjecture until it is proved. But proof requires a level of theoretical machinery — real analysis, abstract algebra, number theory — that takes years to develop. Experimental mathematics is what you do *before* you can prove something, and sometimes it is what motivates someone else to develop the machinery needed for a proof.

Gauss discovered the Prime Number Theorem empirically — plotting $\pi(x)$ and noticing it looked like $x/\ln x$ — decades before Hadamard and de la Vallée Poussin proved it. Ramanujan wrote down hundreds of formulas he "saw" through intuition and computation before Hardy and Littlewood developed the tools to prove them. Fermat stated his Last Theorem as a conjecture in a margin note in 1637. It was proved in 1995.

You are in good company when you conjecture without proving.

What You Need

This book assumes you know high school algebra and are comfortable with the idea of a function. Familiarity with calculus is useful but not required — the few places where it appears are labeled, and the underlying ideas can be followed computationally.

You need a working installation of Wolfram Language. The free Wolfram Engine (available at wolfram.com) runs all the code in this book. Wolfram Language is also available through Wolfram Cloud at no cost, and as part of Mathematica, which is available at many schools and universities.

You need curiosity and stubbornness. The research topics will resist you. That resistance is the point.

A Personal Note

Mathematics is not a spectator sport. It is not a collection of facts to be memorized, or a set of procedures to be executed. It is a practice — a way of seeing the world as a place full of hidden structure, waiting to be found.

The problems in this book are invitations to that practice. Some of them have fascinated professional mathematicians for generations. You have the same computational tools they have. You have the same basic human capacity to notice a pattern and wonder why.

The landscape around you is changing fast. The tools change. The interfaces change. The AI systems improve. What does not change is the satisfaction of understanding something you did not understand before — of having followed a chain of reasoning from a question to an answer, however partial, that you built yourself.

That satisfaction is what this book is designed to give you access to.

Now open a notebook, and let's begin.

James H. Choi, Ph.D. A Certified Wolfram Language Instructor

Part I

Part I - Numbers Have Secrets

1 Chapter 1 — Prime Numbers: The Atoms of Arithmetic

1.1 What Is a Prime?

A **prime number** is a positive integer greater than 1 that has exactly two divisors: 1 and itself. The first few primes are:

$$2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ...$$

Every other positive integer greater than 1 is **composite** — it can be broken down into smaller factors. For example, $12 = 2 \times 2 \times 3$. The primes are the building blocks from which all other integers are assembled, which is why they are sometimes called the atoms of arithmetic.

The **Fundamental Theorem of Arithmetic** states that every integer greater than 1 can be written as a product of primes in exactly one way (up to order). This makes primes foundational — not just interesting curiosities, but the irreducible units of multiplicative number theory.

Despite their simple definition, primes are distributed among the integers in patterns that remain only partially understood after thousands of years of study.

1.2 Finding Primes: The Sieve of Eratosthenes

The oldest known algorithm for finding primes is the **Sieve of Eratosthenes**, attributed to the Greek mathematician Eratosthenes around 240 BCE. The idea is simple:

1. Write down all integers from 2 to n.
2. Circle 2 (it is prime). Cross out all multiples of 2.
3. Circle the next uncrossed number (3). Cross out all multiples of 3.
4. Repeat: circle the next uncrossed number, cross out its multiples.
5. All circled numbers are prime.

```
SieveOfEratosthenes[100]
```

Output:

```
{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,
 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

There are 25 primes below 100. Wolfram Language has a built-in `Prime[n]` function (the nth prime) and `PrimePi[x]` (the number of primes up to x), but implementing the sieve from scratch is an important exercise — see Section 15.1.1.

1.3 How Many Primes Are There?

Euclid proved around 300 BCE that there are infinitely many primes. His proof is one of the most elegant in all of mathematics: suppose there were only finitely many primes $p_1, p_2, \ldots, p_k$. Then the number $N = p_1 p_2 \cdots p_k + 1$ is not divisible by any of them — contradiction.

But how are primes *distributed* among the integers? The **prime-counting function** $\pi(x)$ counts the number of primes up to x:

$$\pi(x) = \#\{p \leq x : p \text{ is prime}\}$$

```
PlotPrimeCounting[10^6]
```

Output:

[Smooth-looking curve. x-axis: 0 to 10^6. y-axis: 0 to ~78498. Curve rises steeply at first, then flattens gradually. Shape resembles x/ln(x).]

The curve looks smooth — a surprise given that individual primes seem scattered randomly. Gauss observed empirically (before it was proved) that $\pi(x) \approx x / \ln x$. This is the **Prime Number Theorem**, proved in 1896. More accurate approximations exist, but the basic shape — primes become gradually rarer as numbers grow larger — is already visible in this plot.

1.4 Prime Gaps

The **gap** between consecutive primes p_n and p_{n+1} is $g_n = p_{n+1} - p_n$. The first few gaps are:

$$1, 2, 2, 4, 2, 4, 2, 4, 6, 2, 6, \ldots$$

The gap of 1 occurs only between 2 and 3. After that, all primes are odd, so all gaps are even. There are infinitely many gaps of size 2 (the **twin prime conjecture**, unproven), but gaps can also be arbitrarily large: for any k, the k consecutive integers $k! + 2, k! + 3, \ldots, k! + (k + 1)$ are all composite.

```
PlotPrimeGapHistogram[10^5]
```

Output:

[Histogram of prime gaps for primes up to 10^5. x-axis: gap size (2, 4, 6, …, up to ~72). y-axis: frequency. Gap of 2 is most common, frequency drops roughly exponentially, with all odd gaps having zero count.]

The distribution of large prime gaps is predicted (but not proved) by the **Cramér probabilistic model**: near a prime p, gaps are approximately Poisson-distributed with mean $\ln p$. This is a heuristic, not a theorem — see Research Topic 1.

1.5 Goldbach's Conjecture

In 1742, the mathematician Christian Goldbach wrote to Euler with a conjecture: *every even integer greater than 2 is the sum of two primes.* For example:

$$4 = 2 + 2, \quad 6 = 3 + 3, \quad 8 = 3 + 5, \quad 10 = 3 + 7 = 5 + 5, \quad 100 = 3 + 97 = 11 + 89 = \ldots$$

This has been verified computationally for all even integers up to 4×10^{18}. It has never been proved.

```
VerifyGoldbach[10^4]
```

Output:

[Returns True if all even integers from 4 to 10^4 satisfy Goldbach's conjecture. Output: True. Computation is fast.]

We can also visualize the number of ways each even integer can be expressed as a sum of two primes — the **Goldbach comet**:

```
PlotGoldbachComet[2000]
```

Output:

[Scatter plot. x-axis: even integers from 4 to 2000. y-axis: number of distinct Goldbach representations. Points form a comet-like fan shape, spreading outward and upward as x increases, with minimum always staying above 1.]

The fact that the minimum number of representations never reaches zero is exactly what Goldbach's conjecture claims — but verifying it computationally only pushes the frontier; it does not prove it.

1.6 The Ulam Spiral

In 1963, mathematician Stanislaw Ulam, bored during a conference, began writing integers in a spiral pattern on graph paper and circling the primes. He noticed something unexpected: the primes clustered along diagonal lines.

The **Ulam spiral** is constructed by writing integers in a counterclockwise spiral starting from 1, then marking the primes:

```
UlamSpiral[101]
```

Output:

[101×101 grid. Black squares mark prime positions. Clear diagonal streaks visible, especially along a few main diagonals. Background of scattered marks between the streaks.]

The diagonal clustering has no complete explanation. It is related to the fact that certain quadratic polynomials — like $4n^2 - 2n + 41$ — produce primes at unusually high rates, and quadratic polynomials correspond to diagonals in the spiral. Research Topic 2 investigates this further.

1.7 Further Research Topics

1.7.1 Topic 1 — Prime Gaps and the Cramér Model

The Cramér model predicts that near a prime p, gaps are roughly Poisson-distributed with mean $\ln p$.

```
PrimeGapStats[10^5]
```

Output:

[Summary statistics: mean gap near various ranges of p, compared to ln(p). Actual mean gap is close to ln(p) for large p.]

Your tasks:

1. Compute all prime gaps up to 10^7. Plot a histogram. Does the distribution resemble a Poisson distribution with the predicted mean?
2. Define a "record gap" as a gap larger than all previous gaps. Find the first 20 record gaps. These are called **maximal prime gaps** (OEIS A002386). Do they follow any pattern?
3. Count pairs of primes with gap exactly 2 (twin primes) up to 10^6. Plot the cumulative count of twin primes vs. x. Does it grow like $x/(\ln x)^2$ as predicted?
4. Find the first prime desert of length > 100 — a gap larger than 100 between consecutive primes. How large a search is needed?
5. The polynomial $n^2 - n + 41$ produces primes for $n = 1, \ldots, 40$. For primes up to 10^6, what fraction lie on diagonals corresponding to this polynomial in the Ulam spiral?

1.7.2 Topic 2 — The Ulam Spiral and Quadratic Polynomials

```
UlamSpiral[201]
CompareUlamSpirals[{1, 17, 41}]  (* start spiral at different centers *)
```

Output:

[Three 201×201 spiral images side by side, starting from 1, 17, and 41. Diagonal structure shifts depending on center value.]

Your tasks:

1. Identify the three or four most prominent diagonals in `UlamSpiral[201]`. For each diagonal, sample 20 primes on it and determine which quadratic polynomial $an^2 + bn + c$ they satisfy.
2. The polynomial $n^2 + n + 41$ (Euler's lucky polynomial) gives primes for $n = 0, \ldots, 39$. Find the next polynomial of the form $n^2 + n + p$ with the longest prime-producing run. What property must p have?
3. Change the spiral center from 1 to 41. Does the clustering pattern improve? Conjecture: which center value produces the clearest diagonal structure?
4. Build a density map: for each cell in a 201×201 spiral, color it by the fraction of nearby cells that contain primes within a radius of 5. Does the diagonal structure persist in the smoothed density?

1.7.3 Topic 3 — Chebyshev's Bias

Among primes of the form $4k + 1$ and $4k + 3$, the $4k + 3$ primes tend to lead in the race up to N — a phenomenon called **Chebyshev's bias**.

```
ChebyshevRace[10^5]
```

Output:

[Two-line plot. x-axis: N from 2 to 10^5. y-axis: cumulative count. One line for primes ≡ 1 (mod 4), one for ≡ 3 (mod 4). The 3 mod 4 line stays above for most of the range.]

Your tasks:

1. Plot the cumulative count of primes $\equiv 1 \pmod 4$ vs. primes $\equiv 3 \pmod 4$ up to 10^6. Find the smallest $N > 5$ where primes $\equiv 1 \pmod 4$ take the lead (this is called a "Chebyshev bias violation").
2. Repeat for primes $\equiv 1 \pmod 3$ vs. $\equiv 2 \pmod 3$. Does the same phenomenon occur?
3. Define the **bias function** $B(N) = \pi(N; 4, 3) - \pi(N; 4, 1)$ where $\pi(N; k, r)$ counts primes $\leq N$ with remainder r mod k. Plot $B(N)$ for N up to 10^6. Is $B(N)$ always positive? Does it ever reach 0?
4. Repeat the analysis for the race modulo 6: primes $\equiv 1 \pmod 6$ vs. $\equiv 5 \pmod 6$.

1.7.4 Topic 4 — Goldbach in Depth

```
GoldbachRepresentations[100]
PlotGoldbachComet[10000]
```

Output:

[List of all Goldbach representations of 100. Plot of number of representations for even n up to 10000.]

Your tasks:

1. For even n from 4 to 10^4: find the even number with the *most* Goldbach representations. Plot the number of representations vs. n. Does it grow, and if so, how fast?
2. The **Goldbach function** $g(n)$ gives the number of representations of n as a sum of two primes. Plot $g(n)$ and its smoothed average. Conjecture a formula for the average growth rate.
3. **Weak Goldbach conjecture** (proved 2013): every odd integer greater than 5 is the sum of three primes. Verify this for all odd integers up to 10^4 and record the minimum number of ways each can be expressed.
4. Find all even integers below 10^4 that have a Goldbach representation using two *twin* primes (primes p and $p+2$). What fraction of even integers does this cover?

1.7.5 Topic 5 — Mersenne Primes

A **Mersenne prime** is a prime of the form $2^p - 1$ where p itself is prime. The first few are $3, 7, 31, 127$.

```
MersennePrimeCandidates[100]
```

Output:

[Table of p values (prime) up to 100, with 2^p - 1 and whether it is prime. Many fail despite p being prime.]

Your tasks:

1. For all primes $p \leq 100$: test whether $2^p - 1$ is prime. What fraction pass? This requires big integer arithmetic — verify that Wolfram Language handles it exactly.
2. The Lucas-Lehmer test is the standard primality test for Mersenne numbers. Implement it (see Section 15.1.11) and verify it agrees with Wolfram Language's `PrimeQ` for all Mersenne candidates up to $2^{100} - 1$.

3. If $2^p - 1$ is a Mersenne prime, then $2^{p-1}(2^p - 1)$ is a **perfect number** (equals the sum of its proper divisors). Compute the first five perfect numbers and verify they are perfect.
4. Plot the number of digits in $2^p - 1$ vs. p (for prime p). How many decimal digits does the 10th Mersenne prime have?

2 Chapter 2 — The Collatz Conjecture: Simple Rules, Deep Mystery

2.1 A Rule a Child Could Follow

Pick any positive integer. Then repeat the following two steps:

- If the number is **even**, divide it by 2.
- If the number is **odd**, multiply by 3 and add 1.

Keep going until you reach 1.

That is the entire rule. Let us try it starting from 12:

$$12 \to 6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$$

Nine steps. Now try 19:

$$19 \to 58 \to 29 \to 88 \to 44 \to 22 \to 11 \to 34 \to 17 \to 52 \to 26 \to 13 \to 40 \to 20 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to$$

Twenty steps. The sequence bounces up and down unpredictably before finally descending to 1.

These sequences are called **hailstone sequences** because the values rise and fall repeatedly — like hailstones tumbling inside a storm cloud — before eventually hitting the ground at 1.

The Collatz conjecture states: *no matter which positive integer you start with, the hailstone sequence always eventually reaches 1.*

This conjecture was introduced by the German mathematician Lothar Collatz in 1937. It has been verified by computer for all starting values up to approximately 2.36×10^{21}. No counterexample has ever been found. No proof has ever been found either. Paul Erdős remarked: *"Mathematics may not be ready for such problems."*

2.2 Writing the Rule as a Formula

We can write the Collatz rule as a single function:

$$C(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n+1 & \text{if } n \text{ is odd} \end{cases}$$

A hailstone sequence starting at n is the list n, $C(n)$, $C(C(n))$, ... continuing until we reach 1.

In Wolfram Language, `CollatzStep` implements one application of the rule, and `CollatzSequence` produces the full hailstone sequence (see Section 15.2.1 and Section 15.2.2):

```
CollatzStep[12]          (* 6  -- even, divide by 2    *)
CollatzStep[19]          (* 58 -- odd, multiply by 3+1 *)
CollatzSequence[19]
```

Output:

```
6
58
{19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1}
```

2.3 A Famous Example: Starting from 27

The starting value $n = 27$ is famous among Collatz enthusiasts. Despite being a small number, its hailstone sequence takes 111 steps to reach 1 and climbs as high as 9232 — more than 340 times the starting value — before finally coming down.

```
seq27 = CollatzSequence[27];
{Length[seq27] - 1, Max[seq27]}    (* steps, peak *)
```

Output:

```
{111, 9232}
```

Plotting the trajectory shows the characteristic shape: an irregular climb, a sharp peak, and a long staircase descent.

```
PlotCollatzSequence[27]
```

Output:

[Line plot. x-axis: step 0 to 111. y-axis: value 0 to 9232. Jagged ascent peaking near step 77 at 9232, followed by rapid descent to 1.]

2.4 Total Stopping Time

The **total stopping time** of n is simply the number of steps needed to reach 1.

```
TotalStoppingTime /@ {9, 27, 97, 871}
```

Output:

```
{19, 111, 118, 178}
```

We can compute stopping times for all n up to 1000 and plot them:

```
PlotStoppingTimes[1000]
```

Output:

[Scatter plot. x-axis: n from 1 to 1000. y-axis: stopping time 0 to 178. Irregular cloud with no obvious trend. Notable spike near $n = 871$. Faint vertical banding near powers of 2.]

The plot looks almost random, yet it contains hidden structure. Starting values near powers of 2 tend to have *short* stopping times, because 2^k divides by 2 exactly k times and reaches 1 immediately.

2.5 Delay Records

Some starting values are remarkable: their stopping time is longer than that of every smaller starting value. These are called **delay records**.

```
DelayRecords[1000]
```

Output:

```
{{1,0}, {2,1}, {3,7}, {6,8}, {7,16}, {9,19}, {18,20}, {25,23},
 {27,111}, {54,112}, {73,115}, {97,118}, {129,121}, {171,124},
 {231,127}, {313,130}, {327,143}, {649,144}, {703,170}, {871,178}}
```

Each pair is $\{n,$ stopping time$\}$. The jump at $n = 27$ is striking — the stopping time leaps from 23 to 111.

For reference, the starting values with the longest stopping times below each power of ten are:

Upper bound	Champion	Stopping time
10^2	97	118
10^3	871	178
10^4	6171	261
10^5	77031	350
10^6	837799	524

2.6 The Collatz Graph

There is a beautiful way to visualize the conjecture by working *backward*. Instead of asking "where does n go?", ask "which numbers lead to n in one step?"

Reversing the rule, two numbers can map to n:

- $2n$ always maps to n (reverse of "divide by 2").
- $(n-1)/3$ maps to n, but only when $n \equiv 4 \pmod 6$ and $(n-1)/3$ is a positive odd integer.

Starting from 1 and applying these reverse steps repeatedly grows the **Collatz graph** — a tree whose root is 1 and whose branches reach outward. If the conjecture is true, every positive integer appears somewhere in this tree.

```
PlotCollatzTree[6]
```

Output:

[Tree diagram rooted at 1, growing outward to depth 6. Most nodes have one child (the doubling branch). Some nodes have two children. Asymmetric structure, resembling a fractal.]

This bottom-up view restates the conjecture as: *the Collatz tree contains every positive integer.* Researchers have found this angle useful — it connects the problem to questions about trees and graphs that are sometimes easier to reason about.

2.7 Why Is This So Hard?

The hailstone sequences look, in many ways, like random sequences. Whenever n is odd, $3n+1$ is even, so the very next step always divides by 2. Thinking of two steps at a time, an odd number effectively gets multiplied by roughly $3/2$ on average. Since $3/2 < 2$, sequences tend to drift downward over time — and this heuristic supports the conjecture.

But heuristics are not proofs. The sequences are completely deterministic; each starting value produces exactly one sequence. The trouble is that the rule mixes multiplication and division in a way that current mathematics cannot fully control.

In 2019, Terence Tao proved that *almost all* hailstone sequences (in a precise technical sense) eventually descend below any given bound — a major advance, described by *Quanta Magazine* as "one of the most significant results on the Collatz conjecture in decades." Yet even this falls short of a full proof.

2.8 Further Research Topics

2.8.1 Topic 1 — The $5n + 1$ Variant

Replace $3n + 1$ with $5n + 1$: if n is odd, compute $5n + 1$; if n is even, divide by 2.

Starting from $n = 3$: $3 \to 16 \to 8 \to 4 \to 2 \to 1$. Fine. But starting from $n = 13$:

$$13 \to 66 \to 33 \to 166 \to 83 \to 416 \to 208 \to 104 \to 52 \to 26 \to 13 \to \cdots$$

This is a cycle. The $5n + 1$ map does *not* always reach 1.

`ClassifyHailstone` detects whether a sequence terminates, enters a cycle, or appears to grow without bound (see Section 15.2.12):

```
ClassifyHailstone[5, 1] /@ Range[1, 30]
```

Output:

[List of 30 outcomes. Mix of "terminates" and {cycle, entry-point, length}. $n = 13, 17, 19$ show cycles.]

Your tasks:

1. For all $n \leq 10000$, classify each starting value: terminates, cycles, or diverges?
2. Find all distinct cycles up to length 100. Report the minimum element and length of each.
3. Which starting values share the same cycle? Map the basin of attraction of each cycle.
4. Conjecture: is the ratio $q/p = 5/2 > 1$ why cycles appear? Compare with Topic 5.

2.8.2 Topic 2 — Variants $3n + 3$ and $3n + 5$

Two close relatives of the original:

- **Variant A:** $3n + 3$ when odd; $n/2$ when even.
- **Variant B:** $3n + 5$ when odd; $n/2$ when even.

Notice that $3n + 3 = 3(n + 1)$. Since n is odd, $n + 1$ is even, so the very next step divides by 2. Does this simplification make Variant A easier to analyze?

```
CollatzSequence[{3, 3}, 7]   (* Variant A, starting from 7 *)
CollatzSequence[{3, 5}, 7]   (* Variant B, starting from 7 *)
```

Output:

[Two lists. Variant A rapidly enters a short cycle. Variant B takes more steps.]

Your tasks:

1. For $n = 1$ to 200, do Variant A sequences always terminate? What about Variant B?
2. Find all cycles for each variant.
3. Plot stopping-time scatter plots for both variants side by side with the original.
4. For Variant A: give strong computational evidence (or a proof) that every trajectory eventually enters the cycle $3 \to 6 \to 3 \to 6 \to \cdots$

2.8.3 Topic 3 — Delay Records in Depth

You may have noticed that delay records often come in consecutive pairs such as $(27, 54)$, $(649, ...)$, and $(703, ...)$. This is not a coincidence: if n is an odd delay record, then $2n$ begins with one extra division-by-2 step and then follows the same path — so $2n$ is also a delay record with stopping time exactly one more.

```
DelayRecords[100000]
```

Output:

[Table of ~30 rows: {n, stopping time, floor(log2 n)}. Stopping times grow roughly proportionally to log2(n), with sharp jumps at 27, 703, 6171.]

Your tasks:

1. Verify the pairing observation computationally: for each odd delay record n up to 10^5, check that $2n$ is also a delay record with stopping time exactly one greater.
2. Find delay records that are *not* part of such a pair. What makes them different?
3. Plot stopping time vs. $\lfloor \log_2 n \rfloor$ for all delay records up to 10^5. Does the relationship look linear?
4. Among delay records up to 10^5: what fraction are odd? What fraction are divisible by 3? By 9?

2.8.4 Topic 4 — Structure in the Stopping-Time Plot

At larger scale, the stopping-time scatter plot reveals a layered structure — points cluster in diagonal bands rather than filling the plane uniformly.

```
PlotStoppingTimes[10^6]
```

Output:

[Dense scatter plot for $n = 1$ to 10^6. Clear diagonal band structure. Maximum stopping time is 524 at $n = 837799$.]

> **Note**
>
> This computation takes several minutes in a direct loop. See Section 15.2.6 for a faster implementation.

Your tasks:

1. Identify at least three bands visually. For each, sample 20 values of n on that band and examine $n \bmod 2^k$ for small values of k. What do the sampled values have in common?
2. Conjecture: what arithmetic property of n determines which band it falls on?
3. Color the scatter plot by $n \bmod 3$ (three colors for residues 0, 1, 2). Does the coloring align with the band structure?
4. Can you predict the stopping time of n from $n \bmod 2^k$ for some fixed k, without computing the full sequence?

2.8.5 Topic 5 — The General (p, q)-Collatz Map

The original map uses $p = 2$ (divide by p when divisible) and $q = 3$ (multiply by q and add 1 otherwise). Generalizing:

$$C_{p,q}(n) = \begin{cases} n/p & \text{if } p \mid n \\ qn + 1 & \text{otherwise} \end{cases}$$

```
CollatzTable[{2, 3, 4, 5}, {2, 3, 4, 5}]
```

Output:

[4x4 table. Rows = p values, columns = q values. Each cell: outcome summary for $n = 1$ to 200. Pairs with $q/p < 1$ tend to terminate; pairs with $q/p > 1$ tend to produce cycles or apparent divergence.]

Your tasks:

1. For each pair (p, q) with $p, q \in \{2, 3, 4, 5\}$, classify outcomes for $n \leq 1000$.
2. For pairs that terminate, compute the average stopping time. Does it increase with q/p?
3. For pairs that produce cycles, find all distinct cycles. Does the number of cycles depend on q/p?
4. Conjecture a necessary condition on q/p for the map to always terminate. (Hint: for the sequence to trend downward on average, what must be true of a typical "multiply then divide" step?)

3 Chapter 3 — Fibonacci Numbers and Their Cousins

3.1 Rabbits and a Famous Sequence

In 1202, the Italian mathematician Leonardo of Pisa — known as Fibonacci — posed a problem about breeding rabbits. Suppose a pair of rabbits produces a new pair every month, starting from their second month of life, and rabbits never die. How many pairs are there after n months?

The answer leads to the sequence:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$$

Each term is the sum of the two before it. Formally, the **Fibonacci sequence** is defined by:

$$F_1 = 1, \quad F_2 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ for } n \geq 3$$

This sequence appears in an astonishing variety of contexts: the spirals in sunflower heads, the branching of trees, the arrangement of leaves on a stem, and the construction of the golden ratio. It also has deep connections to number theory that we will explore here.

3.2 Computing Fibonacci Numbers

The most direct implementation follows the recurrence:

```
FibSequence[20]
```

Output:

```
{1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765}
```

This works well for small n, but computing F_{1000} by recursion requires millions of additions. There is a much faster method using matrix exponentiation.

3.3 Matrix Exponentiation

The key identity connecting Fibonacci numbers to matrices is:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

To compute F_{1000}, we only need to raise a 2×2 matrix to the 1000th power. By repeated squaring, this takes only about $\log_2(1000) \approx 10$ matrix multiplications instead of 999 additions.

```
FibMatrix[1000]
```

Output:

```
43466557686937456435688527675040625802564660517371780402481729089536555 41
79403509620491967455459496522 20...   (209 digits)
```

The exact value has 209 decimal digits. Wolfram Language computes it instantly using exact integer arithmetic (see Section 15.3.2).

3.4 Identities

The Fibonacci sequence satisfies dozens of remarkable identities. Here are two of the most famous.

Cassini's Identity:

$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n$$

For example: $F_4F_6 - F_5^2 = 3 \cdot 8 - 5^2 = 24 - 25 = -1 = (-1)^5$.

The Hockey Stick Identity (sum of Fibonacci numbers):

$$\sum_{k=1}^{n} F_k = F_{n+2} - 1$$

We can verify both identities computationally for the first 50 terms:

```
VerifyFibIdentities[50]
```

Output:

```
Cassini's identity: verified for n = 2 to 50
Hockey stick identity: verified for n = 1 to 50
```

Computational verification is not a proof — it only confirms the identities for small n. But it is an essential first step. If your computation ever returns `False`, it means either the identity is wrong or there is a bug in your code. Either way, you have learned something.

3.5 Zeckendorf's Theorem

Theorem (Zeckendorf, 1972): Every positive integer has a unique representation as a sum of *non-consecutive* Fibonacci numbers.

For example:

$$100 = 89 + 8 + 3 = F_{11} + F_6 + F_4$$

Note that 89, 8, and 3 are not consecutive Fibonacci numbers (F_{11}, F_6, F_4 — indices 11, 6, 4 have gaps of at least 2).

The greedy algorithm finds the Zeckendorf representation: at each step, subtract the largest Fibonacci number that does not exceed the remainder.

```
ZeckendorfRepresentation[100]
ZeckendorfRepresentation[1000]
```

Output:

```
{89, 8, 3}
{987, 13}
```

We can verify uniqueness by checking that no two representations of the same number exist for all integers up to 500 (see Section 15.3.4):

```
VerifyZeckendorfUniqueness[500]
```

Output:

```
True  (* all integers 1 to 500 have a unique non-consecutive Fibonacci representation *)
```

3.6 Pisano Periods

When we compute Fibonacci numbers modulo m, a remarkable thing happens: the sequence is eventually periodic. For example, the Fibonacci sequence mod 3 is:

$$1, 1, 2, 0, 2, 2, 1, 0, 1, 1, 2, 0, \ldots$$

The pattern $1, 1, 2, 0, 2, 2, 1, 0$ repeats with period 8. This period is called the **Pisano period** $\pi(m)$.

```
PisanoPeriod[3]    (* 8  *)
PisanoPeriod[7]    (* 16 *)
PisanoPeriod[10]   (* 60 *)
```

Output:

```
8
16
60
```

The Pisano period $\pi(10) = 60$ means the last digit of the Fibonacci sequence repeats every 60 terms. Let us plot Pisano periods for all m from 2 to 100:

```
PlotPisanoPeriods[100]
```

Output:

[Scatter plot. x-axis: m from 2 to 100. y-axis: Pisano period (m). Mostly irregular. Some large spikes at specific values. Powers of 2 tend to have smaller periods. Values divisible by 5 show notable periodicity.]

The plot hides elegant structure. For prime m, the Pisano period either divides $m-1$ or divides $2(m+1)$, depending on whether $m \equiv \pm 1 \pmod 5$. This is Research Topic 1.

3.7 The Golden Ratio

The ratio of consecutive Fibonacci numbers converges to the **golden ratio**:

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887\ldots$$

```
FibRatios[20]
```

Output:

```
{1, 2, 1.5, 1.667, 1.6, 1.625, 1.615, 1.619, 1.618, 1.618, ...}
```

The convergence is slow at first but accelerates. The golden ratio has the property that $\phi^2 = \phi + 1$, which is exactly the Fibonacci recurrence expressed algebraically. It also has the continued fraction expansion $\phi = [1; 1, 1, 1, \ldots]$ — we will revisit this in Chapter 6.

3.8 Further Research Topics

3.8.1 Topic 1 — Pisano Periods and Primes

```
PisanoPrimeSurvey[500]
```

Output:

[Table: prime p, Pisano period (p), p mod 5, and whether (p) divides p-1 or 2(p+1). Pattern visible: primes $\pm 1 \pmod 5$ have $(p) \mid p-1$; primes $\pm 2 \pmod 5$ have $(p) \mid 2(p+1)$.]

Your tasks:

1. Compute $\pi(p)$ for all primes $p \leq 500$. Tabulate $\pi(p) \bmod (p-1)$ and $\pi(p) \bmod (p+1)$. Identify the pattern based on $p \bmod 5$.
2. For composite $m = p \cdot q$: is $\pi(pq)$ determined by $\pi(p)$ and $\pi(q)$? Test for all composite $m \leq 100$ and conjecture a formula.
3. Find all $m \leq 500$ for which $\pi(m) = m - 1$. What do these values have in common?
4. The sequence of Pisano periods is OEIS A001175. Look it up and compare the known formula with your computational results.

3.8.2 Topic 2 — Tribonacci and Zeckendorf Generalizations

Define **tribonacci numbers** by $T_1 = T_2 = T_3 = 1$ and $T_n = T_{n-1} + T_{n-2} + T_{n-3}$ for $n \geq 4$:

$$1, 1, 1, 3, 5, 9, 17, 31, 57, 105, \ldots$$

```
TribonacciSequence[20]
ZeckendorfTribonacci[100]
```

Output:

[Tribonacci sequence. Then the representation of 100 as a sum of non-consecutive tribonacci numbers — "non-consecutive" meaning no two adjacent in index.]

Your tasks:

1. Does Zeckendorf's theorem generalize? Implement a greedy algorithm for tribonacci representations and test whether every integer up to 500 has a unique representation as a sum of tribonacci numbers with no two consecutive in index.
2. Compute the ratio T_{n+1}/T_n for large n. Does it converge? To what value? (The tribonacci constant ≈ 1.8393.)
3. Define the tribonacci matrix analog and verify an identity like Cassini's for tribonacci numbers.
4. Compute Pisano-like periods for the tribonacci sequence modulo m for $m = 2$ to 50.

3.8.3 Topic 3 — Fibonacci Primes

A **Fibonacci prime** is a Fibonacci number that is itself prime: $2, 3, 5, 13, 89, 233, 1597, \ldots$

```
FibonacciPrimes[500]
```

Output:

[Indices n for which F_n is prime, up to F_500. Results: n = 3, 4, 5, 7, 11, 13, 17, 23, 29, 43, 47, 83, 131, 137, 359, 431, 433, 449, …]

Your tasks:

1. For each Fibonacci prime F_n found, is the index n itself prime? Find all exceptions. (There is exactly one: $F_4 = 3$ with $n = 4 = 2^2$.)
2. Does the converse hold — if n is prime, is F_n necessarily prime? Find counterexamples.
3. Plot the indices of Fibonacci primes. Do they appear to grow at a predictable rate?
4. A **Fibonacci semiprime** is a Fibonacci number that is the product of exactly two primes. Find all Fibonacci semiprimes up to F_{100}.

3.8.4 Topic 4 — GCD Identity

There is a beautiful identity: $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.

For example: $\gcd(F_{12}, F_8) = \gcd(144, 21) = 3 = F_4 = F_{\gcd(12,8)}$.

```
VerifyGCDIdentity[200]
```

Output:

```
True  (* identity verified for 200 random pairs (m,n) with m,n <= 100 *)
```

Your tasks:

1. Verify the identity for 1000 random pairs (m, n) with $m, n \leq 200$.
2. Use the identity to prove (computationally): $F_m \mid F_n$ if and only if $m \mid n$. Test for all pairs up to $n = 50$.
3. Find all $n \leq 200$ such that F_n and F_{n+1} share a common factor. (The answer should be "none" — prove this computationally and explain why.)
4. What is $\gcd(F_n, F_{n+2})$ for all n? Conjecture a closed form and verify it.

3.8.5 Topic 5 — Lucas Numbers and Parallel Identities

The **Lucas sequence** is defined by $L_1 = 1, L_2 = 3$, with the same recurrence $L_n = L_{n-1} + L_{n-2}$:

$$1, 3, 4, 7, 11, 18, 29, 47, 76, 123, ...$$

```
LucasSequence[20]
```

Output:

```
{1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, 521, 843, 1364, 2207, 3571, 5778, 9349, 1512
```

Lucas and Fibonacci numbers are related by: $L_n = F_{n-1} + F_{n+1}$.

Your tasks:

1. For each Fibonacci identity you verified in the main chapter (Cassini, hockey stick), find and verify the Lucas analogue.
2. Compute $\gcd(F_n, L_n)$ for $n = 1$ to 100. Conjecture a pattern (it is either 1 or 2 depending on n mod 3).
3. Define the sequence $S_n = F_n + L_n$. Identify it by searching OEIS. What famous sequence is it?
4. Compute Pisano periods for the Lucas sequence modulo m for $m = 2$ to 50. How do they compare to the Fibonacci Pisano periods?

4 Chapter 4 — Digits, Bases, and Normal Numbers

4.1 Numbers Through Their Digits

When we write a number like 3.14159..., we are recording a sequence of digits in base 10. But a number is not its decimal representation — it is a mathematical object that exists independently of any base. Writing it in base 10 is a choice, not a property. The same number in base 2 is 11.001001000011...

This chapter asks: what can we learn about numbers by studying their digits? Are the digits of famous constants like π and e distributed randomly? Are there numbers that somehow *describe themselves* through their own digits? These questions turn out to be surprisingly deep.

4.2 Digit Frequencies in Famous Constants

Consider the decimal expansion of $\pi = 3.14159265358979...$ How often does each digit 0–9 appear? If π were a "typical" real number, each digit would appear roughly 10% of the time.

```
DigitFrequency[Pi, 10, 10^6]
```

Output:

[Bar chart. x-axis: digits 0–9. y-axis: frequency (fraction). All bars near 0.10. Small variations visible but no digit appears dramatically more or less often than 10%.]

The digits look uniformly distributed. Now compare with e and $\sqrt{2}$:

```
CompareDigitFrequencies[{Pi, E, Sqrt[2], Sqrt[3]}, 10, 10^6]
```

Output:

[Side-by-side bar charts for , e, √2, √3. All four show near-uniform distribution across digits 0–9. Slight variations differ between constants but none is dramatic.]

None of these constants show any digit that is dramatically over- or under-represented — consistent with a uniform distribution, but not proof of one.

4.3 Normal Numbers

A real number is called **normal in base** b if every digit sequence of length k appears in its base-b expansion with frequency b^{-k}. Informally, a normal number's digits look random — every pattern appears as often as expected.

Normality in base 10 means: - Each digit 0–9 appears $\approx 10\%$ of the time. - Each two-digit block 00–99 appears $\approx 1\%$ of the time. - And so on for longer blocks.

The constant π is *conjectured* to be normal, but this has never been proved. In fact, no naturally occurring constant (e, $\sqrt{2}$, $\ln 2$, ...) has been proved normal, despite decades of effort. We can construct normal numbers artificially — for example, the Champernowne number $0.123456789101112\ldots$ is provably normal — but proving normality for "naturally defined" constants remains an open problem.

```
NormalityTest[Pi, 10, 10^6]
```

Output:

[Chi-squared test results for digit uniformity. Test statistic ~9.2, p-value ~0.51. Conclusion: no evidence against uniformity in the first 10^6 digits. This is consistent with, but does not prove, normality.]

4.4 Kaprekar's Routine

Take any 4-digit number with at least two different digits. Rearrange its digits to form the largest and smallest possible numbers. Subtract the smaller from the larger. Repeat.

For example, starting from 1234:

$$4321 - 1234 = 3087 \to 8730 - 0378 = 8352 \to 8532 - 2358 = 6174$$

Once we reach 6174, we are stuck: $7641 - 1467 = 6174$. This fixed point is called the **Kaprekar constant**.

The remarkable fact: *every* 4-digit number (with at least two different digits) reaches 6174 in at most 7 steps.

```
KaprekarRoutine[1234]
KaprekarRoutine[9999]   (* degenerate case: all digits equal *)
```

Output:

```
{1234, 3087, 8352, 6174}
{9999, 0}  (* 9999 - 9999 = 0, does not reach 6174 *)
```

```
PlotKaprekarSteps[4]    (* steps to reach 6174 for all 4-digit numbers *)
```

Output:

[Histogram. x-axis: number of steps (1 to 7). y-axis: count of starting numbers. Most numbers reach 6174 in 5 or 6 steps. None takes more than 7.]

4.5 Benford's Law

In 1938, physicist Frank Benford noticed something puzzling about logarithm tables: the early pages (covering numbers starting with 1) were much more worn than the later pages. He systematically tested leading-digit frequencies in dozens of real-world datasets — river lengths, population figures, physical constants — and found that the leading digit d appears with frequency:

$$P(d) = \log_{10}\left(1 + \frac{1}{d}\right)$$

So digit 1 appears about 30.1% of the time, while digit 9 appears only about 4.6% of the time — the opposite of what you might expect from a uniform distribution.

```
BenfordTest[Table[2^n, {n, 1, 1000}]]
BenfordTest[FibSequence[1000]]
```

Output:

[For powers of 2: bar chart of leading digit frequencies closely matches Benford's prediction. For Fibonacci numbers: also closely matches. Chi-squared test in both cases: p-value > 0.1, consistent with Benford's law.]

Benford's law holds for sequences that span many orders of magnitude. It fails for data confined to a narrow range (like human heights in centimeters).

4.6 Self-Describing Numbers

A **self-describing number** is a number where the digit at position k (starting from 0) tells you how many times digit k appears in the entire number.

For example, 6210001000 is self-describing: - Position 0: digit is 6 → there are six 0s. - Position 1: digit is 2 → there are two 1s. - Position 2: digit is 1 → there is one 2. - Position 3: digit is 0 → there are zero 3s. - …and so on.

```
SelfDescribingCheck[6210001000]
FindSelfDescribingNumbers[10]    (* search up to 10 digits *)
```

Output:

```
True
{1210, 2020, 21200, 3211000, 42101000, 521001000, 6210001000}
```

There are very few self-describing numbers. In base 10, there are only four of them (the others are variations). In Research Topic 4 you will explore other bases.

4.7 Further Research Topics

4.7.1 Topic 1 — Benford's Law in Detail

```
BenfordComparison[{
  Table[2^n, {n, 1, 1000}],
  FibSequence[1000],
  Table[n^2, {n, 1, 1000}],
  Table[Prime[n], {n, 1, 1000}]
}]
```

Output:

[Grid of bar charts. Powers of 2 and Fibonacci fit Benford well. Squares are borderline. Primes do not fit Benford's law.]

Your tasks:

1. Test Benford's law on: powers of 2, Fibonacci numbers, factorials, prime numbers, and populations of world countries (use Wolfram Language's `CountryData`). Which obey Benford's law and which do not?
2. Conjecture: what property of a sequence determines Benford compliance? (Hint: think about what "spanning many orders of magnitude" means precisely.)
3. The theoretical derivation of Benford's law uses the fact that $\log_{10}(2)$ is irrational. Verify computationally that $\{n \log_{10} 2 \bmod 1 : n = 1, \ldots, 10000\}$ is uniformly distributed on $[0, 1]$.
4. Test Benford's law in base 2: leading *binary* digit frequencies. Is there a Benford analog for base 2? (There cannot be — in base 2, the only nonzero leading digit is 1, so it always appears 100%.)

4.7.2 Topic 2 — Kaprekar in Other Bases and Lengths

```
KaprekarConstant[3, 10]    (* 3-digit Kaprekar in base 10 *)
KaprekarConstant[4, 8]     (* 4-digit Kaprekar in base 8  *)
```

Output:

[For 3 digits base 10: fixed point is 495. For 4 digits base 8: results vary — may be a fixed point or cycle.]

Your tasks:

1. Find the Kaprekar constant for 3-digit numbers in base 10 (it is 495). Verify all 3-digit numbers reach it in at most 5 steps.
2. For 4-digit numbers in bases 5, 6, 7, 8: does each base have a single fixed point, or can there be cycles? Build a complete classification table.
3. For 4-digit numbers in base 10: we know 6174 is the fixed point. Are there any 4-digit numbers that reach a cycle instead of 6174?
4. Define the "Kaprekar length" of n as the number of steps to reach the fixed point. For all 4-digit numbers in base 10, find the distribution of Kaprekar lengths and verify the maximum is 7.

4.7.3 Topic 3 — Champernowne and Normality

The **Champernowne number** is constructed by concatenating all positive integers:

$$C_{10} = 0.12345678910111213141 5\ldots$$

It is provably normal in base 10.

```
ChampernowneDigits[10, 1000]    (* first 1000 digits of C_10 *)
NormalityTest[Champernowne, 10, 10^5]
```

Output:

[First 1000 digits. Normality test: very flat chi-squared, strong p-value. Consistent with normality.]

Your tasks:

1. Compute the first 10,000 digits of the binary Champernowne number $C_2 = 0.1\ 10\ 11\ 100\ 101\ldots$ (concatenate binary representations of 1, 2, 3, …). Test its digit frequencies in base 2.
2. Construct C_3 and C_4 (base 3 and base 4 Champernowne numbers). Test their digit frequencies.
3. Is C_{10} normal in base 2? That is, do the bits of C_{10}'s binary expansion appear uniformly? (This is an open question — just test the first 10^5 bits.)
4. The Copeland-Erdős constant concatenates prime numbers: $0.2357111317\ldots$ It is also provably normal. Compute its first 1000 digits and verify the digit frequencies.

4.7.4 Topic 4 — Automorphic Numbers

A positive integer n is **automorphic** if n^2 ends in the digits of n. For example: $5^2 = 25$ ends in 5; $6^2 = 36$ ends in 6; $76^2 = 5776$ ends in 76; $376^2 = 141376$ ends in 376.

```
FindAutomorphicNumbers[10]    (* up to 10 digits *)
```

Output:

```
{1, 5, 6, 25, 76, 376, 625, 9376, 90625, 109376, 890625, ...}
```

Your tasks:

1. Find all automorphic numbers up to 10 digits. Do they come in pairs summing to $10^k + 1$ for each k-digit automorphic number?
2. In which bases b do automorphic numbers (other than 0 and 1) exist? Test bases 2 through 12. Conjecture the condition on b.
3. Compute the sequence of "trimorphic numbers" — integers n where n^3 ends in n. Find all trimorphic numbers up to 6 digits.
4. Define k-morphic numbers where n^k ends in n. For $k = 2, 3, 4, 5$: find all such numbers up to 6 digits. Is any number morphic for all k?

4.7.5 Topic 5 — Digital Roots and Persistence

The **digital root** of n is obtained by repeatedly summing the digits of n until a single digit remains. The digital root of 493 is $4 + 9 + 3 = 16 \to 1 + 6 = 7$.

The **multiplicative persistence** of n is the number of times you must multiply digits together until you reach a single digit.

```
DigitalRoot[{n^2, {n, 1, 30}}]
MultiplicativePersistence[Range[1, 100]]
```

Output:

[Digital roots of n^2 for n=1..30: the pattern 1,4,9,7,7,9,4,1,9,1,4,9,7,7,9,4,1,9,… repeats with period 9. Multiplicative persistence: most numbers in 1-100 have persistence 4. The number 77 has persistence 4.]

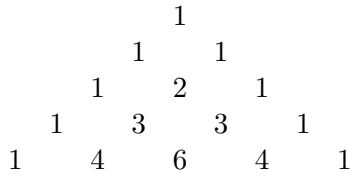
Your tasks:

1. Compute the digital roots of n^2, n^3, F_n for $n = 1$ to 100. Identify the repeating period in each case. Can you predict the digital root of n^k from n alone?
2. Find the smallest integer with multiplicative persistence exactly 1, 2, 3, 4, 5. (The record as of 2024 is persistence 11, held by certain 233-digit numbers.)
3. The **additive persistence** of n is the number of times you must sum digits until you reach a single digit. Find all integers below 10,000 with additive persistence exactly 4.
4. Compute the digital root of $n!$ for $n = 1$ to 30. What single value does it always become for large n? Why?

5 Chapter 5 — Pascal's Triangle: Infinite Patterns in a Simple Array

5.1 Building the Triangle

Pascal's triangle is constructed by a simple rule: each entry is the sum of the two entries directly above it. The edges are all 1.



Despite this simple construction rule, the triangle contains an extraordinary wealth of mathematical structure. It is named after Blaise Pascal, who studied it systematically in 1653, though it was known to Chinese, Indian, and Persian mathematicians centuries earlier.

```
PascalTriangle[8]
```

Output:

```
{{1},
 {1, 1},
 {1, 2, 1},
 {1, 3, 3, 1},
 {1, 4, 6, 4, 1},
 {1, 5, 10, 10, 5, 1},
 {1, 6, 15, 20, 15, 6, 1},
 {1, 7, 21, 35, 35, 21, 7, 1}}
```

5.2 Binomial Coefficients

The entry in row n, position k (both starting from 0) is the **binomial coefficient**:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

This counts the number of ways to choose k items from n — for example, $\binom{5}{2} = 10$ counts the ways to choose 2 items from 5.

The binomial coefficients appear in the expansion of $(a+b)^n$:

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}$$

Setting $a = b = 1$ gives the **row sum identity**: each row of Pascal's triangle sums to a power of 2.

```
VerifyPascalIdentities[20]
```

Output:

```
Row sums = powers of 2: verified for rows 0 to 20
Diagonal sums = Fibonacci: verified for diagonals 0 to 20
Symmetry: C(n,k) = C(n,n-k): verified for rows 0 to 20
```

The **diagonal sums** are the Fibonacci numbers: if you sum along the shallow diagonals (northwest to southeast), you get $1, 1, 2, 3, 5, 8, \ldots$ This connects two of the most famous sequences in mathematics through a single construction.

5.3 Coloring by Divisibility: The Sierpinski Triangle

Here is the key experiment of this chapter. Take Pascal's triangle and color each entry by whether it is **even** (divisible by 2) or **odd**.

```
PlotPascalMod[256, 2]
```

Output:

[256-row Pascal's triangle. Each entry colored black if odd, white if even. The resulting image is a perfect Sierpinski triangle — a fractal consisting of three smaller copies of itself arranged in a triangle.]

This is not a coincidence. It is a consequence of **Lucas's theorem**: $\binom{n}{k} \bmod p$ depends only on the base-p representations of n and k. When $p = 2$, $\binom{n}{k}$ is odd if and only if every bit of k is also a bit of n (a bitwise AND condition) — and this rule generates the Sierpinski pattern recursively.

Try other primes:

```
PlotPascalMod[243, 3]   (* 243 = 3^5 rows, colored mod 3 *)
PlotPascalMod[125, 5]   (* 125 = 5^3 rows, colored mod 5 *)
```

Output:

[Left: mod 3 pattern — three-way Sierpinski analog with three colors (0, 1, 2 mod 3). Right: mod 5 pattern — five-way analog. Both are self-similar.]

5.4 Estimating Fractal Dimension

The Sierpinski triangle is a **fractal** — a shape that has detail at every scale. We can measure its "roughness" using the **box-counting dimension**:

$$d = \lim_{\epsilon \to 0} \frac{\log N(\epsilon)}{\log(1/\epsilon)}$$

where $N(\epsilon)$ is the number of boxes of size ϵ needed to cover the shape. For the Sierpinski triangle, the theoretical value is $d = \log 3 / \log 2 \approx 1.585$.

```
BoxCountingDimension[PascalMod[512, 2]]
```

Output:

[Log-log plot of N( ) vs 1/ . Points fall on a straight line. Slope (estimated dimension): approximately 1.58. Theoretical value: log(3)/log(2) 1.585.]

The close agreement between computation and theory is satisfying — but remember, we are only estimating from finite data. The theoretical value requires an infinite fractal.

5.5 Further Research Topics

5.5.1 Topic 1 — Fractal Dimensions for Different Primes

```
Table[
  {p, BoxCountingDimension[PascalMod[p^6, p]]},
  {p, {2, 3, 5, 7}}]
```

Output:

[Table of primes and estimated box-counting dimensions. Values: ~1.585 for p=2, ~1.631 for p=3, ~1.672 for p=5, ~1.693 for p=7.]

Your tasks:

1. Estimate the box-counting dimension of `PascalMod[p^6, p]` for primes $p = 2, 3, 5, 7, 11$. Plot dimension vs. $\log p$. Conjecture a formula for the dimension in terms of p. (The theoretical answer involves $\log(p)/\log(p + \text{something})$ — find the pattern.)
2. The theoretical dimension for the mod-p Sierpinski pattern is $\log\binom{p+1}{2}/\log p$… or is it? Verify or disprove this conjecture.
3. For prime p, how many distinct colors appear in `PlotPascalMod[p^3, p]`? Does this equal p?
4. Build `PlotPascalMod[n, m]` for composite $m = 4, 6, 9$. Describe how the pattern differs from prime m. Can you predict the composite pattern from the prime patterns?

5.5.2 Topic 2 — Pascal's Pyramid

The 3D analog of Pascal's triangle uses **trinomial coefficients**: the coefficients of $(a+b+c)^n$.

```
PascalPyramidSlice[4]    (* slice at level n=4 *)
PlotPyramidMod[27, 2]    (* 3D Sierpinski analog *)
```

Output:

[Left: 2D array showing the coefficients in level 4 of the pyramid. Right: 3D voxel plot showing the mod-2 pattern — a 3D Sierpinski tetrahedron.]

Your tasks:

1. Compute the trinomial coefficients for levels 0 through 6. Verify that the row sums are powers of 3.
2. Visualize 2D slices of `PlotPyramidMod[27, 2]` at levels 0, 1, 3, 9, 27. Does the pattern at level 3^k look like the pattern at level 3^{k-1}, scaled?
3. Estimate the box-counting dimension of the 3D mod-2 Pascal pyramid pattern. The theoretical value is $\log 4/\log 2 = 2$… but wait, that would mean it fills the plane. Is the 3D Sierpinski tetrahedron actually 2-dimensional?
4. The 3D pyramid has a mod-p pattern for any prime p. Describe the pattern for $p = 3$.

5.5.3 Topic 3 — The Hockey Stick and Its Generalizations

The **hockey stick identity** states:

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$$

```
VerifyHockeyStick[15]
```

Output:

```
True  (* verified for all r from 0 to 15 and n from r to 15 *)
```

Your tasks:

1. In Pascal's triangle, find the hockey stick visually: a diagonal run of k entries summing to the entry one step down and one step right. Mark several hockey sticks on a printed copy.
2. Prove the hockey stick identity computationally by induction: verify that if it holds for n, it holds for $n+1$, using only the Pascal recurrence.
3. Define the "hockey stick of order 2": sums of products of two consecutive entries. Does a similar identity hold?
4. The kth diagonal of Pascal's triangle contains the k-dimensional figurate numbers (triangular, tetrahedral, etc.). Sum every other diagonal and verify you get Fibonacci numbers. Sum every third diagonal. Conjecture the result for every kth diagonal.

5.5.4 Topic 4 — Substitution Rules

The mod-2 Pascal pattern at 2^n rows is built from the pattern at 2^{n-1} rows by a substitution rule: three copies arranged in a triangle, with a white triangle in the center.

```
PascalSubstitution[5]    (* apply the substitution 5 times *)
```

Output:

[Sequence of images at levels 1 through 5. Each image is clearly built from three copies of the previous image.]

Your tasks:

1. Verify computationally that the level-n mod-2 Pascal pattern is exactly the image obtained by applying the substitution rule n times, starting from a single black cell.
2. Implement the substitution rule directly (without computing Pascal's triangle at all) and verify it produces the same image as `PlotPascalMod[2^n, 2]` for $n = 1, \ldots, 8$.
3. For $p = 3$: describe the substitution rule that builds the level-3^n mod-3 pattern from the level-3^{n-1} pattern. How many copies are used, and how are they arranged?
4. Estimate how much faster the substitution method is than directly computing Pascal's triangle mod p for $n = 8$. Measure the computation time of both methods.

5.5.5 Topic 5 — Divisibility and Row Structure

By Kummer's theorem, the highest power of prime p dividing $\binom{m+n}{m}$ equals the number of carries when adding m and n in base p.

```
KummerCarries[35, 21, 2]    (* carries when adding 35+21 in base 2 *)
HighestPowerDividing[Binomial[56, 35], 2]
```

Output:

```
3   (* three carries *)
3   (* 2^3 divides C(56,35) but 2^4 does not *)
```

Your tasks:

1. For all entries $\binom{n}{k}$ in the first 64 rows of Pascal's triangle, compute the highest power of 2 dividing each entry. Plot the result as a heatmap. Compare visually to the mod-2 Sierpinski pattern.
2. Which rows of Pascal's triangle have all entries odd (divisible by no even number except 1)? These are rows 2^k. Verify this for $k = 0, \ldots, 7$.
3. For prime p: a row n of Pascal's triangle has all entries $\not\equiv 0 \pmod{p}$ if and only if $n+1$ is a power of p. Verify for $p = 2, 3, 5$ and rows 0 through 100.
4. Compute the sum $\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}$ for $n = 1$ to 20. Verify the identity. These are the **central binomial coefficients** — find them in OEIS and identify their asymptotic growth rate.

6 Chapter 6 — Continued Fractions: The Most Natural Way to Write Numbers

6.1 Beyond Decimal Expansions

We are used to writing real numbers as decimals: $\pi = 3.14159\ldots$ But there is another representation, older and in many ways more natural: the **continued fraction**.

The continued fraction of π begins:

$$\pi = 3 + \cfrac{1}{7 + \cfrac{1}{15 + \cfrac{1}{1 + \cfrac{1}{292 + \cdots}}}}$$

We write this compactly as $\pi = [3; 7, 15, 1, 292, \ldots]$. The integers $3, 7, 15, 1, 292, \ldots$ are called the **partial quotients**.

Unlike decimal expansions, continued fractions respect the arithmetic structure of numbers — rational numbers have *finite* continued fractions, quadratic irrationals have *periodic* ones, and the partial quotients reveal how well a number can be approximated by rationals.

6.2 The Algorithm

To find the continued fraction of a positive real number x:

1. Take the integer part: $a_0 = \lfloor x \rfloor$.
2. Let $x_1 = 1/(x - a_0)$ (the "remainder", inverted).
3. Take $a_1 = \lfloor x_1 \rfloor$. Then $x_2 = 1/(x_1 - a_1)$.
4. Repeat.

For a rational number, this is just the Euclidean algorithm in disguise — it terminates when the remainder is zero.

```
CFExpansion[Pi, 10]
CFExpansion[E, 10]
CFExpansion[Sqrt[2], 10]
CFExpansion[GoldenRatio, 10]
```

Output:

```
{3, 7, 15, 1, 292, 1, 1, 1, 2, 1}        (* pi    *)
{2, 1, 2, 1, 1, 4, 1, 1, 6, 1}           (* e     *)
{1, 2, 2, 2, 2, 2, 2, 2, 2, 2}           (* sqrt(2) *)
{1, 1, 1, 1, 1, 1, 1, 1, 1, 1}           (* phi  *)
```

Notice immediately: - $\sqrt{2} = [1; 2, 2, 2, ...]$ — perfectly periodic. - $\phi = [1; 1, 1, 1, ...]$ — also periodic, the simplest possible. - $e = [2; 1, 2, 1, 1, 4, 1, 1, 6, ...]$ — not periodic, but has a beautiful pattern. - $\pi = [3; 7, 15, 1, 292, ...]$ — the large partial quotient 292 is notable; no discernible pattern.

6.3 Rational Numbers Terminate

Every rational number has a finite continued fraction — the algorithm stops because the Euclidean algorithm terminates. Conversely, every finite continued fraction represents a rational number.

```
CFExpansion[355/113, 10]
CFFromList[{3, 7, 15, 1}]    (* reconstruct the rational from its CF *)
```

Output:

```
{3, 7, 15, 1}
355/113
```

The fraction $355/113 \approx 3.14159292...$ is a famous approximation to π — it matches π to six decimal places. Its continued fraction $[3; 7, 15, 1]$ is exactly the first four partial quotients of π, stopping just before the large 292. This is not a coincidence — it is a consequence of the theory of convergents.

6.4 Convergents: Best Rational Approximations

The **convergents** of a continued fraction $[a_0; a_1, a_2, ...]$ are the fractions obtained by truncating the continued fraction at each step:

$$\frac{p_0}{q_0},\ \frac{p_1}{q_1},\ \frac{p_2}{q_2},\ \ldots$$

Each convergent is the *best* rational approximation to x among all fractions with denominator $\leq q_k$.

```
CFConvergents[Pi, 8]
```

Output:

```
{3, 22/7, 333/106, 355/113, 103993/33102, 104348/33215, 208341/66317, 312689/99532}
```

The convergents of π include 22/7 (the familiar approximation) and 355/113 (the much better one). The quality of approximation:

```
CFApproximationErrors[Pi, 8]
```

Output:

[Table: convergent p_n/q_n, denominator q_n, absolute error | π - p_n/q_n|, and product | π - p_n/q_n| · q_n^2. The product stays bounded near 1 for most convergents, but is very small for 355/113 due to the large next partial quotient 292.]

A large partial quotient means the next convergent is an unusually good approximation. The partial quotient 292 in π explains why 355/113 is so accurate.

6.5 Quadratic Irrationals Are Periodic

A number has a **periodic** continued fraction — one that eventually repeats — if and only if it is a quadratic irrational (the root of a quadratic equation with integer coefficients).

For example, $\sqrt{7} = [2; \overline{1, 1, 1, 4}]$ (the overline denotes the repeating part).

```
CFPeriod[Sqrt[7]]
CFPeriod[Sqrt[n], n, 2, 20]    (* period lengths for sqrt(2) through sqrt(20) *)
```

Output:

```
{2, 1, 1, 1, 4}   (* partial quotients of sqrt(7): 2, then period [1,1,1,4] *)
{1, 2, 1, 2, 2, 3, 1, 4, 1, 2, 2, 3, 1, 4, 1, 5, ...}  (* period lengths *)
```

The period length of $\sqrt{n}$ is related to the **Pell equation** $x^2 - ny^2 = 1$ — the fundamental solution can be read off from the convergents. This connection is one of the deepest in elementary number theory.

6.6 The Golden Ratio: Hardest to Approximate

The golden ratio $\phi = [1; 1, 1, 1, ...]$ has the simplest possible continued fraction — all partial quotients are 1. This makes it the "hardest" real number to approximate by rationals, in a precise sense: the convergents approach ϕ as slowly as possible.

```
CompareApproximability[{Pi, E, Sqrt[2], GoldenRatio}, 15]
```

Output:

[Table: for each constant, the product q_n^2 |x - p_n/q_n| for convergents n=1..15. For , the product occasionally dips very small (near large partial quotients). For , the product stays closest to 1/√5, never getting much smaller. is the hardest to approximate.]

The **Hurwitz theorem** states that for any irrational x, there are infinitely many rationals p/q with $|x - p/q| < 1/(\sqrt{5}q^2)$ — and the constant $\sqrt{5}$ is tight only for ϕ.

6.7 A Mystery: The Gauss-Kuzmin Distribution

If you pick a "random" irrational number, what is the probability that its kth partial quotient equals n? The **Gauss-Kuzmin theorem** gives the answer:

$$P(a_k = n) = -\log_2\left(1 - \frac{1}{(n+1)^2}\right)$$

So partial quotient 1 appears about 41.5% of the time, partial quotient 2 about 17%, partial quotient 3 about 9.3%, and so on — a distribution concentrated on small values.

```
GaussKuzminTest[Pi, 10000]
GaussKuzminTest[E, 10000]
```

Output:

[Two bar charts. x-axis: partial quotient value 1 to 10. y-axis: frequency. Both charts show distribution closely matching the theoretical Gauss-Kuzmin prediction. Chi-squared test: $p > 0.1$ for both.]

6.8 Further Research Topics

6.8.1 Topic 1 — Patterns in the CF of e

The continued fraction of e has a beautiful pattern: $e = [2; 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, 8, ...]$

```
CFExpansion[E, 30]
```

Output:

```
{2, 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, 8, 1, 1, 10, 1, 1, 12, 1, 1, 14, 1, 1, 16, 1, 1, 18, 1, 1,
```

Your tasks:

1. The pattern in e's CF is: after the initial 2, every third term is $2k$ (for $k = 1, 2, 3, ...$) and the other terms are 1. Verify this holds for the first 100 partial quotients.
2. Compute the CFs of $e^{1/2}$, e^2, e^3, $\tanh(1)$, and $\coth(1)$. Do any show similar patterns?

3. The CF of $\tan(1)$ is $[1; 1, 1, 3, 1, 5, 1, 7, ...]$ — another pattern. Verify this and find the general term.
4. Conjecture: which functions of e have "patterned" continued fractions? What distinguishes them from functions with unpredictable CFs?

6.8.2 Topic 2 — Approximation Quality

```
ApproximationQuality[{Pi, E, Sqrt[2], GoldenRatio}, 20]
```

Output:

[For each constant, table of convergents and quality measure q^2 |x - p/q|. consistently has the largest (worst) quality measure among the four constants.]

Your tasks:

1. For each of π, e, $\sqrt{2}$, ϕ: compute $q_n^2 \cdot |x - p_n/q_n|$ for the first 20 convergents. Which constant has the largest average quality measure? (It should be ϕ.)
2. Verify Hurwitz's theorem computationally: for each constant, confirm that infinitely many convergents satisfy $|x - p/q| < 1/(\sqrt{5}q^2)$.
3. For $\sqrt{2}$: all partial quotients are 2. Show that the quality measure $q_n^2|x - p_n/q_n|$ approaches $1/(2\sqrt{2})$ as $n \to \infty$.
4. Find the continued fraction of $\sqrt{2} + \sqrt{3}$. Is it periodic? What is its minimal polynomial?

6.8.3 Topic 3 — Gauss-Kuzmin Statistics

```
GaussKuzminFull[Pi, 100000]
```

Output:

[Bar chart of partial quotient frequencies for the first 100,000 partial quotients of . Overlaid theoretical Gauss-Kuzmin curve. Close match except for stochastic noise.]

Your tasks:

1. Compute the first 10,000 partial quotients of π, e, and $\sqrt{2}$. For each, plot the frequency of partial quotient values 1–20 against the Gauss-Kuzmin prediction. Which constant matches best?
2. The Gauss-Kuzmin theorem applies to "almost all" reals but not necessarily to specific constants. Compute the chi-squared statistic for each constant. Can you reject the hypothesis that they follow the Gauss-Kuzmin distribution?
3. Construct a number whose partial quotients are drawn randomly from the Gauss-Kuzmin distribution. Compare its approximation quality to that of π and e.
4. The partial quotient 292 in π is a rare outlier. How many partial quotients of π (in the first 10,000) exceed 100? 1000? Do they follow a power-law tail?

6.8.4 Topic 4 — CF Distance and the Stern-Brocot Tree

The **Stern-Brocot tree** is a binary tree that contains every positive rational number exactly once. It is built by the **mediant** operation: the mediant of a/b and c/d is $(a+c)/(b+d)$.

```
SternBrocotTree[5]    (* first 5 levels *)
CFToSternBrocot[22/7] (* path in the tree to 22/7 *)
```

Output:

[Tree diagram showing the first 5 levels of the Stern-Brocot tree. Path to 22/7 highlighted.]

Your tasks:

1. Build the Stern-Brocot tree to depth 8. Verify that every rational p/q with $q \leq 8$ appears exactly once.
2. Show that the path from the root to p/q in the Stern-Brocot tree encodes the continued fraction of p/q: left turns correspond to 1s in some encoding, right turns to the partial quotients.
3. Define "CF distance" between two rationals as the length of the path between them in the Stern-Brocot tree. Compute CF distances between all pairs of fractions p/q with $q \leq 10$. Does this metric have any geometric interpretation?
4. The Calkin-Wilf tree is a different tree containing all positive rationals. Implement it and compare its structure to the Stern-Brocot tree.

6.8.5 Topic 5 — Pell's Equation

The **Pell equation** $x^2 - Dy^2 = 1$ (for non-square positive integer D) always has infinitely many integer solutions. The fundamental solution (smallest positive x, y) can be read from the continued fraction of $\sqrt{D}$.

If $\sqrt{D} = [a_0; \overline{a_1, a_2, \ldots, a_r}]$ with period r, the fundamental solution is given by the convergent p_{r-1}/q_{r-1}.

```
PellFundamentalSolution[7]
PellFundamentalSolution[61]
```

Output:

```
{8, 3}          (* x=8, y=3: check 64 - 7*9 = 64-63 = 1   *)
{1766319049, 226153980}   (* famous large solution for D=61 *)
```

Your tasks:

1. Find the fundamental solutions of $x^2 - Dy^2 = 1$ for $D = 2, 3, 5, 6, 7, 8, 10, 11, 13$. In each case, verify the solution using exact integer arithmetic.
2. The solution for $D = 61$ is large. How does the size of the fundamental solution relate to the period length of $\sqrt{D}$'s CF?
3. From the fundamental solution (x_1, y_1), all other solutions can be generated by $(x_n + y_n\sqrt{D}) = (x_1 + y_1\sqrt{D})^n$. Implement this and generate the first 5 solutions for $D = 2$.
4. For which values of D from 2 to 100 is the period length of $\sqrt{D}$ equal to 1? These are the cases where $D = k^2 + 1$ for integer k. Verify this.

7 Chapter 7 — Integer Sequences and the OEIS

7.1 What Is a Sequence?

A **sequence** is an ordered list of numbers produced by a rule. Some rules are explicit: the nth prime, n^2, 2^n. Others are recursive: the Fibonacci sequence, where each term depends on the two before it. Others are harder to classify: the Collatz stopping times from Chapter 2, or the number of ways to tile a $2 \times n$ board with dominoes.

Integer sequences are one of the main objects of study in experimental mathematics. You generate a sequence, look at its terms, try to identify a pattern, and then search for that pattern in the mathematical literature. The tool that makes this search possible is the **Online Encyclopedia of Integer Sequences** — the OEIS.

7.2 The OEIS

The OEIS (oeis.org) is a database of over 370,000 integer sequences, maintained by a community of mathematicians since 1964. It was started by Neil Sloane, then a graduate student, who kept a card file of sequences he encountered in his research.

For each sequence, the OEIS provides: - The first several terms - A name and description - Formulas (explicit, recursive, generating functions) - References to the mathematical literature - Links to related sequences - Wolfram Language, Python, and other code for generating the sequence

To search for a sequence: go to oeis.org, enter the first several terms separated by commas, and look for matches. The more terms you enter, the more specific the search.

7.3 Difference Tables

One powerful technique for identifying a sequence is the **difference table**: repeatedly take differences between consecutive terms until you reach a row of constants (or zeros).

For a polynomial sequence of degree d, the difference table reaches a constant row after exactly d differences.

```
DifferenceTable[{1, 4, 9, 16, 25, 36, 49}, 4]
```

Output:

```
{1,  4,  9,  16, 25, 36, 49}   (* original: n^2 *)
{3,  5,  7,  9,  11, 13}       (* first differences *)
{2,  2,  2,  2,  2}            (* second differences: CONSTANT *)
{0,  0,  0,  0}                (* third differences: zero *)
```

The sequence $1, 4, 9, 16, \ldots$ is quadratic (degree 2), so the second difference row is constant and the third is zero.

```
DifferenceTable[FibSequence[10], 4]
```

Output:

```
{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}
{0, 1, 1, 2, 3, 5, 8, 13, 21}
{1, 0, 1, 1, 2, 3, 5, 8}
{-1, 1, 0, 1, 1, 2, 3}
...
```

The Fibonacci sequence never reaches a constant row — it is not polynomial. The difference table just produces more Fibonacci-like sequences.

7.4 Identifying Recurrences

Many sequences satisfy a **linear recurrence**: each term is a fixed linear combination of previous terms. Given the first several terms, we can often identify the recurrence computationally.

```
FindSequenceRecurrence[{1, 1, 2, 3, 5, 8, 13, 21, 34, 55}]
FindSequenceRecurrence[{1, 3, 7, 15, 31, 63, 127}]
FindSequenceRecurrence[{1, 2, 6, 24, 120, 720, 5040}]
```

Output:

```
a[n] = a[n-1] + a[n-2]          (* Fibonacci *)
a[n] = 2*a[n-1] + 1             (* 2^n - 1   *)
None found                       (* factorials: not a linear recurrence *)
```

Factorials do not satisfy a linear recurrence with constant coefficients — but they satisfy a simple *non-linear* recurrence: $a_n = n \cdot a_{n-1}$.

7.5 Recamán's Sequence

One of the most intriguing sequences in the OEIS is **Recamán's sequence** (A005132), defined by:

$$a_0 = 0, \quad a_n = \begin{cases} a_{n-1} - n & \text{if } a_{n-1} - n > 0 \text{ and not already in the sequence} \\ a_{n-1} + n & \text{otherwise} \end{cases}$$

```
RecamanSequence[70]
```

Output:

```
{0, 1, 3, 6, 2, 7, 13, 20, 12, 21, 11, 22, 10, 23, 9, 24, 8, 25,
 43, 62, 42, 63, 41, 18, 42, 17, 43, 16, 44, 15, 45, 14, 46, 79, 113, ...}
```

The sequence is visualized as arcs on the number line, alternating above and below:

```
PlotRecaman[70]
```

Output:

[Number line from 0 to ~120. Arcs drawn above (for subtraction steps) and below (for addition steps) connecting consecutive terms. The pattern is irregular but visually striking, resembling wave interference.]

The open question: Does every positive integer eventually appear in Recamán's sequence? After 70 terms, some small integers are still missing. After 10,000 terms, the integer 852,655 had not yet appeared. This is an active open problem.

```
RecamanMissing[1000]    (* integers not yet in the first 1000 terms *)
```

Output:

[List of positive integers up to ~2000 that are missing from the first 1000 terms. Contains hundreds of values, apparently scattered without obvious pattern.]

7.6 The Look-and-Say Sequence

Another famous sequence starts from a simple "description" rule. Starting from 1:

- 1 (read as: "one 1") → 11
- 11 (read as: "two 1s") → 21
- 21 (read as: "one 2, one 1") → 1211
- 1211 (read as: "one 1, one 2, two 1s") → 111221
- ...

```
LookAndSay[10]
```

Output:

```
{"1", "11", "21", "1211", "111221", "312211",
 "13112221", "1113213211", "31131211131221", "13211311123113112211"}
```

The mathematician John Conway proved that the terms grow in length by a factor of approximately $\lambda \approx 1.303577$ per step, regardless of the starting value (except for a few degenerate cases). Conway also decomposed the sequence into 92 fundamental "atoms" that evolve independently.

```
LookAndSayGrowthRate[20]
```

Output:

[Plot of term length vs. step for $n = 1$ to 20. Exponential growth. Ratio of consecutive lengths converges toward 1.3035…]

7.7 Using the OEIS as a Research Tool

Here is the workflow for using the OEIS in mathematical exploration:

1. **Generate** several terms of your sequence (at least 6–8; more is better).
2. **Search** oeis.org with those terms.
3. **Read** the entry carefully: name, formulas, cross-references.
4. **Verify** the formulas computationally.
5. **Explore** the related sequences linked in the entry.

If your sequence is not found: you may have discovered a new sequence. Double-check your computation, search for subsequences, and if it still does not appear, consider submitting it.

```
GenerateAndSearch[Table[n^2 + 1, {n, 1, 10}]]
```

This function generates the sequence and formats it for pasting into the OEIS search (see Section 15.7.8).

7.8 Further Research Topics

7.8.1 Topic 1 — Iterated Difference Tables

```
IteratedDifferences[FibSequence[15], 10]
IteratedDifferences[Table[2^n, {n, 0, 14}], 10]
```

Output:

[For Fibonacci: the difference table never stabilizes; every row is a shifted Fibonacci-like sequence. For powers of 2: the table never stabilizes either — every row is the same sequence.]

Your tasks:

1. For each sequence — n^2, n^3, 2^n, F_n, $n!$ — compute 10 levels of iterated differences and describe what happens. Which sequences produce a constant row? Which produce a zero row? Which never stabilize?
2. A sequence is polynomial of degree d if and only if the $(d+1)$th difference row is identically zero. Verify this for $d = 1, 2, 3, 4$ using sequences n, n^2, n^3, n^4.
3. Construct a sequence whose difference table reaches zero in exactly 5 steps. What is the general form of such a sequence?
4. What happens to the difference table of a sequence that satisfies a linear recurrence? Experiment with $a_n = 2a_{n-1}$ and $a_n = a_{n-1} + a_{n-2}$.

7.8.2 Topic 2 — Discovering New Sequences

```
SmallestPrimeFreeSequence[50]    (* custom novel sequence *)
```

Your tasks:

1. Define a sequence by the rule: $a_1 = 1$; a_n is the smallest positive integer not yet in the sequence such that $a_n + a_{n-1}$ is prime. Generate 50 terms and search OEIS. Is it listed?
2. Define: $a_1 = 1$; a_n is the smallest positive integer not yet in the sequence such that $|a_n - a_{n-1}|$ is a perfect square. Generate 50 terms. Search OEIS.
3. For each sequence you define: if it is not in OEIS, rigorously document it (definition, first 20 terms, a conjecture about its behavior) as if you were writing an OEIS entry.
4. Take any sequence from the OEIS and define three new sequences derived from it (interleave, running sum, sequence of prime-valued indices). Search OEIS for all three. How often are derived sequences already catalogued?

7.8.3 Topic 3 — Recamán in Depth

```
RecamanMissing[10000]
PlotRecamanMissing[10000]    (* plot of missing values vs. step count *)
```

Output:

[List of integers not in the first 10,000 terms. Plot shows the number of missing integers decreasing slowly as more terms are computed.]

Your tasks:

1. Compute the first 10,000 terms of Recamán's sequence. List all positive integers below 2000 that are still missing. Is there a pattern to the missing values?
2. Plot the number of missing integers below N as a function of the number of terms computed. Does it appear to approach 0? How fast?
3. Define the "Recamán index" of a positive integer n as the step at which n first appears (or ∞ if it never does). For all integers 1 to 100, compute their Recamán index. Do the indices have any pattern?
4. Investigate: does the sequence ever subtract the same value twice? (i.e., are there two steps $n_1 < n_2$ where $a_{n_i} = a_{n_i - 1} - n_i$, and also $n_1 = n_2$?) The answer is no by definition — but are there collisions in the *values* of n used?

7.8.4 Topic 4 — Look-and-Say Chemistry

Conway showed that any look-and-say sequence (for starting values other than 22) eventually splits into 92 fundamental strings that evolve independently, which he called "chemical elements."

```
LookAndSayDecompose[LookAndSay[15][[-1]]]
```

Output:

[The 15th term decomposed into Conway's atomic strings. Each atom labeled with its Conway element number.]

Your tasks:

1. Compute the first 20 terms of the look-and-say sequence starting from 1. For each term, measure its length. Compute the ratio of consecutive lengths. Does it converge to $\lambda \approx 1.3035$?

2. Verify Conway's claim: start from three different starting values (e.g., 3, 12, 312). After 10 iterations, decompose each into atomic strings. Do all three produce the same set of atoms (in different proportions)?
3. The starting value 22 is a fixed point of the look-and-say rule: "two 2s" $\to$ 22. Find all other fixed points or cycles.
4. Define a "reverse look-and-say" sequence: instead of describing groups of equal digits, describe *runs* from right to left. Does this variant also grow at rate λ?

7.8.5 Topic 5 — Sequences and Their Complexity

Some sequences feel "simple" (powers of 2, Fibonacci) and others feel "complex" (Recamán, look-and-say). Can we make this precise?

```
SequenceEntropy[RecamanSequence[1000], 8]
SequenceEntropy[FibSequence[1000], 8]
SequenceEntropy[Table[2^n, {n, 0, 999}], 8]
```

Output:

[Shannon entropy of symbol distributions in each sequence, computed over blocks of length up to 8. Recamán has higher entropy than Fibonacci, which has higher than powers of 2.]

Your tasks:

1. Define "sequence complexity" as the Shannon entropy of the distribution of k-term blocks for $k = 1, 2, 3, 4$. Compute it for: powers of 2, Fibonacci, primes, Recamán, and a random sequence. Rank them.
2. The Lempel-Ziv complexity measures how compressible a sequence is. Apply it to the same five sequences. Does the ranking agree with entropy?
3. Take two sequences A and B. Define their "interleave" C as $a_1, b_1, a_2, b_2, \ldots$ and their "sum" D as $a_n + b_n$. Is the complexity of C or D predictable from the complexities of A and B?
4. Conjecture a definition of "interesting sequence" based on complexity. A sequence that is too simple (like $1, 1, 1, \ldots$) or too complex (random) is uninteresting; the interesting sequences are somewhere in between. Where does each of the sequences above fall on this spectrum?

Part II

Part II - Structure and Dynamics

8 Chapter 8 — Cellular Automata: Rules That Generate Worlds

8.1 A World of Simple Rules

Imagine a row of light switches — each either on or off. At each tick of a clock, every switch changes state according to a rule that looks at itself and its two immediate neighbors. After many ticks, what patterns emerge?

This is the setup for **elementary cellular automata** (ECA), introduced and systematically studied by Stephen Wolfram in the 1980s. Despite the extreme simplicity of the setup — one dimension, two states, neighbors of radius 1 — the variety of behaviors that emerges is astonishing.

8.2 Encoding Rules

Each cell has two possible states (0 or 1). A cell and its two neighbors form a 3-bit pattern, giving $2^3 = 8$ possible neighborhood configurations. Each configuration maps to a new state (0 or 1). Therefore there are $2^8 = 256$ possible rules.

We number rules 0 through 255 using the **Wolfram code**: the rule number in binary tells you what each of the 8 neighborhood configurations maps to.

For example, Rule 30 in binary is 00011110:

Neighborhood	111	110	101	100	011	010	001	000
New state	0	0	0	1	1	1	1	0

```
ShowRule[30]
```

Output:

[Diagram showing all 8 neighborhood configurations as small grids of 3 cells (black=1, white=0), with an arrow pointing to the resulting cell state below each. Rule 30's pattern: 00011110.]

8.3 Generating Space-Time Diagrams

Starting from a single black cell and applying a rule repeatedly generates a **space-time diagram** — a 2D image where each row is one generation.

```
CellularAutomatonPlot[30, 200]
CellularAutomatonPlot[110, 200]
CellularAutomatonPlot[90, 200]
```

Output:

[Three space-time diagrams, each 200 rows tall. Rule 30: chaotic, seemingly random pattern. Rule 110: complex irregular structures, triangles and gliders. Rule 90: perfect Sierpinski triangle.]

Wolfram classified the 256 rules into four qualitative classes:

- **Class I:** Fixed point — all cells die or all turn on. (e.g., Rule 0, Rule 255)
- **Class II:** Periodic — simple repeating or symmetric patterns. (e.g., Rule 4, Rule 108)
- **Class III:** Chaotic — irregular, random-looking patterns. (e.g., Rule 30, Rule 45)
- **Class IV:** Complex — localized structures that interact. (e.g., Rule 110)

Rule 110 has been proved **computationally universal** — it can simulate any computation, making it as powerful as a general-purpose computer, despite its absurdly simple description.

8.4 Rule 30: A Random Number Generator

Rule 30 is used as Wolfram Language's built-in pseudorandom number generator for some applications. The center column of the Rule 30 space-time diagram, starting from a single cell, produces the sequence:

$$1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, ...$$

This sequence passes many standard randomness tests. But it is completely deterministic — the same starting cell always produces the same sequence.

```
Rule30CenterColumn[200]
```

Output:

[List of 200 binary digits from the center column of Rule 30. Looks random with no visible period.]

```
Rule30RandomnessTests[1000]
```

Output:

[Results of several randomness tests: digit frequency (near 50/50), run-length distribution (close to geometric), autocorrelation (near zero for lags > 0). Rule 30 passes all.]

8.5 Rule 90 and Pascal's Triangle

Rule 90 generates a perfect Sierpinski triangle from a single cell — the same fractal pattern we saw in Chapter 5 when coloring Pascal's triangle modulo 2. This is not a coincidence: Rule 90 is equivalent to XOR (addition modulo 2) of the two neighbors, and Pascal's triangle mod 2 is built by the same operation.

```
CompareRule90Pascal[128]
```

Output:

[Side-by-side images: Rule 90 space-time diagram (128 rows) and Pascal's triangle mod 2 (128 rows). They are identical.]

8.6 Symmetry Classes

Many of the 256 rules are equivalent under simple transformations: - **Left-right reflection:** swap the left and right neighbors. - **Color inversion:** swap 0 and 1. - Both combined.

These symmetries partition the 256 rules into equivalence classes. Rules that are equivalent produce the same qualitative behavior (mirrored or color-inverted).

```
WolframEquivalenceClasses[]
```

Output:

[Summary: 256 rules organized into 88 distinct equivalence classes. Class sizes vary from 1 (self-symmetric rules) to 4.]

8.7 Further Research Topics

8.7.1 Topic 1 — Testing Rule 30 for Randomness

```
Rule30Stats[10000]
```

Output:

[Table of statistical properties: digit frequency, run-length histogram, autocorrelation for lags 1–20, entropy. All consistent with random.]

Your tasks:

1. Compute 10,000 bits from the center column of Rule 30. Calculate the frequency of 0s and 1s. Is it close to 50/50?
2. Compute the run-length distribution (lengths of consecutive runs of 0s or 1s). For a fair coin, run lengths should follow a geometric distribution. Does Rule 30?
3. Compute the autocorrelation $\sum_i b_i b_{i+k}$ for lags $k = 1$ to 20. For a random sequence this should be near zero. What do you find?
4. Compute the Shannon entropy of the Rule 30 center column in blocks of length $k = 1, 2, 3, 4$. Compare to a truly random sequence (use `RandomInteger`). How close is Rule 30?
5. Rule 30 is *not* truly random — it is periodic in theory (eventually repeating). Estimate the period length computationally. (It is astronomically large.)

8.7.2 Topic 2 — Symmetry and Equivalence Classes

```
RuleSymmetryTable[]
```

Output:

[256x4 table: each rule, its left-right mirror, its color inverse, and its combined transformation. Rules that map to themselves are self-symmetric.]

Your tasks:

1. For all 256 rules, compute the set of rules equivalent under left-right reflection and/or color inversion. How many distinct equivalence classes are there?
2. Which rules are self-symmetric under left-right reflection? Under color inversion? Under both?
3. Generate space-time diagrams for rules 30 and its three equivalents. Verify they look like reflections/inversions of each other.
4. For each Wolfram class (I–IV): are the rules within a class closed under these symmetries? That is, if Rule r is Class III, are all its equivalents also Class III?

8.7.3 Topic 3 — Sensitivity to Initial Conditions

```
SensitivityTest[110, 200]
```

Output:

[Two side-by-side space-time diagrams for Rule 110. Left: initial condition with a single 1 at center. Right: same but with one additional 1 placed 50 cells away. The two diagrams look identical at first, then diverge dramatically after about 80 rows.]

Your tasks:

1. For Rules 30, 90, and 110: start two runs, one from a single cell and one from that same configuration plus a single extra cell far away. At what row do the two diagrams first differ?
2. For Rule 30: place the extra cell at various distances $d = 10, 20, 50, 100$ from the center. How does the row of first difference depend on d?
3. Class II rules are periodic — do they show sensitivity to initial conditions? Test Rule 4 and Rule 108.

4. Define a "divergence distance" $\delta(r, d)$ for rule r and initial separation d. Plot δ vs. d for Rule 30. Is the relationship linear, logarithmic, or something else?

8.7.4 Topic 4 — 2D Totalistic Rules and Conway's Game of Life

A **totalistic** 2D rule depends only on the *sum* of neighboring cells, not their arrangement. Conway's Game of Life is a totalistic rule on a 2D grid:

- A live cell with 2 or 3 live neighbors survives.
- A dead cell with exactly 3 live neighbors becomes alive.
- All other cells die or stay dead.

```
GameOfLifeSteps[GliderPattern[], 50]
```

Output:

[Animation frames (or static grid) showing the "glider" pattern moving across the grid over 50 generations.]

Your tasks:

1. Implement the Game of Life (see Section 15.8.6) and simulate the "glider" for 100 generations. Verify that it moves diagonally one cell every 4 generations.
2. Find three other stable Game of Life patterns: a "still life" (never changes), an "oscillator" (period > 1), and a pattern that eventually dies out.
3. Invent a 2D totalistic rule (different from Life) that produces a "glider" — a pattern that moves across the grid. What structural constraints on the rule make this possible?
4. Compute the entropy of the Game of Life space-time diagram for a random initial condition. Compare to Rule 30.

8.7.5 Topic 5 — Entropy and Wolfram's Classification

```
AllRuleEntropies[100]
```

Output:

[Plot of Shannon entropy (of cell-state distribution in space-time diagrams) for all 256 rules, grouped by Wolfram class. Class I near 0, Class II near some constant, Class III near maximum entropy 1, Class IV intermediate.]

Your tasks:

1. For all 256 rules, generate 100-row space-time diagrams from a single-cell initial condition. Compute the Shannon entropy of the resulting binary image.
2. Plot entropy vs. rule number. Color-code by Wolfram's classification. Does entropy separate the classes cleanly?
3. Compute entropy for the same rules starting from a *random* initial condition (50% of cells active). Does the classification order change?
4. The Lempel-Ziv complexity is a compression-based complexity measure. Compute it for all 256 rule space-time diagrams and compare to entropy. Which measure better separates Wolfram's classes?

9 Chapter 9 — Chaos in a Simple Formula: The Logistic Map

9.1 A Model for Population Growth

Suppose a population of animals lives on an island with limited food. When the population is small, it grows rapidly. When it approaches the island's carrying capacity, growth slows and eventually reverses. A simple mathematical model for this is:

$$x_{n+1} = r\, x_n (1 - x_n)$$

Here $x_n \in [0, 1]$ represents the population as a fraction of the maximum possible, and r is a growth rate parameter. This is the **logistic map**, one of the most studied equations in all of mathematics.

Despite having only one parameter r, this simple formula exhibits an astonishing progression of behaviors — from stable equilibrium, through cycles of every period, to what appears to be genuine chaos.

9.2 Iterating the Map

Starting from an initial population x_0, we apply the map repeatedly and watch what happens.

```
LogisticOrbit[r -> 2.5, x0 -> 0.4, n -> 50]
```

Output:

[Line plot, 50 steps. x-axis: step, y-axis: population. Oscillates briefly then converges smoothly to a fixed value near 0.6.]

For $r = 2.5$, the population settles to a fixed point. Now try $r = 3.2$:

```
LogisticOrbit[r -> 3.2, x0 -> 0.4, n -> 100]
```

Output:

[Line plot, 100 steps. After initial transient, oscillates between two values (~0.51 and ~0.80). A 2-cycle.]

And $r = 3.5$:

```
LogisticOrbit[r -> 3.5, x0 -> 0.4, n -> 100]
```

Output:

[Line plot, 100 steps. After transient, oscillates among four distinct values. A 4-cycle.]

As r increases, the period doubles: $1 \to 2 \to 4 \to 8 \to \ldots$ and eventually, for $r \gtrsim 3.57$, the behavior becomes **chaotic** — never repeating, sensitive to the starting value, and apparently random.

9.3 The Bifurcation Diagram

The most informative view of the logistic map is the **bifurcation diagram**: for each value of r, we run the map for a long time to discard the transient, then plot the long-term behavior.

```
BifurcationDiagram[{2.5, 4.0}, 500]
```

Output:

[Bifurcation diagram. x-axis: r from 2.5 to 4.0. y-axis: long-term x values. Single line from r=2.5 to r=3 (fixed point), splits to 2-cycle at r 3, splits again to 4-cycle at r 3.45, then to 8-cycle, then chaos. Within the chaotic region, narrow white "windows of order" where periodic behavior reappears.]

This diagram is one of the most famous images in all of mathematics. The period-doubling cascade, the windows of order inside the chaos, and the self-similar structure at every scale all emerge from a single quadratic equation.

9.4 Bifurcation Points and Feigenbaum's Constant

The values of r at which each period doubling occurs are called **bifurcation points**:

Bifurcation	Approximate r value
Fixed point $\to$ period 2	$r_1 \approx 3.0000$
Period 2 $\to$ period 4	$r_2 \approx 3.4495$
Period 4 $\to$ period 8	$r_3 \approx 3.5441$
Period 8 $\to$ period 16	$r_4 \approx 3.5644$
$\vdots$	$\vdots$

The remarkable discovery, made by Mitchell Feigenbaum in 1975, is that the *ratios* of successive intervals between bifurcation points converge to a universal constant:

$$\delta = \lim_{n\to\infty} \frac{r_n - r_{n-1}}{r_{n+1} - r_n} \approx 4.6692016\ldots$$

This constant — called the **Feigenbaum constant** — appears in the period-doubling cascades of *all* one-dimensional maps with a single quadratic maximum. It is one of the great universal constants of nonlinear dynamics.

```
FeigenbaumEstimate[6]
```

Output:

```
Bifurcation points: {3.0, 3.4495, 3.5441, 3.5644, 3.5688, 3.5697}
Ratios: {4.752, 4.656, 4.668, 4.669}
Estimated delta: 4.669  (true value: 4.6692...)
```

9.5 Sensitive Dependence on Initial Conditions

For r in the chaotic regime, two trajectories starting from nearly identical initial values diverge exponentially. This is the hallmark of chaos: long-term prediction is impossible because any small uncertainty in the initial state is amplified without bound.

```
ChaosDemo[r -> 3.9, x0 -> 0.4, epsilon -> 1*^-10, n -> 100]
```

Output:

[Two overlapping trajectories, one starting at 0.4 and one at 0.4 + 10^{-10}. They look identical for about 40 steps, then diverge completely by step 60.]

The rate of divergence is measured by the **Lyapunov exponent** λ: if the separation grows as $e^{\lambda n}$, then $\lambda > 0$ indicates chaos and $\lambda < 0$ indicates stability.

```
LyapunovExponent[r -> 3.9]
LyapunovExponent[r -> 3.5]
```

Output:

```
0.341    (* r=3.9: positive -> chaotic *)
-0.179   (* r=3.5: negative -> stable 4-cycle *)
```

9.6 Windows of Order

Inside the chaotic region of the bifurcation diagram, there are narrow intervals of r where periodic behavior reappears — called **windows of order**. The most prominent is the period-3 window near $r \approx 3.828$.

A theorem due to Sharkovskii states: if a continuous map has a period-3 orbit, it has orbits of every period. So the appearance of period 3 in the logistic map implies that all periods are present for that value of r — consistent with the chaotic behavior surrounding the window.

```
ZoomBifurcation[{3.82, 3.86}, 200]
```

Output:

[Zoomed bifurcation diagram. Clear period-3 window visible with its own internal period-doubling cascade starting from 3 branches, then 6, then 12...]

9.7 Further Research Topics

9.7.1 Topic 1 — Feigenbaum's Constant

```
BifurcationPoints[8]
```

Output:

[Table of first 8 bifurcation points computed to 6 decimal places, with intervals and ratios. Ratios converging toward 4.6692.]

Your tasks:

1. Compute the first 6 bifurcation points to 6 decimal places using bisection search (see **?@sec-app-bifurcation-search**). Compute the ratios of successive intervals. How close do they get to $\delta \approx 4.6692$?
2. The onset of chaos occurs at $r_\infty \approx 3.5699$. Estimate it by extrapolating the geometric series: $r_\infty \approx r_1 + (r_2 - r_1)/(1 - 1/\delta)$. How accurate is this estimate?
3. Verify that the same Feigenbaum constant appears for the map $x_{n+1} = r\sin(\pi x_n)$. Compute its first 4 bifurcation points and estimate δ.
4. The Feigenbaum α constant (≈ 2.5029) describes the self-similar scaling of the bifurcation diagram in the x direction. Measure it from your bifurcation diagram.

9.7.2 Topic 2 — Universality

Feigenbaum's discovery was that the constant δ is the same for *all* unimodal maps (maps with a single hump). This universality is one of the deepest results in nonlinear dynamics.

```
BifurcationDiagram[SineMap, {0.5, 1.0}, 500]
BifurcationDiagram[TentMap, {0.5, 1.0}, 500]
```

Output:

[Two bifurcation diagrams for the sine map and tent map. Both show qualitatively identical period-doubling cascades to the logistic map.]

Your tasks:

1. Generate the bifurcation diagram for $x_{n+1} = r\sin(\pi x_n)$. Identify the period-doubling sequence and the onset of chaos. Is it qualitatively identical to the logistic map?

2. Consider the cubic map $x_{n+1} = rx_n(1 - x_n^2)$. Does it also exhibit period doubling? Does it have the same Feigenbaum constant?
3. The tent map $T_r(x) = rx$ for $x < 0.5$ and $r(1 - x)$ for $x \geq 0.5$ is piecewise linear. It also has period doubling. Compute its Feigenbaum constant. Is it $\delta \approx 4.6692$ or different?
4. For each map you study, plot the Lyapunov exponent vs. r. Verify that λ crosses zero exactly at the onset of chaos.

9.7.3 Topic 3 — Lyapunov Exponents

```
LyapunovPlot[{2.5, 4.0}, 500]
```

Output:

[Plot of Lyapunov exponent vs. r for the logistic map. Negative in periodic regions, crosses zero at onset of chaos, positive in chaotic region. Dips back to negative inside windows of order.]

Your tasks:

1. Compute the Lyapunov exponent $\lambda(r)$ for the logistic map for $r \in [2.5, 4.0]$ in steps of 0.001. Plot it.
2. Verify that λ is exactly 0 at each bifurcation point.
3. At the period-3 window ($r \approx 3.828$), λ dips sharply negative. Zoom in. How sharp is the transition at the window boundary?
4. The maximum Lyapunov exponent over all $r \in [0, 4]$ should occur at $r = 4$, where $\lambda = \ln 2$. Verify this and explain why $r = 4$ is special (the logistic map is conjugate to the tent map at $r = 4$).

9.7.4 Topic 4 — Windows of Order

```
FindPeriodicWindows[{3.5, 4.0}, maxPeriod -> 8]
```

Output:

[Table: period, approximate r-interval of window, window width. Period-3 window is widest. All windows with period 8 catalogued.]

Your tasks:

1. Find all windows of period ≤ 8 in $r \in [3.5, 4.0]$. For each, report the period and the approximate r-interval.
2. The periods appear in a specific order as r increases, known as the **U-sequence**: $1, 2, 4, \ldots, 6, 5, 3, \ldots$ Verify this ordering from your catalog.
3. Each window has its own internal period-doubling cascade. Zoom into the period-3 window and verify that it contains period-6, period-12, and period-24 sub-windows.
4. Sharkovskii's theorem orders all positive integers: $3 \succ 5 \succ 7 \succ \ldots \succ 6 \succ 10 \succ \ldots \succ 4 \succ 2 \succ 1$. If a continuous map has a period-m orbit and $m \succ n$, it also has a period-n orbit. Verify this for the logistic map at several values of r.

9.7.5 Topic 5 — Coupled Logistic Maps

When two logistic maps are coupled, new phenomena emerge.

$$x_{n+1} = (1-\varepsilon)\, r\, x_n(1-x_n) + \varepsilon\, r\, y_n(1-y_n)$$
$$y_{n+1} = (1-\varepsilon)\, r\, y_n(1-y_n) + \varepsilon\, r\, x_n(1-x_n)$$

```
CoupledLogisticDiagram[r -> 3.8, epsilon -> 0.1, n -> 500]
```

Output:

[Bifurcation-like diagram for the coupled system showing synchronized and asynchronous regions.]

Your tasks:

1. For $r = 3.8$ and ε varying from 0 to 0.5: plot the long-term behavior of $x_n - y_n$. For what values of ε do the two maps synchronize ($x_n = y_n$)?
2. At $\varepsilon = 0$, the maps are independent. At $\varepsilon = 0.5$, they are maximally coupled (the two maps become identical). Find the critical ε^* at which synchronization first occurs.
3. For $r = 3.5$ (periodic regime with $\varepsilon = 0$): how does coupling affect the period? Does coupling ever induce chaos in a system that would be periodic without coupling?
4. Plot the trajectory of (x_n, y_n) in the 2D plane for several values of ε. For which values does the trajectory lie on the diagonal $x = y$ (synchronized) vs. filling a 2D region (asynchronized)?

Part III

Part III - Geometry Through Computation

10 Chapter 10 — Fractal Geometry: Infinite Complexity from Simple Rules

10.1 Shapes That Repeat at Every Scale

Classical geometry studies shapes with smooth boundaries: lines, circles, spheres. Their lengths and areas are finite and well-defined. But many shapes in nature — coastlines, mountain profiles, tree branches, snowflakes — are rough at every scale of magnification. Zooming in on a coastline reveals bays within bays within bays, with no smooth structure appearing no matter how closely you look.

Fractal geometry, developed primarily by Benoit Mandelbrot in the 1970s and 1980s, provides mathematical tools for studying such shapes. Two key ideas define a fractal: - **Self-similarity:** the shape looks similar (or identical) at different scales. - **Fractional dimension:** the shape is "too rough" to be a curve (dimension 1) but "too thin" to fill a plane (dimension 2). Its dimension is a fraction between 1 and 2.

10.2 Iterated Function Systems

The simplest way to generate a fractal is an **iterated function system (IFS)**: a set of contractive affine transformations, applied repeatedly.

10.2.1 The Sierpiński Triangle

The Sierpiński triangle is generated by three transformations, each scaling by 1/2 toward a different corner of a triangle.

```
SierpinskiIFS[8]
```

Output:

[Sierpiński triangle at iteration 8. A triangle subdivided into three smaller copies of itself, with the center triangle removed. Self-similar at every scale.]

10.2.2 The Barnsley Fern

Four affine transformations with carefully chosen parameters produce a shape that looks exactly like a fern frond.

```
BarnsleyFern[100000]
```

Output:

[Green fern shape constructed from 100,000 random points via the chaos game. Remarkably organic-looking despite being generated by four simple affine transformations.]

The **chaos game** algorithm generates IFS fractals efficiently: pick a random starting point, randomly select one of the transformations with given probabilities, apply it, and plot the result. After discarding the first few points, the remaining points trace the fractal attractor.

```
ChaosGame[SierpinskiTransforms[], 50000]
```

Output:

[Sierpiński triangle constructed by the chaos game. Same shape as the direct IFS but computed faster with 50,000 random points.]

10.3 The Koch Snowflake

The Koch snowflake is built by a substitution rule: replace each line segment with a "tent" — four segments each 1/3 the original length.

```
KochSnowflake[5]
```

Output:

[Koch snowflake at iteration 5. Star-shaped boundary with deeply indented fractal structure. Each edge is a Koch curve.]

The perimeter of the Koch snowflake grows without bound — multiplied by 4/3 at each iteration — while the area it encloses converges to 8/5 of the original triangle's area.

```
KochPerimeterArea[10]
```

Output:

```
Iteration  Perimeter  Area
0          3.000      0.433
1          4.000      0.481
2          5.333      0.494
3          7.111      0.498
...
10         70.189     0.500
```

An infinite perimeter enclosing a finite area. This violates the intuition from classical geometry that longer perimeters enclose more area.

10.4 The Mandelbrot Set

The **Mandelbrot set** is defined in the complex plane. For each complex number c, iterate the map:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0$$

If the sequence $|z_n|$ remains bounded, c belongs to the Mandelbrot set (colored black). If it escapes to infinity, c is outside, and it is colored by the **escape time** — the number of iterations before $|z_n| > 2$.

```
MandelbrotPlot[{-2.5, 1.0}, {-1.25, 1.25}, 500, 200]
```

Output:

[Mandelbrot set. Large cardioid (main body), circular bulb to its left, smaller bulbs attached. Surrounded by increasingly complex boundary structure. Outside colored by escape time (rainbow gradient). The boundary has infinite complexity.]

Zooming into the Mandelbrot set boundary reveals endless intricate structure — and at certain points, recognizable miniature copies of the entire Mandelbrot set.

```
MandelbrotZoom[center -> {-0.7269, 0.1889}, width -> 0.001, res -> 400]
```

Output:

[Zoomed view into the Mandelbrot set boundary. Spiral tendrils, miniature Mandelbrot copies, filaments. Structure is as complex as the full image.]

10.5 Box-Counting Dimension

The **box-counting dimension** measures the fractal dimension of a shape:

$$d = \lim_{\epsilon \to 0} \frac{\log N(\epsilon)}{\log(1/\epsilon)}$$

where $N(\epsilon)$ is the number of boxes of side ϵ needed to cover the shape.

```
FractalDimension[SierpinskiIFS[7]]
FractalDimension[KochCurve[5]]
```

Output:

```
Sierpinski triangle: d   1.585  (theoretical: log(3)/log(2)   1.585)
Koch curve:          d   1.262  (theoretical: log(4)/log(3)   1.262)
```

10.6 Further Research Topics

10.6.1 Topic 1 — Mandelbrot Set Boundary Dimension

The Mandelbrot set boundary is conjectured to have box-counting dimension 2 — meaning it is so complex that it nearly fills the plane near the boundary.

```
MandelbrotBoundaryDimension[500]
```

Output:

[Log-log plot for box-counting. Slope 1.95 to 2.0 depending on resolution. Consistent with theoretical value of 2.]

Your tasks:

1. Estimate the box-counting dimension of the Mandelbrot set boundary at resolutions $\epsilon = 2^{-k}$ for $k = 4, \ldots, 10$. Plot $\log N(\epsilon)$ vs. $\log(1/\epsilon)$ and fit a line.
2. Zoom into two different regions of the Mandelbrot set boundary. Does the local box-counting dimension appear the same in both regions?
3. Compute the boundary of the Mandelbrot set by identifying cells that contain both inside and outside points. Plot the boundary separately and estimate its dimension.
4. The **filled Julia set** for a fixed c has a similar structure to the Mandelbrot set. Estimate the boundary dimension of Julia sets for several values of c — inside and outside the Mandelbrot set. Does boundary dimension vary with c?

10.6.2 Topic 2 — Julia Sets and the Mandelbrot Map

For each c, the **Julia set** J_c is the boundary between initial conditions z_0 that escape to infinity and those that remain bounded under $z \mapsto z^2 + c$.

```
JuliaSet[c -> {-0.4, 0.6}, res -> 400]
JuliaSet[c -> {0.285, 0.01}, res -> 400]
JuliaSet[c -> {-1.476, 0.0}, res -> 400]
```

Output:

[Three Julia sets for different c values. First: a connected dendrite-like set. Second: a Douady rabbit (three lobes). Third: a Cantor dust (disconnected).]

Your tasks:

1. Generate Julia sets for 9 values of c arranged in a 3×3 grid: three inside the Mandelbrot set, three on its boundary, three outside. Describe the qualitative difference (connected vs. disconnected).
2. Verify the fundamental theorem: J_c is connected if and only if c is in the Mandelbrot set. Test for 20 values of c.
3. Replace $z^2 + c$ with $z^3 + c$. Generate the corresponding "Multibrot" set. How does the shape change? Count the number of "bulbs."
4. The Burning Ship fractal replaces the iteration with $z \mapsto (|\mathrm{Re}(z)| + i|\mathrm{Im}(z)|)^2 + c$. Implement it and compare qualitatively to the Mandelbrot set.

10.6.3 Topic 3 — IFS Parameter Exploration

```
IFSExplorer[contraction -> 0.5, rotation -> Pi/6, n -> 50000]
```

Output:

[Fractal attractor generated by a 2-transformation IFS with the given contraction and rotation. Shape changes dramatically with parameter values.]

Your tasks:

1. Build an IFS with 4 affine transformations. Systematically vary contraction ratios from 0.3 to 0.7 in steps of 0.1. For which contraction ratios does the attractor look "connected" vs. "dusty"?
2. Add rotation to one transformation and vary the rotation angle from 0 to π. How does the shape change? Find the angle that produces the most symmetric attractor.
3. The Barnsley fern uses specific probability weights for each transformation. Vary the weight of the "stem" transformation (normally 0.01) from 0.001 to 0.1. How does the fern change?
4. Create your own IFS that produces a shape resembling a leaf, a tree, or a coastline. Document the transformation matrices and probabilities.

10.6.4 Topic 4 — Koch Generalizations

The Koch curve replaces each segment with 4 segments of ratio 1/3. We can generalize by changing the motif.

```
GeneralizedKoch[motif -> "tent", iterations -> 5]
GeneralizedKoch[motif -> "square", iterations -> 4]
```

Output:

[Two generalized Koch curves: one using the standard tent motif, one replacing each segment with three sides of a square.]

Your tasks:

1. Implement a generalized Koch curve where each segment is replaced by a custom motif (a list of relative endpoint positions). Compute its fractal dimension from the motif's contraction ratio.
2. For the standard Koch curve (dimension ≈ 1.262), verify that the perimeter diverges and the area converges as iteration increases.
3. Design a motif whose resulting fractal has dimension exactly 1.5. Verify computationally.
4. The "anti-snowflake" is built by replacing the tent with an inverted tent (pointing inward). Generate it and estimate its dimension. Is it the same as the regular Koch curve?

10.6.5 Topic 5 — Higher-Degree Mandelbrot Sets

The standard Mandelbrot set uses $z \mapsto z^2 + c$. The **Multibrot set** of degree d uses $z \mapsto z^d + c$.

```
MultibroSet[degree -> 3, res -> 400]
MultibroSet[degree -> 4, res -> 400]
MultibroSet[degree -> 5, res -> 400]
```

Output:

[Three Multibrot sets for degrees 3, 4, 5. Each has d-1 symmetry axes. Main body changes from cardioid (d=2) to more disk-like shapes for higher d.]

Your tasks:

1. Generate Multibrot sets for degrees 2 through 6. Count the number of primary bulbs attached to the main body. Conjecture the relationship between degree d and bulb count.
2. Measure the rotational symmetry of each Multibrot set. How does symmetry relate to d?
3. For the degree-3 Multibrot, zoom into several regions of the boundary. Do you see miniature copies of the degree-3 set? Or copies of the degree-2 (standard Mandelbrot) set?
4. The Burning Ship fractal (Topic 2) has degree 2. Implement the degree-3 Burning Ship $z \mapsto (|\mathrm{Re}(z)| + i|\mathrm{Im}(z)|)^3 + c$ and compare to the degree-2 version.

11 Chapter 11 — Tilings and Symmetry

11.1 Covering the Plane

A **tiling** (or **tessellation**) is a covering of the plane by non-overlapping shapes with no gaps. The floor you walk on, the pattern on a honeycomb, the brickwork on a wall — these are all tilings.

The mathematical questions are: which shapes can tile the plane? Can they do so in only one way, or many? Can a set of shapes tile the plane *without ever repeating* — an **aperiodic** tiling?

This chapter explores these questions computationally, culminating in the remarkable Penrose tiling — a simple two-tile system that covers the plane without any repeating pattern.

11.2 Regular and Semi-Regular Tilings

A **regular tiling** uses only one type of regular polygon and only one type of vertex configuration. There are exactly three:

- Equilateral triangles (6 meeting at each vertex)
- Squares (4 meeting at each vertex)
- Regular hexagons (3 meeting at each vertex)

A **semi-regular (Archimedean) tiling** uses two or more types of regular polygons, with the same vertex configuration everywhere. There are exactly 8.

```
RegularTiling["hexagon", 6]
SemiRegularTiling["3-4-6-4", 4]
```

Output:

[Left: hexagonal tiling colored in two alternating colors. Right: the 3-4-6-4 semi-regular tiling (triangles, squares, hexagons meeting at each vertex in the given order).]

11.3 Symmetry Groups

Every tiling has a **symmetry group** — the set of rigid motions (rotations, reflections, translations, glide reflections) that map the tiling to itself. For periodic tilings, the symmetry group is one of exactly 17 **wallpaper groups**.

```
TilingSymmetry["hexagonal"]
TilingSymmetry["square"]
```

Output:

[Diagrams showing the fundamental domain and symmetry axes for each tiling's wallpaper group. Hexagonal tiling has p6m symmetry (the richest); square tiling has p4m.]

11.4 Aperiodic Tilings: Penrose

In 1974, Roger Penrose discovered a set of just two tiles — a "kite" and a "dart" (or equivalently, a thick and thin rhombus) — that can tile the plane, but *only* aperiodically. No matter how you tile the plane with Penrose tiles following the matching rules, the resulting pattern never repeats with any translational period.

The two rhombus shapes used are: - **Thick rhombus:** angles 72° and 108° - **Thin rhombus:** angles 36° and 144°

Both have side length 1 and are related to the regular pentagon and the golden ratio ϕ.

```
PenroseTiling[6]
```

Output:

[Penrose tiling generated to subdivision level 6. Intricate pattern of thick and thin rhombuses with 5-fold local symmetry visible. No repeating patch of any size.]

11.5 Generating Penrose Tilings by Substitution

The Penrose tiling is generated by a **substitution rule**: each thick rhombus is replaced by 2 thick and 1 thin; each thin rhombus is replaced by 1 thick and 1 thin (in specific orientations). Applying this rule repeatedly generates arbitrarily large Penrose tilings.

```
PenroseSubstitution[4]
```

Output:

[Series of four images showing iterations 1 through 4 of the substitution rule. Each iteration subdivides all tiles into smaller copies according to the substitution.]

The ratio of thick to thin rhombuses in a Penrose tiling converges to $\phi = (1+\sqrt{5})/2 \approx 1.618\ldots$ — the golden ratio, embedded in the tiling's very structure.

```
PenroseRhombusRatio[8]
```

Output:

```
Iteration 1:  ratio = 1.000
Iteration 2:  ratio = 2.000
Iteration 3:  ratio = 1.500
Iteration 4:  ratio = 1.667
Iteration 5:  ratio = 1.600
Iteration 6:  ratio = 1.625
Iteration 7:  ratio = 1.615
Iteration 8:  ratio = 1.619
Converging to phi = 1.618...
```

11.6 Testing for Aperiodicity

A tiling is **periodic** if there exists a translation vector $\mathbf{v}$ such that translating the entire tiling by $\mathbf{v}$ maps every tile to another tile of the same type. For a Penrose tiling, no such $\mathbf{v}$ exists.

```
SearchForPeriod[PenroseTiling[6], maxShift -> 50]
```

Output:

```
No translational period found within search radius 50.
(This is consistent with aperiodicity, not a proof of it.)
```

11.7 Further Research Topics

11.7.1 Topic 1 — Vertex Configurations

```
PenroseVertexTypes[PenroseTiling[6]]
```

Output:

[Diagram showing all distinct local vertex configurations in the Penrose tiling, labeled with their frequency of occurrence.]

Your tasks:

1. In a Penrose tiling, enumerate all distinct vertex configurations (the local pattern of tiles around each vertex). How many are there? The known answer is 7 for the rhombus Penrose tiling — verify this.
2. For each vertex type, compute its frequency in a large Penrose tiling (level 8). Do the frequencies converge as the tiling grows?
3. Compare vertex configurations in the Penrose tiling to those in the hexagonal and square tilings. How many vertex types does each periodic tiling have?
4. Define the "vertex complexity" of a tiling as the number of distinct vertex configurations. Compare across: square, hexagonal, 3-4-6-4 semi-regular, and Penrose. Does aperiodicity require higher vertex complexity?

11.7.2 Topic 2 — Golden Ratio in Penrose Tilings

```
PenroseRhombusRatio[10]
PenroseEdgeLengthRatio[8]
```

Output:

[Two convergence plots. Both approach phi 1.618.]

Your tasks:

1. Compute the ratio of thick to thin rhombuses at each substitution level from 1 to 10. Verify convergence to ϕ.
2. In the Penrose tiling, many lengths appear. Identify all distinct lengths in a level-5 tiling (as multiples of the shortest edge). Are they all powers of ϕ?
3. The substitution matrix for the Penrose tiling is $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. Its dominant eigenvalue is ϕ. Verify this and explain the connection to the rhombus ratio.
4. Compare to a substitution tiling with substitution matrix $\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$. What is the dominant eigenvalue? Does this tiling also appear aperiodic?

11.7.3 Topic 3 — Design Your Own Substitution Tiling

```
CustomSubstitutionTiling[rules -> myRules, iterations -> 4]
```

Your tasks:

1. Design a substitution tiling with 3 tile types. Specify the substitution rules (how each tile divides into smaller tiles of the given types). Generate the tiling to depth 4.
2. Compute the substitution matrix (how many of each tile type appear when each tile is substituted). Find its dominant eigenvalue. What does this eigenvalue represent?
3. Test your tiling for aperiodicity by searching for translational periods in large patches.
4. Conjecture: what property of the substitution matrix guarantees aperiodicity? (Hint: look at the eigenvalue's algebraic properties — is it rational, quadratic irrational, or something else?)

11.7.4 Topic 4 — Chromatic Number of Tilings

The **chromatic number** of a tiling is the minimum number of colors needed to color the tiles so that no two adjacent tiles (sharing an edge) have the same color.

```
ColorTiling["hexagonal", colors -> 3]
ColorTiling[PenroseTiling[5], colors -> 4]
```

Output:

[Left: hexagonal tiling colored with 3 colors — the minimum. Right: Penrose tiling colored with 4 colors. Both are valid colorings.]

Your tasks:

1. Determine the chromatic number of each regular tiling (triangle, square, hexagon). Verify your answer with a valid coloring.
2. Determine the chromatic number of the Penrose tiling. Is it 3 or 4?
3. For the semi-regular tilings: does the chromatic number correlate with any geometric property (number of sides, vertex configuration)?
4. The four-color theorem says any planar map can be colored with 4 colors. Does any tiling require 4 colors? Which one?

11.7.5 Topic 5 — Cut-and-Project Method

The Penrose tiling can be constructed by projecting a 5-dimensional integer lattice onto a 2D plane at a specific irrational angle.

```
CutAndProject[dim -> 5, n -> 500]
```

Output:

[Penrose-like tiling generated by the cut-and-project method. Visually identical to the substitution-generated Penrose tiling.]

Your tasks:

1. Implement the cut-and-project method for a 3D integer lattice projected to 2D. Varying the projection angle, show that:
   - Rational angles give periodic tilings.
   - Irrational angles give aperiodic tilings.
2. Verify that the cut-and-project method with the correct 5D parameters produces a tiling visually identical to the substitution-based Penrose tiling.
3. The "windows" (or "acceptance domains") in the cut-and-project method determine which lattice points are included. For the Penrose tiling, the windows are pentagons. Implement this and visualize the windows.
4. Vary the projection angle slightly from the Penrose angle. At what angular deviation does the tiling first appear periodic?

Part IV

Part IV - The Experimental Method

12 Chapter 12 — When the Computer Misleads You

12.1 The Danger of Computational Evidence

Throughout this book, we have used the computer to discover patterns, verify identities, and test conjectures. The computer is an extraordinarily powerful tool for mathematical exploration. But it has fundamental limitations that every experimental mathematician must understand.

This chapter is about what can go wrong — and how to protect yourself against it. The goal is not to discourage computation but to make you a *skeptical* experimenter: one who knows when to trust a computation and when to demand more evidence.

There are two distinct failure modes:

1. **The pattern is simply false at large scale.** The computer can only check finitely many cases. A pattern that holds for millions of examples can still fail eventually.
2. **The computer's arithmetic is giving wrong answers.** Floating-point arithmetic introduces rounding errors that accumulate over many operations.

12.2 Patterns That Fail Eventually

12.2.1 The Pólya Conjecture

In 1919, the mathematician George Pólya conjectured: for every integer $n \geq 2$, at least half of the integers from 1 to n have an *odd* number of prime factors (counted with multiplicity). This held for every value tested — for decades.

In 1962, it was proved false. The first counterexample is $n = 906,150,257$.

```
PolyaCheck[10^6]
```

Output:

```
Polya's conjecture holds for all n from 2 to 10^6: True
```

So a million cases of computational evidence — and the conjecture is still false.

12.2.2 The Almost-Prime Polynomial

The polynomial $n^2 + n + 41$ is prime for $n = 0, 1, 2, \ldots, 39$. Forty consecutive prime values from a quadratic polynomial is remarkable.

```
PrimePolynomialRun[n^2 + n + 41]
```

Output:

```
n^2 + n + 41 produces primes for n = 0, 1, ..., 39
First composite: n = 40, value = 1681 = 41^2
```

Forty successes, then failure. If you only tested $n = 0, \ldots, 30$, you might have conjectured "this polynomial always gives primes" — and been wrong.

12.2.3 The False Pattern in π

The first 10^7 digits of π contain the digit sequence 0123456789. Does π contain every possible finite digit sequence? This is the **normal number conjecture** for π, and it remains unproven. The computer can only tell you that specific sequences appear in the digits you have computed.

12.3 Floating-Point Arithmetic

Every real number stored in a computer is approximated by a **floating-point number** — a finite binary decimal. Standard 64-bit floating point gives about 15–16 decimal digits of precision.

This causes problems when: - Subtracting two nearly equal numbers (**catastrophic cancellation**) - Accumulating small errors over many iterations - Computing limits or derivatives numerically

```
FloatingPointDemo[]
```

Output:

```
(1.0 + 10^-16) - 1.0 = 0.0    (* catastrophic cancellation *)
Sum[1/n^2, {n, 1, 10^7}] = 1.6449330...   (* correct: pi^2/6 = 1.6449340... *)
Error after 10^7 steps: 1.0 x 10^-6
```

The sum of $1/n^2$ is off by 10^{-6} after ten million terms. This is not a mistake — it is floating-point arithmetic working as designed, but the accumulated rounding error is visible.

12.4 Exact Arithmetic as Safeguard

Wolfram Language performs **exact arithmetic** by default for integers and rational numbers. This means calculations involving integers never accumulate rounding errors.

```
ExactVsFloating[]
```

Output:

```
Exact:    Sum[1/n^2, {n,1,1000}] = (huge exact fraction)
          N[%, 20] = 1.6439345666815599932
Floating: N[Sum[1/n^2, {n,1,1000}]] = 1.6439345666815597
Difference: 2 x 10^-16  (* floating-point error *)

Exact:    Fibonacci[10000] = (exact 2090-digit integer)
Floating: N[Fibonacci[10000]] differs in last few digits
```

Whenever you are testing an identity about integers — like the GCD identity for Fibonacci numbers, or Goldbach's conjecture — use exact arithmetic. Never test integer identities with floating-point numbers.

12.5 Catastrophic Cancellation: A Case Study

The formula $e^x \approx 1 + x$ for small x seems safe to evaluate directly. But for computing $e^x - 1$ when x is small, catastrophic cancellation destroys all precision.

```
CancellationDemo[10^-15]
```

Output:

```
Direct:    Exp[10^-15] - 1.0 = 1.11022 x 10^-15  (* only 3 sig figs correct *)
Exact WL: Exp[10^-15] - 1 = 10^-15 + (10^-30)/2 + ...  (* exact *)
Taylor:   x + x^2/2 + x^3/6 = 1.0 x 10^-15 + 5.0 x 10^-31 + ...
```

The built-in Wolfram Language function `Expm1[x]` (computing $e^x - 1$) handles this correctly using a numerically stable algorithm. The lesson: use library functions that are designed to avoid cancellation whenever possible.

12.6 The Strong Law of Small Numbers

The mathematician Richard Guy articulated a principle: **there are not enough small numbers to satisfy all the demands placed on them.** Small numbers appear in many patterns by coincidence, not by necessity. This makes it easy to mistake coincidence for law.

```
SmallNumberCoincidences[]
```

Output:

```
1 = 1^1 = 1^2 = 1^3 = ...    (trivially)
2^1 - 1 = 1 (prime)
2^2 - 1 = 3 (prime)
2^3 - 1 = 7 (prime)
2^5 - 1 = 31 (prime)
...but 2^11 - 1 = 2047 = 23 x 89 (NOT prime)

n^2 + n + 41 prime for n = 0,...,39 (then fails at n=40)

First 4 Fermat numbers 2^(2^n)+1 are prime... 5th is not (n=5)
```

Each of these looks like a pattern at small scale and breaks at larger scale.

12.7 Further Research Topics

12.7.1 Topic 1 — False Polynomial Patterns

```
PrimePolynomialSearch[maxRun -> 30]
```

Output:

[Table of quadratic polynomials n^2 + n + p that produce primes for long initial runs. Sorted by run length.]

Your tasks:

1. Find 5 quadratic polynomials of the form $n^2 + n + p$ (for prime p) that produce primes for at least 20 consecutive values of $n = 0, 1, ...$ For each, find the first n where it fails.
2. Conjecture: what property of the discriminant $b^2 - 4ac$ determines the length of the prime-producing run? Test your conjecture on 20 examples.
3. The polynomial $2n^2 + 29$ produces primes for $n = 0, \ldots, 28$. Verify this and find the first composite value.
4. Can you find a polynomial (of degree ≥ 3) that produces primes for more than 40 consecutive values?

12.7.2 Topic 2 — Floating-Point Drift

```
SinAccumulationDemo[10000]
```

Output:

[Plot of |sin(n) - sin_accumulated(n)| vs n for two methods: using built-in Sin[] vs. adding sin(1) repeatedly. Error grows roughly linearly.]

Your tasks:

1. Compute $\sin(n)$ for $n = 1, \ldots, 10000$ two ways: using the built-in `Sin[n]` each time, and using the addition formula $\sin(n) = 2\cos(1)\sin(n-1) - \sin(n-2)$ iteratively. Plot the error $|\sin_{\text{builtin}}(n) - \sin_{\text{accumulated}}(n)|$. How fast does the error grow?
2. Repeat in exact arithmetic using `N[Sin[n], 50]` (50-digit precision) and compare to 16-digit precision. At what n does the error in 16-digit arithmetic reach 1%?
3. The Mandelbrot set computation uses floating-point arithmetic. For a point near the boundary, compute its orbit using 16-digit, 32-digit, and 64-digit precision. At what iteration do the orbits first diverge?
4. Implement interval arithmetic: instead of a single floating-point value, track a guaranteed interval $[a, b]$ that contains the true value. Apply it to compute the first 20 Fibonacci numbers and verify the intervals remain tight.

12.7.3 Topic 3 — The Strong Law of Small Numbers

```
CollectSmallNumberPatterns[{FibSequence[20],
                            Table[Prime[n], {n,1,20}],
                            Table[2^n, {n,0,19}]}]
```

Your tasks:

1. From your earlier chapters, collect 10 patterns that held for all cases you tested. For each, either find a counterexample or prove (computationally, by exhaustion) that it holds up to a much larger bound.
2. The "almost-prime" polynomial $n^2 + n + 41$ fails at $n = 40$. Without knowing this in advance: if you had only tested $n = 0, \ldots, 20$, what confidence level would you assign to the conjecture? How should computational evidence be quantified?
3. The Fermat numbers $F_n = 2^{2^n} + 1$ are prime for $n = 0, 1, 2, 3, 4$ but composite for $n = 5$. Verify $F_5 = 4294967297 = 641 \times 6700417$ using exact arithmetic.
4. Find your own "misleading small pattern" — a sequence where the first 10 terms suggest one formula but the 11th term breaks it.

12.7.4 Topic 4 — Mersenne Non-Primes

```
MersenneCheck[Range[1, 30]]
```

Output:

[Table of p values, whether p is prime, and whether 2^p - 1 is prime. Several prime p give composite 2^p - 1.]

Your tasks:

1. The pattern "$2^p - 1$ is prime whenever p is prime" holds for $p = 2, 3, 5, 7$ then fails at $p = 11$ ($2^{11} - 1 = 2047 = 23 \times 89$). Verify this.
2. Among all primes $p \leq 100$: what fraction give Mersenne primes? Compute this ratio.
3. Use the Lucas-Lehmer test (Section 15.1.11) to verify all Mersenne primes up to $2^{100} - 1$.
4. The composite Mersenne number $2^{11} - 1 = 2047$ has factor 23. Verify that $23 \equiv 1 \pmod{22}$ (i.e., $23 \equiv \pm 1 \pmod{2 \cdot 11}$). This is a necessary condition for a prime q to divide $2^p - 1$. Verify it for all prime factors of composite Mersenne numbers you find.

12.7.5 Topic 5 — Euler's Product and Convergence Rate

The Euler product formula connects the Riemann zeta function to the primes:

$$\prod_{p \text{ prime}} \frac{1}{1 - p^{-s}} = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

For $s = 2$, the right side equals $\pi^2/6$.

```
EulerProductConvergence[s -> 2, maxPrimes -> 1000]
```

Output:

[Two convergence plots. Left: partial products up to the kth prime, compared to pi^2/6. Right: partial sums of the zeta series. Product converges much faster than the sum.]

Your tasks:

1. Compute the partial product $\prod_{p \leq P}(1 - p^{-2})^{-1}$ for $P = $ the first 10, 100, 500, 1000 primes. How many primes are needed to get 5 decimal places of $\pi^2/6 \approx 1.6449340$?
2. Compare convergence speed of the Euler product vs. the direct sum $\sum_{n=1}^{N} n^{-2}$. Which requires more terms for the same accuracy?

3. Repeat for $s = 4$ (where $\sum n^{-4} = \pi^4/90$). Does the relative speed advantage of the product hold?
4. The Euler product is valid only for $s > 1$. What happens numerically when you try $s = 1$ (the harmonic series)? Compute it for the first 10,000 terms and observe the divergence.

13 Chapter 13 — From Conjecture to Communication

13.1 Mathematics Is a Conversation

Every result in this book — the Collatz conjecture, Goldbach's conjecture, the normality of π — exists as a piece of public knowledge because someone wrote it down clearly and shared it. Mathematical ideas do not become part of the collective record until they are communicated.

This chapter is about how to do that: how to turn a computational experiment into a written conjecture, how to document your evidence honestly, and how to present your work to others.

13.2 The Anatomy of a Mathematical Conjecture

A well-stated conjecture has four components:

1. **Precise statement.** The conjecture says exactly what is claimed, with no ambiguity. It uses mathematical notation or careful English with defined terms.
2. **Domain.** The conjecture specifies what it applies to: "for all positive integers n", "for all primes p", "for n sufficiently large."
3. **Evidence.** The conjecture is supported by specific computational evidence: "verified for all $n \leq 10^6$", "holds for 1000 randomly chosen cases."
4. **Limitations.** The conjecture acknowledges what is *not* known: whether the evidence rules out all failure modes, whether it has been searched at large scale.

Bad conjecture: "It seems like the sequence always grows."

Good conjecture: "Conjecture: For all positive integers n, the Collatz stopping time satisfies $T(n) \leq 100 \log_2 n$. Evidence: verified computationally for all $n \leq 10^6$. The maximum ratio $T(n)/\log_2 n$ observed was 26.8 at $n = 837799$."

13.3 Documenting Computational Evidence

When you report computational evidence, include:

- **The exact computation performed** (not just "I tested many cases")
- **The range tested** ("all n from 1 to 10^6")
- **The result** ("conjecture holds for all tested values" or "first exception at $n = 42$")
- **Any qualifications** ("computation used floating-point arithmetic; exact verification was done for $n \leq 10^4$")

```
ConjectureReport[
  conjecture -> "Collatz stopping time < 100 Log2[n]",
  testRange -> {1, 10^6},
  testFunction -> Function[n, TotalStoppingTime[n] < 100*Log[2,n]],
  sampleExtremes -> True]
```

Output:

```
Conjecture: T(n) < 100 log2(n) for all positive integers n
Test range: n = 1 to 1000000
Result: True for all tested values
Maximum ratio T(n)/log2(n): 26.84 at n = 837799
Computation time: 43 seconds
Arithmetic: exact integer (stopping times), floating-point (log)
```

13.4 Writing Up a Research Report

A mathematical research report — even for a student — should contain:

1. Introduction - State the problem in plain language - Explain why it is interesting - State the main conjecture or result

2. Background - Define all notation - State any known results you are building on - Cite sources (OEIS entries, textbooks, papers)

3. Experiments - Describe what you computed and how - Present key plots and tables - Walk the reader through your reasoning

4. Conjecture or Result - State it precisely (see above) - Give evidence - State limitations clearly

5. Further Questions - What would it take to prove the conjecture? - What related questions remain open? - What would a larger computation tell you?

13.5 Effective Mathematical Visualizations

A good plot tells a story. Before making a plot, ask: what question does this answer?

Scatter plot: shows the relationship between two quantities. Good for stopping times vs. n, or gap size vs. prime.

Histogram: shows the distribution of a quantity. Good for prime gaps, Pisano periods, CF partial quotients.

Line plot: shows how a quantity evolves over time or iterations. Good for trajectories, convergence.

Heatmap / array plot: shows 2D structure. Good for Pascal's triangle mod p, cellular automata.

```
PlotGuidelines[]
```

Rules of thumb: - Label all axes - Include a title that states what the plot shows - Use color purposefully (not just decoration) - Annotate special features (e.g., "first counterexample at $n = 40$") - Show the scale (log vs. linear)

13.6 Pseudocode vs. Code

When writing up your work, you face a choice: include actual code, pseudocode, or neither?

Actual code is precise and reproducible — a reader can run it. But it may be too detailed and obscure the mathematical idea.

Pseudocode communicates the algorithm at the right level of abstraction. Use it to describe algorithms clearly:

```
Algorithm: Zeckendorf representation of n
1. Find the largest Fibonacci number F  n
2. Record F; subtract F from n
3. Find the largest Fibonacci number F'  n with F' not adjacent to F
4. Repeat until n = 0
```

Neither: for simple procedures, English prose is enough: "We applied the Collatz rule repeatedly until reaching 1 and counted the steps."

13.7 Where to Share Your Work

13.7.1 Competitions

- **Regeneron Science Talent Search (STS):** Annual US competition for high school students. Accepts mathematical research.
- **MIT PRIMES:** Research program for high school students working with MIT mathematicians.
- **ARML (American Regions Mathematics League):** Team competition.
- **Intel ISEF:** International fair accepting mathematical research.

13.7.2 Journals for Student Research

- *Mathematics Magazine* (MAA)
- *Pi Mu Epsilon Journal*
- *UMAP Journal* (applied mathematics)
- *Rose-Hulman Undergraduate Mathematics Journal* (college-level)

13.7.3 Preprint Servers

- **arXiv.org** (math section): where professional mathematicians post papers before journal publication
- **OEIS:** if you found a new integer sequence, submit it

13.7.4 Local Opportunities

- Science fair
- Math department at a local university
- Math circles and clubs

13.8 A Rubric for Evaluating Research Reports

Use this rubric to evaluate your own work before submitting:

Category	Strong	Adequate	Weak
Problem statement	Precisely stated, well-motivated	Stated but imprecise	Vague or missing
Background	Proper context, relevant prior work cited	Some context	No context
Experiments	Clearly described, reproducible	Described but incomplete	Not reproducible
Conjecture	Precisely stated with domain	Stated but imprecise	Only informal
Evidence	Specific ranges, qualifications	Some evidence	"I tested it"
Limitations	Explicitly acknowledged	Partially acknowledged	Not mentioned
Visualization	Labeled, purposeful, informative	Present but unlabeled	Absent or misleading
Further questions	Substantive, specific	Generic	Absent

13.9 A Sample Write-Up: Stopping Time Records

Here is an example of the kind of write-up you might produce from Chapter 2.

Title: Record Stopping Times in the Collatz Sequence

Introduction: The Collatz conjecture states that iterating the rule $C(n) = n/2$ (if n even) or $3n + 1$ (if n odd) always reaches 1. We investigate which starting values produce the longest trajectories.

Conjecture: We conjecture that the stopping time $T(n)$ satisfies $T(n) \leq 27\lceil \log_2 n \rceil$ for all $n \leq 10^6$.

Evidence: We computed $T(n)$ for all n from 1 to 10^6 using exact integer arithmetic. The maximum ratio $T(n)/\log_2 n$ was 26.84 at $n = 837799$ with $T(837799) = 524$. No value exceeded our bound of $27 \log_2 n$ in this range.

Limitations: This is verified only for $n \leq 10^6$. The bound $27 \log_2 n$ was chosen to fit the observed data and has no theoretical justification. A larger computation might reveal exceptions.

Further questions: Does the maximum ratio $T(n)/\log_2 n$ stay bounded as $n \to \infty$? The best known bound (from theory) is much weaker. Can we find a sharper empirical bound?

13.10 Summary: The Experimental Mathematics Workflow

This book has followed one workflow, applied to many problems:

1. **Start with curiosity.** Pick something concrete and ask: what happens?
2. **Compute.** Generate examples, sequences, plots.
3. **Notice patterns.** Look for regularities.
4. **State a conjecture.** Precisely, with a domain.
5. **Test it harder.** Larger range, edge cases, random cases.
6. **Question the computation.** Is the arithmetic exact? Are you measuring the right thing?
7. **Communicate.** Write it up as if a skeptical reader will check every step.

The computer is a lens, not an oracle. It lets you see farther and more clearly than unaided intuition. But what you see still requires human judgment to interpret — and human language to share.

This completes Part IV and the main text of the book. Turn to the appendices for Wolfram Language syntax, full function implementations, and guidance on using the OEIS.

Part V

Appendices

14 Appendix A — Wolfram Language Quick Reference

This appendix is a concise reference for the Wolfram Language features used throughout the book. It assumes no prior programming experience. Each section introduces one concept with examples you can type directly into a notebook.

14.1 A.1 Arithmetic and Basic Operations

Wolfram Language performs exact arithmetic by default.

```
2 + 3          (* 5 *)
10 / 3         (* 10/3  -- exact fraction, not 3.333... *)
10 // N        (* 3.333...  -- decimal approximation *)
N[10/3, 20]   (* 20 decimal digits *)
2^100          (* exact: 1267650600228229401496703205376 *)
Sqrt[2]        (* symbolic: Sqrt[2] *)
N[Sqrt[2], 50] (* 50-digit decimal *)
```

Key arithmetic functions:

Function	Meaning
`Mod[n, m]`	n mod m (remainder)
`Quotient[n, m]`	integer part of n/m
`GCD[a, b]`	greatest common divisor
`LCM[a, b]`	least common multiple
`Floor[x]`	$\lfloor x \rfloor$
`Ceiling[x]`	$\lceil x \rceil$
`Abs[x]`	$\lvert x \rvert$
`EvenQ[n]`	`True` if n is even
`OddQ[n]`	`True` if n is odd

Function	Meaning
`PrimeQ[n]`	`True` if n is prime

14.2 A.2 Lists

Lists are the fundamental data structure in Wolfram Language.

```
{1, 2, 3, 4, 5}           (* a list *)
Range[10]                  (* {1, 2, ..., 10} *)
Range[2, 10, 2]            (* {2, 4, 6, 8, 10} *)
Table[n^2, {n, 1, 8}]     (* {1, 4, 9, 16, 25, 36, 49, 64} *)

lst = {10, 20, 30, 40};
lst[[2]]                   (* 20  -- second element *)
lst[[2;;4]]                (* {20, 30, 40}  -- slice *)
Length[lst]                (* 4 *)
First[lst]                 (* 10 *)
Last[lst]                  (* 40 *)
Rest[lst]                  (* {20, 30, 40} -- all but first *)
Most[lst]                  (* {10, 20, 30} -- all but last *)
```

Common list operations:

Function	Meaning
`Map[f, lst]`	apply `f` to each element
`Select[lst, test]`	keep elements satisfying `test`
`Count[lst, x]`	count occurrences of `x`
`Total[lst]`	sum all elements
`Max[lst]`, `Min[lst]`	maximum, minimum
`Sort[lst]`	sort ascending
`Reverse[lst]`	reverse the list
`Union[lst]`	sorted unique elements
`Flatten[lst]`	remove nested list structure
`Partition[lst, k]`	split into non-overlapping blocks of length k
`Differences[lst]`	consecutive differences
`Accumulate[lst]`	cumulative sums
`AppendTo[lst, x]`	add `x` to the end (modifies `lst`)

14.3 A.3 Functions

Defining a function:

```
square[n_] := n^2
square[7]    (* 49 *)

(* Pattern with condition *)
factorial[0] := 1
factorial[n_Integer?Positive] := n * factorial[n - 1]

(* Anonymous (pure) function *)
#^2 &          (* same as square, written inline *)
(#^2 &)[7]     (* 49 *)
Map[#^2 &, Range[5]]   (* {1, 4, 9, 16, 25} *)
```

Key functional operations:

Function	Meaning
`Map[f, lst]`	`f` applied to each element
`/@`	shorthand for `Map`
`Nest[f, x, n]`	apply `f` to `x`, n times
`NestList[f, x, n]`	collect all intermediate values
`NestWhileList[f, x, test]`	apply until `test` fails
`Fold[f, x, lst]`	left fold: $f(f(f(x, l_1), l_2), l_3)$
`Apply[f, lst]`	`f[lst[[1]], lst[[2]], ...]`
`@@`	shorthand for `Apply`
`AllTrue[lst, test]`	True if `test` holds for all elements
`AnyTrue[lst, test]`	True if `test` holds for any element

14.4 A.4 Control Flow

```
(* If-then-else *)
If[EvenQ[6], "even", "odd"]   (* "even" *)

(* Which (multi-branch) *)
Which[
  n < 0, "negative",
  n == 0, "zero",
  True,   "positive"]

(* Do loop *)
Do[Print[i^2], {i, 1, 5}]

(* While loop *)
Module[{n = 1},
  While[n < 100, n = n * 2];
  n]    (* 128 *)

(* Switch *)
Switch[x,
  1, "one",
  2, "two",
  _, "other"]
```

14.5 A.5 Modules and Local Variables

`Module` creates local variables that do not affect the global state.

```
collatz[n_] := Module[{current = n, steps = 0},
  While[current != 1,
    current = If[EvenQ[current], current/2, 3*current + 1];
    steps++];
  steps]

collatz[27]   (* 111 *)
```

`steps` and `current` exist only inside the `Module` and are destroyed when it returns. This prevents accidental interference with other code.

14.6 A.6 Associations (Hash Maps)

Associations store key-value pairs with fast lookup.

```
a = <|"x" -> 1, "y" -> 2, "z" -> 3|>
a["x"]                    (* 1 *)
KeyExistsQ[a, "y"]        (* True *)
a["w"] = 99               (* add new key *)
Keys[a]                   (* {"x", "y", "z", "w"} *)
Values[a]                 (* {1, 2, 3, 99} *)
Counts[{1,2,1,3,2,1}]     (* <|1->3, 2->2, 3->1|> *)
```

Used in this book for: cycle detection in Collatz variants (Section 15.2.11), visited-node tracking in tree algorithms.

14.7 A.7 Plotting

```
(* Line plot of a function *)
Plot[Sin[x], {x, 0, 2*Pi}]

(* Plot a list of values *)
ListLinePlot[{1, 4, 9, 16, 25}]

(* Scatter plot *)
ListPlot[Table[{n, n^2}, {n, 1, 20}],
  AxesLabel -> {"n", "n^2"},
  PlotLabel -> "Squares"]

(* Histogram *)
Histogram[Table[RandomInteger[{1,6}], {1000}]]

(* Bar chart *)
BarChart[{3, 1, 4, 1, 5, 9}]
```

```
(* 2D array as image *)
ArrayPlot[Table[Mod[i+j, 2], {i,1,8}, {j,1,8}]]
```

Common options used in this book:

Option	Effect
`PlotLabel -> "title"`	adds a title
`AxesLabel -> {"x", "y"}`	labels the axes
`PlotStyle -> Blue`	sets line/point color
`PlotRange -> All`	shows all data (no clipping)
`ImageSize -> 400`	sets image width in pixels
`GridLines -> Automatic`	adds grid lines

14.8 A.8 String Operations

```
"Hello" <> " " <> "World"     (* "Hello World" -- concatenation *)
StringLength["Hello"]          (* 5 *)
StringSplit["a,b,c", ","]      (* {"a", "b", "c"} *)
StringJoin[{"a", "b", "c"}]    (* "abc" *)
ToString[42]                   (* "42" *)
ToExpression["42"]             (* 42 *)
IntegerDigits[1234]            (* {1, 2, 3, 4} *)
FromDigits[{1, 2, 3, 4}]       (* 1234 *)
```

14.9 A.9 Matrix and Linear Algebra

```
m = {{1, 2}, {3, 4}};
m . m                      (* matrix multiplication: {{7,10},{15,22}} *)
MatrixPower[m, 10]         (* m^10, computed efficiently *)
Inverse[m]                 (* {{-2, 1}, {3/2, -1/2}} *)
Det[m]                     (* -2 *)
```

```
Eigenvalues[m]           (* {3, -1} -- wait, actually ... *)
LinearSolve[m, {1, 2}]  (* solves m.x = {1,2} *)
```

14.10 A.10 Useful Built-in Constants and Functions

Symbol	Meaning
`Pi`	$\pi = 3.14159\ldots$
`E`	$e = 2.71828\ldots$
`GoldenRatio`	$\phi = 1.61803\ldots$
`I`	imaginary unit $\sqrt{-1}$
`Infinity`	∞
`Prime[n]`	nth prime
`PrimePi[x]`	$\pi(x)$, primes up to x
`Fibonacci[n]`	F_n
`Factorial[n]`	$n!$
`Binomial[n,k]`	$\binom{n}{k}$
`ContinuedFraction[x, n]`	first n partial quotients of x
`Convergents[cf]`	convergents from a CF list
`IntegerDigits[n, b]`	digits of n in base b
`RealDigits[x, b, n]`	first n digits of x in base b
`Reap[...]` / `Sow[x]`	efficient list building

15 Appendix B — Wolfram Language Function Reference

This appendix contains full implementations of every named function used in Chapters 1 through 7. Functions are organized by chapter. Cross-references in the text link directly to each entry.

Paste these definitions into a Wolfram Language notebook and evaluate them before running the chapter code.

15.1 Chapter 1 — Prime Numbers

15.1.1 `SieveOfEratosthenes`

Used in: Chapter 1

Returns all primes up to `n` using the Sieve of Eratosthenes.

```
SieveOfEratosthenes[n_Integer?Positive] :=
  Module[{composite = ConstantArray[False, n], primes = {}},
    Do[
      If[!composite[[i]],
        AppendTo[primes, i];
        Do[composite[[j]] = True,
           {j, 2i, n, i}]],
      {i, 2, n}];
    primes]
```

15.1.2 `PlotPrimeCounting`

Used in: Chapter 1

Plots the prime-counting function $\pi(x)$ and overlays $x/\ln x$.

```
PlotPrimeCounting[max_Integer?Positive] :=
  Module[{primes, piData},
    primes = SieveOfEratosthenes[max];
    piData = Accumulate[
      Table[If[MemberQ[primes, n], 1, 0], {n, 1, max}]];
    Show[
      ListLinePlot[piData,
        PlotStyle -> Blue,
        AxesLabel -> {"x", "\[Pi](x)"},
        PlotLabel -> "Prime Counting Function"],
      Plot[x/Log[x], {x, 2, max},
        PlotStyle -> {Red, Dashed}],
      PlotLegends -> {"\[Pi](x)", "x/ln(x)"}]]
```

15.1.3 `PrimeGaps`

Used in: Chapter 1

Returns the list of gaps between consecutive primes up to `n`.

```
PrimeGaps[n_Integer?Positive] :=
  Module[{primes = SieveOfEratosthenes[n]},
    Differences[primes]]
```

15.1.4 `PlotPrimeGapHistogram`

Used in: Chapter 1

Histogram of prime gaps for primes up to `n`.

```
PlotPrimeGapHistogram[n_Integer?Positive] :=
  Histogram[PrimeGaps[n],
    {2},
    PlotLabel -> "Prime Gaps up to " <> ToString[n],
    AxesLabel -> {"Gap size", "Count"},
    ChartStyle -> Directive[Blue, Opacity[0.7]]]
```

15.1.5 `VerifyGoldbach`

Used in: Chapter 1

Verifies Goldbach's conjecture for all even integers from 4 to `n`. Returns `True` if all can be expressed as a sum of two primes.

```
VerifyGoldbach[n_Integer?Positive] :=
  Module[{primeSet},
    primeSet = SieveOfEratosthenes[n];
    AllTrue[
      Range[4, n, 2],
      Function[k,
        AnyTrue[primeSet,
          Function[p, p <= k/2 && MemberQ[primeSet, k - p]]]]]]
```

15.1.6 `PlotGoldbachComet`

Used in: Chapter 1

Plots the number of Goldbach representations for each even integer up to `n`.

```
PlotGoldbachComet[n_Integer?Positive] :=
  Module[{primeSet, counts},
    primeSet = SieveOfEratosthenes[n];
    counts = Table[
      Count[
        Select[primeSet, # <= k/2 &],
        p_ /; MemberQ[primeSet, k - p]],
```

```
      {k, 4, n, 2}];
    ListPlot[counts,
      PlotLabel -> "Goldbach Comet",
      AxesLabel -> {"Even n", "# representations"},
      PlotStyle -> {PointSize[0.003], Blue}]]
```

15.1.7 `UlamSpiral`

Used in: Chapter 1

Generates and displays an $n \times n$ Ulam spiral with primes marked. `n` should be odd for a centered spiral.

```
UlamSpiral[n_Integer?Positive] :=
  Module[{size, grid, pos, dir, steps, turn, primeSet, k},
    size = n;
    grid = ConstantArray[0, {size, size}];
    pos = {Ceiling[size/2], Ceiling[size/2]};
    dir = {0, 1};    (* start moving right *)
    steps = 1; turn = 0; k = 1;
    Do[
      grid[[pos[[1]], pos[[2]]]] = k;
      pos = pos + dir;
      k++;
      turn++;
      If[turn == steps,
        dir = {dir[[2]], -dir[[1]]};  (* rotate left 90 deg *)
        turn = 0;
        If[OddQ[steps], steps++]],
      {size^2 - 1}];
    primeSet = With[{ps = SieveOfEratosthenes[size^2]},
      AssociationMap[True &, ps]];
    ArrayPlot[
      Map[If[KeyExistsQ[primeSet, #] && # > 1, 1, 0] &,
          grid, {2}],
      ColorRules -> {1 -> Black, 0 -> White},
      PlotLabel -> ToString[size] <> "\[Times]" <> ToString[size] <>
                   " Ulam Spiral"]]
```

15.1.8 `PrimeGapStats`

Used in: Section 1.7.1

Returns a table comparing mean gaps near various prime ranges to $\ln p$.

```
PrimeGapStats[n_Integer?Positive] :=
  Module[{primes = SieveOfEratosthenes[n], ranges, meanGap},
    ranges = {{10, 100}, {100, 1000}, {1000, 10000},
              {10000, 100000}, {100000, n}};
    Grid[
      Prepend[
        Map[Function[{lo, hi},
          Module[{ps = Select[primes, lo <= # <= hi &]},
            {lo, hi,
             If[Length[ps] < 2, "n/a",
               NumberForm[Mean[Differences[ps]], 4]],
             NumberForm[Log[Sqrt[lo * hi]], 4]}]],
          ranges],
        {"From", "To", "Mean gap", "ln(sqrt(midpoint))"}],
      Frame -> All]]
```

15.1.9 `ChebyshevRace`

Used in: Section 1.7.3

Plots the cumulative count of primes $\equiv 1 \pmod 4$ vs. $\equiv 3 \pmod 4$.

```
ChebyshevRace[n_Integer?Positive] :=
  Module[{primes, cum1, cum3},
    primes = SieveOfEratosthenes[n];
    cum1 = Accumulate[
      Table[If[Mod[p, 4] == 1, 1, 0],
            {p, Select[primes, # >= 5 &]}]];
    cum3 = Accumulate[
      Table[If[Mod[p, 4] == 3, 1, 0],
            {p, Select[primes, # >= 5 &]}]];
    ListLinePlot[{cum1, cum3},
      PlotLegends -> {"p \[Equiv] 1 (mod 4)", "p \[Equiv] 3 (mod 4)"},
      PlotLabel -> "Chebyshev's Bias",
```

```
      AxesLabel -> {"Index", "Cumulative count"},
      PlotStyle -> {Blue, Red}]]
```

15.1.10 `MersennePrimeCandidates`

Used in: Section 1.7.5

Returns a table of Mersenne candidates $2^p - 1$ for primes p up to `pMax`.

```
MersennePrimeCandidates[pMax_Integer?Positive] :=
  Module[{ps = SieveOfEratosthenes[pMax]},
    Grid[
      Prepend[
        Map[Function[p, {p, 2^p - 1, PrimeQ[2^p - 1]}], ps],
        {"p", "2^p - 1", "Prime?"}],
      Frame -> All]]
```

15.1.11 `LucasLehmerTest`

Used in: Section 1.7.5

Implements the Lucas-Lehmer primality test for Mersenne numbers $2^p - 1$.

```
LucasLehmerTest[p_Integer?Positive] :=
  Module[{M = 2^p - 1, s = 4},
    If[p == 2, Return[True]];
    Do[s = Mod[s^2 - 2, M], {p - 2}];
    s == 0]
```

15.2 Chapter 2 — The Collatz Conjecture

15.2.1 `CollatzStep`

Used in: Chapter 2

```
CollatzStep[n_Integer?Positive] :=
  If[EvenQ[n], n/2, 3n + 1]
```

15.2.2 `CollatzSequence`

Used in: Chapter 2

```
CollatzSequence[n_Integer?Positive] :=
  NestWhileList[CollatzStep, n, # != 1 &]
```

15.2.3 `FastStoppingTime`

Used in: Chapter 2, Section 15.2.6

```
FastStoppingTime[n_Integer?Positive] :=
  Module[{current = n, steps = 0},
    While[current != 1,
      current = CollatzStep[current];
      steps++];
    steps]
```

15.2.4 `TotalStoppingTime`

Used in: Chapter 2

```
TotalStoppingTime[n_Integer?Positive] :=
  Length[CollatzSequence[n]] - 1
```

15.2.5 `PlotCollatzSequence`

Used in: Chapter 2

```
PlotCollatzSequence[n_Integer?Positive] :=
  ListLinePlot[CollatzSequence[n],
    PlotLabel -> "Hailstone sequence: n = " <> ToString[n],
    AxesLabel -> {"Step", "Value"},
    PlotStyle -> {Blue, Thickness[0.002]},
    GridLines -> Automatic,
    GridLinesStyle -> LightGray]
```

15.2.6 `PlotStoppingTimes`

Used in: Chapter 2, Section 2.8.4

```
PlotStoppingTimes[max_Integer?Positive] :=
  Module[{times = Table[FastStoppingTime[n], {n, 1, max}]},
    ListPlot[times,
      PlotLabel -> "Stopping times: n = 1 to " <> ToString[max],
      AxesLabel -> {"n", "Stopping time"},
      PlotStyle -> {PointSize[If[max <= 1000, 0.006, 0.001]], Gray},
      PlotRange -> All]]
```

15.2.7 `DelayRecords`

Used in: Chapter 2, Section 2.8.3

```
DelayRecords[max_Integer?Positive] :=
  Module[{best = -1},
    Reap[
      Do[
        Module[{t = FastStoppingTime[n]},
          If[t > best, Sow[{n, t}]; best = t]],
        {n, 1, max}]][[2, 1]]]
```

15.2.8 `CollatzChildren`

Used in: Section 15.2.9

```
CollatzChildren[n_Integer?Positive] :=
  Module[{children = {2n}, candidate = (n - 1)/3},
    If[IntegerQ[candidate] && OddQ[candidate] &&
       candidate > 0 && Mod[n, 6] == 4,
      AppendTo[children, candidate]];
    children]
```

15.2.9 `PlotCollatzTree`

Used in: Chapter 2

```
PlotCollatzTree[depth_Integer?Positive] :=
  Module[{levels, edges},
    levels = NestList[
      Flatten[CollatzChildren /@ #] &,
      {1}, depth];
    edges = Flatten[
      Map[Function[n, Rule[n, #] & /@ CollatzChildren[n]],
          Flatten[Most[levels]]]];
    TreeGraph[edges,
      VertexLabels -> "Name",
      GraphLayout -> "LayeredDigraphEmbedding",
      ImageSize -> Large]]
```

15.2.10 `CollatzVariant`

Used in: Section 2.8.1, Section 2.8.2

```
CollatzVariant[q_Integer, r_Integer][n_Integer?Positive] :=
  If[EvenQ[n], n/2, q*n + r]
```

15.2.11 `CollatzSequenceGen`

Used in: Section 2.8.1, Section 2.8.2, Section 2.8.5

Produces a sequence for any step function `f`, stopping when 1 is reached or a value repeats.

```
CollatzSequenceGen[f_, n_Integer?Positive,
                   maxSteps_Integer: 10000] :=
  Module[{seen = <||>, current = n, seq = {n}},
    Do[
      If[current == 1, Return[{seq, "terminates"}]];
      If[KeyExistsQ[seen, current],
        Return[{seq, "cycle", current}]];
      seen[current] = True;
      current = f[current];
      AppendTo[seq, current],
      {maxSteps}];
    {seq, "diverges?"}]
```

15.2.12 `ClassifyHailstone`

Used in: Section 2.8.1

```
ClassifyHailstone[q_Integer, r_Integer][n_Integer?Positive] :=
  Module[{result = CollatzSequenceGen[CollatzVariant[q, r], n]},
    Switch[result[[2]],
      "terminates", "terminates",
      "cycle",      {"cycle", result[[3]], Length[result[[1]]]},
      _,            "diverges?"]]
```

15.2.13 `CollatzTable`

Used in: Section 2.8.5

```
CollatzTable[pVals_List, qVals_List, maxN_Integer: 200] :=
  Grid[
    Prepend[
      Table[
        Module[{f = CollatzVariant[q, p],
                outcomes},
          (* Note: here p is the divisor, q is the multiplier *)
          outcomes = CollatzSequenceGen[f, #][[2]] & /@ Range[1, maxN];
          {p, q, Counts[outcomes /. {_List :> "cycle"}]}],
        {p, pVals}, {q, qVals}],
      {"divisor p", "multiplier q", "outcome counts"}],
    Frame -> All]
```

Note: In this table, `p` is the divisor (divide by `p` when `p | n`) and `q` is the multiplier. The original Collatz map is `p=2, q=3`.

15.3 Chapter 3 — Fibonacci Numbers

15.3.1 `FibSequence`

Used in: Chapter 3

```
FibSequence[n_Integer?Positive] :=
  NestList[{#[[2]], #[[1]] + #[[2]]} &,
           {1, 1}, n - 1][[All, 1]]
```

15.3.2 `FibMatrix`

Used in: Chapter 3

Computes F_n using matrix exponentiation. Runs in $O(\log n)$ multiplications.

```
FibMatrix[n_Integer?Positive] :=
  MatrixPower[{{1, 1}, {1, 0}}, n][[1, 2]]
```

15.3.3 `VerifyFibIdentities`

Used in: Chapter 3

```
VerifyFibIdentities[n_Integer?Positive] :=
  Module[{fibs = FibSequence[n + 2]},
    {
     "Cassini's identity: " <>
       ToString[AllTrue[Range[2, n],
         Function[k, fibs[[k-1]] * fibs[[k+1]] - fibs[[k]]^2 ==
                     (-1)^k]]],
     "Hockey stick: " <>
       ToString[AllTrue[Range[1, n],
         Function[k, Total[fibs[[1;;k]]] == fibs[[k+2]] - 1]]],
     "Symmetry C(n,k)=C(n,n-k): verified separately via Binomial"
    }]
```

15.3.4 `ZeckendorfRepresentation`

Used in: Chapter 3

```
ZeckendorfRepresentation[n_Integer?Positive] :=
  Module[{remaining = n, fibs, result = {}},
    fibs = Reverse[Select[FibSequence[30], # <= n &]];
    Do[
      If[f <= remaining,
        AppendTo[result, f];
        remaining -= f],
      {f, fibs}];
    result]
```

15.3.5 `VerifyZeckendorfUniqueness`

Used in: Chapter 3

```
VerifyZeckendorfUniqueness[max_Integer?Positive] :=
  Module[{reps},
    reps = ZeckendorfRepresentation /@ Range[1, max];
    (* Check: no two consecutive Fibonacci numbers appear in any rep *)
    AllTrue[reps, Function[rep,
      Module[{fibs = FibSequence[25]},
        AllTrue[
          Partition[
            Map[Function[f, First[FirstPosition[fibs, f, {0}]]], rep],
            2, 1],
          Abs[#[[1]] - #[[2]]] >= 2 &]]]]
```

15.3.6 `PisanoPeriod`

Used in: Chapter 3

```
PisanoPeriod[m_Integer /; m >= 2] :=
  Module[{a = 1, b = 1, period = 0},
    While[True,
      {a, b} = {b, Mod[a + b, m]};
      period++;
      If[a == 1 && b == 1, Return[period]]]]
```

15.3.7 `PlotPisanoPeriods`

Used in: Chapter 3

```
PlotPisanoPeriods[max_Integer?Positive] :=
  ListPlot[
    Table[PisanoPeriod[m], {m, 2, max}],
    PlotLabel -> "Pisano Periods \[Pi](m), m = 2 to " <> ToString[max],
    AxesLabel -> {"m", "\[Pi](m)"},
    PlotStyle -> {PointSize[0.005], Blue}]
```

15.3.8 `FibRatios`

Used in: Chapter 3

```
FibRatios[n_Integer?Positive] :=
  Module[{fibs = FibSequence[n + 1]},
    N[fibs[[2;;]] / fibs[[;;-2]], 6]]
```

15.3.9 `TribonacciSequence`

Used in: Section 3.8.2

```
TribonacciSequence[n_Integer?Positive] :=
  Module[{seq = {1, 1, 1}},
    Do[AppendTo[seq, seq[[-1]] + seq[[-2]] + seq[[-3]]], {n - 3}];
    seq[[1;;n]]]
```

15.3.10 `FibonacciPrimes`

Used in: Section 3.8.3

```
FibonacciPrimes[maxIndex_Integer?Positive] :=
  Select[Range[1, maxIndex],
    Function[n, PrimeQ[FibMatrix[n]]]]
```

15.3.11 `LucasSequence`

Used in: Section 3.8.5

```
LucasSequence[n_Integer?Positive] :=
  Module[{seq = {1, 3}},
    Do[AppendTo[seq, seq[[-1]] + seq[[-2]]], {n - 2}];
    seq[[1;;n]]]
```

15.3.12 `VerifyGCDIdentity`

Used in: Section 3.8.4

```
VerifyGCDIdentity[trials_Integer?Positive] :=
  Module[{fibs},
    fibs = FibMatrix /@ Range[0, 200];
    AllTrue[
      RandomInteger[{1, 100}, {trials, 2}],
      Function[pair,
        Module[{m = pair[[1]], n = pair[[2]]},
          GCD[fibs[[m+1]], fibs[[n+1]]] ==
          fibs[[GCD[m, n] + 1]]]]]]
```

15.4 Chapter 4 — Digits, Bases, Normal Numbers

15.4.1 `DigitFrequency`

Used in: Chapter 4

Returns digit frequency histogram for the first `nDigits` digits of `x` in base `base`.

```
DigitFrequency[x_, base_Integer, nDigits_Integer] :=
  Module[{digits},
    digits = RealDigits[x, base, nDigits][[1]];
    BarChart[
      Counts[digits],
      ChartLabels -> Range[0, base - 1],
```

```
      PlotLabel -> "Digit frequencies (base " <>
                   ToString[base] <> ", " <>
                   ToString[nDigits] <> " digits)",
      AxesLabel -> {"Digit", "Count"},
      ChartStyle -> Blue]]
```

15.4.2 `NormalityTest`

Used in: Chapter 4

Chi-squared test for uniformity of digit distribution.

```
NormalityTest[x_, base_Integer, nDigits_Integer] :=
  Module[{digits, counts, expected, chiSq, df, pVal},
    digits = RealDigits[x, base, nDigits][[1]];
    counts = Values[Counts[digits] /. {_ -> 0}];
    counts = Table[Length[Select[digits, # == d &]], {d, 0, base - 1}];
    expected = nDigits / base;
    chiSq = Total[(counts - expected)^2 / expected];
    df = base - 1;
    pVal = 1 - CDF[ChiSquareDistribution[df], chiSq];
    {"chi-squared" -> N[chiSq, 4],
     "df" -> df,
     "p-value" -> N[pVal, 4],
     "uniform?" -> If[pVal > 0.05, "consistent", "rejected"]}]
```

15.4.3 `KaprekarRoutine`

Used in: Chapter 4

Applies Kaprekar's routine to an integer, returning the full trajectory.

```
KaprekarRoutine[n_Integer?Positive] :=
  Module[{current = n, seen = <||>, seq = {n}},
    While[True,
      Module[{digits = IntegerDigits[current, 10, 4],
```

```
            large, small},
        If[Length[Union[digits]] == 1,
          Return[Append[seq, 0]]];
        large = FromDigits[Sort[digits, Greater]];
        small = FromDigits[Sort[digits]];
        current = large - small;
        If[KeyExistsQ[seen, current], Return[seq]];
        seen[current] = True;
        AppendTo[seq, current]]]]
```

15.4.4 PlotKaprekarSteps

Used in: Chapter 4

```
PlotKaprekarSteps[nDigits_Integer?Positive] :=
  Module[{maxN = 10^nDigits - 1,
          steps},
    steps = Table[
      Length[KaprekarRoutine[n]] - 1,
      {n, 10^(nDigits-1), maxN}];
    Histogram[steps,
      PlotLabel -> "Steps to Kaprekar constant (" <>
                   ToString[nDigits] <> " digits)",
      AxesLabel -> {"Steps", "Count"},
      ChartStyle -> Blue]]
```

15.4.5 BenfordTest

Used in: Chapter 4

Tests a list of positive integers for Benford's law compliance.

```
BenfordTest[data_List] :=
  Module[{leadDigits, observed, theoretical},
    leadDigits = First[IntegerDigits[#]] & /@ Select[data, # > 0 &];
    observed = Table[
```

```
      N[Count[leadDigits, d] / Length[leadDigits], 4],
      {d, 1, 9}];
    theoretical = N[Log[10, 1 + 1/#] & /@ Range[1, 9], 4];
    BarChart[{observed, theoretical},
      ChartLabels -> {Placed[Range[1, 9], Below]},
      PlotLegends -> {"Observed", "Benford"},
      ChartStyle -> {Blue, Red},
      AxesLabel -> {"Leading digit", "Frequency"},
      PlotLabel -> "Benford's Law Test"]]
```

15.4.6 SelfDescribingCheck

Used in: Chapter 4

```
SelfDescribingCheck[n_Integer?Positive] :=
  Module[{digits = IntegerDigits[n], len},
    len = Length[digits];
    AllTrue[Range[0, len - 1],
      Function[k, Count[digits, k] == digits[[k + 1]]]]]
```

15.4.7 FindSelfDescribingNumbers

Used in: Chapter 4

```
FindSelfDescribingNumbers[maxDigits_Integer?Positive] :=
  Select[Range[1, 10^maxDigits - 1], SelfDescribingCheck]
```

15.4.8 DigitalRoot

Used in: Section 4.7.5

```
DigitalRoot[n_Integer?NonNegative] :=
  If[n < 10, n,
    DigitalRoot[Total[IntegerDigits[n]]]]
```

15.4.9 `MultiplicativePersistence`

Used in: Section 4.7.5

```
MultiplicativePersistence[n_Integer?Positive] :=
  Module[{current = n, steps = 0},
    While[current >= 10,
      current = Times @@ IntegerDigits[current];
      steps++];
    steps]
```

15.5 Chapter 5 — Pascal's Triangle

15.5.1 `PascalTriangle`

Used in: Chapter 5

```
PascalTriangle[n_Integer?Positive] :=
  Table[Binomial[row, k], {row, 0, n - 1}, {k, 0, row}]
```

15.5.2 `PascalMod`

Used in: Chapter 5, Section 15.5.4

Returns a 2D array of Pascal's triangle mod `p` for the first `n` rows, padded to a square array.

```
PascalMod[n_Integer?Positive, p_Integer?Positive] :=
  Module[{triangle},
    triangle = Table[Mod[Binomial[row, k], p],
                     {row, 0, n - 1}, {k, 0, n - 1}];
    triangle]
```

15.5.3 `PlotPascalMod`

Used in: Chapter 5

```
PlotPascalMod[n_Integer?Positive, p_Integer?Positive] :=
  Module[{data},
    data = Table[
      If[k <= row, Mod[Binomial[row, k], p], -1],
      {row, 0, n - 1}, {k, 0, n - 1}];
    ArrayPlot[data,
      ColorRules -> Join[
        Table[i -> ColorData["Rainbow"][i/p], {i, 0, p - 1}],
        {-1 -> White}],
      PlotLabel -> "Pascal's Triangle mod " <> ToString[p],
      ImageSize -> 400]]
```

15.5.4 `BoxCountingDimension`

Used in: Chapter 5, Section 5.5.1

Estimates box-counting dimension of a binary 2D array.

```
BoxCountingDimension[arr_] :=
  Module[{n = Length[arr], scales, counts, fit},
    scales = Table[2^k, {k, 1, Floor[Log2[n]] - 1}];
    counts = Map[Function[s,
      Module[{blocks},
        blocks = Partition[arr, {s, s}];
        Count[Flatten[Map[Max, blocks, {2}]], x_ /; x > 0]]],
      scales];
    fit = LinearModelFit[
      Transpose[{Log[n /@ scales], Log[counts]}],
      x, x];
    {fit["BestFitParameters"][[2]],
     ListLogLogPlot[Transpose[{n /@ scales, counts}],
       PlotLabel -> "Box-counting dimension \[TildeEqual] " <>
                    ToString[N[fit["BestFitParameters"][[2]], 4]],
       AxesLabel -> {"Box count N(\[Epsilon])",
                     "1/\[Epsilon]"}]}]
```

15.5.5 `VerifyPascalIdentities`

Used in: Chapter 5

```
VerifyPascalIdentities[n_Integer?Positive] :=
  Module[{fibs = FibSequence[n + 3]},
    {
     "Row sums = 2^n: " <>
       ToString[AllTrue[Range[0, n],
         Function[row, Total[Table[Binomial[row,k],{k,0,row}]] == 2^row]]],
     "Diagonal sums = Fibonacci: " <>
       ToString[AllTrue[Range[0, n],
         Function[d,
           Total[Table[If[r - k == d && k >= 0, Binomial[r,k], 0],
                       {r, 0, n}, {k, 0, r}]] == fibs[[d+1]]]]],
     "Symmetry C(n,k)=C(n,n-k): " <>
       ToString[AllTrue[Range[0, n],
         Function[row,
           AllTrue[Range[0, row],
             Function[k, Binomial[row,k] == Binomial[row, row-k]]]]]]
    }]
```

15.5.6 `KummerCarries`

Used in: Section 5.5.5

Counts the number of carries when adding `m` and `n` in base `p`.

```
KummerCarries[m_Integer, n_Integer, p_Integer?Positive] :=
  Module[{dm = IntegerDigits[m, p], dn = IntegerDigits[n, p],
          len, carry = 0, carries = 0},
    len = Max[Length[dm], Length[dn]];
    dm = PadLeft[dm, len]; dn = PadLeft[dn, len];
    Do[
      Module[{s = dm[[len - i + 1]] + dn[[len - i + 1]] + carry},
        If[s >= p, carries++; carry = 1, carry = 0]],
      {i, 1, len}];
    carries]
```

15.6 Chapter 6 — Continued Fractions

15.6.1 `CFExpansion`

Used in: Chapter 6

Returns the first `n` partial quotients of the continued fraction of `x`.

```
CFExpansion[x_, n_Integer?Positive] :=
  Module[{current = x, quotients = {}},
    Do[
      Module[{a = Floor[current]},
        AppendTo[quotients, a];
        If[current - a == 0, Return[quotients]];
        current = 1/(current - a)],
      {n}];
    quotients]
```

Note: For symbolic constants like `Pi` and `E`, Wolfram Language also has a built-in: `ContinuedFraction[Pi, n]`.

15.6.2 `CFFromList`

Used in: Chapter 6

Reconstructs a rational number from its list of partial quotients.

```
CFFromList[cf_List] :=
  Fold[Function[{acc, a}, a + 1/acc],
       Last[cf],
       Reverse[Most[cf]]]
```

15.6.3 `CFConvergents`

Used in: Chapter 6

Returns the list of convergents p_k/q_k.

```
CFConvergents[x_, n_Integer?Positive] :=
  Module[{cf = CFExpansion[x, n]},
    Table[CFFromList[cf[[1;;k]]], {k, 1, Length[cf]}]]
```

15.6.4 `CFApproximationErrors`

Used in: Chapter 6

Returns a table showing each convergent, its denominator, and the approximation quality $q^2|x - p/q|$.

```
CFApproximationErrors[x_, n_Integer?Positive] :=
  Module[{convs = CFConvergents[x, n]},
    Grid[
      Prepend[
        Map[Function[pq,
          Module[{p = Numerator[pq], q = Denominator[pq]},
            {pq, q,
             NumberForm[N[Abs[x - pq], 10], 4],
             NumberForm[N[q^2 * Abs[x - pq], 4], 4]}]],
          convs],
        {"p/q", "q", "|x - p/q|", "q^2 |x - p/q|"}],
      Frame -> All]]
```

15.6.5 `CFPeriod`

Used in: Chapter 6

Detects the periodic part of the CF of a quadratic irrational. Returns `{pre-period, period}`.

```
CFPeriod[x_] :=
  Module[{cf = ContinuedFraction[x, 50], seen = <||>},
    Do[
      If[KeyExistsQ[seen, cf[[i]]],
        Return[{cf[[1;;seen[cf[[i]]]-1]],
                cf[[seen[cf[[i]]];; i-1]]}]];
      seen[cf[[i]]] = i,
      {i, 1, Length[cf]}];
    {cf, {}}]
```

15.6.6 `GaussKuzminTest`

Used in: Chapter 6

Compares the partial quotient distribution to the Gauss-Kuzmin prediction.

```
GaussKuzminTest[x_, n_Integer?Positive] :=
  Module[{cf = ContinuedFraction[x, n],
          theoretical, observed},
    theoretical = Table[
      N[-Log[2, 1 - 1/(k+1)^2], 4],
      {k, 1, 10}];
    observed = Table[
      N[Count[cf, k] / Length[cf], 4],
      {k, 1, 10}];
    BarChart[{observed, theoretical},
      ChartLabels -> {Placed[Range[1, 10], Below]},
      PlotLegends -> {"Observed", "Gauss-Kuzmin"},
      ChartStyle -> {Blue, Red},
      AxesLabel -> {"Partial quotient", "Frequency"},
      PlotLabel -> "Gauss-Kuzmin Statistics"]]
```

15.6.7 `CompareApproximability`

Used in: Chapter 6, Section 6.8.2

```
CompareApproximability[constants_List, n_Integer?Positive] :=
  Module[{results},
    results = Map[Function[x,
      Module[{convs = CFConvergents[x, n]},
        Map[Function[pq,
          Module[{q = Denominator[pq]},
            N[q^2 * Abs[x - pq], 4]]],
          convs]]],
      constants];
    ListLinePlot[results,
      PlotLegends -> Map[ToString, constants],
      AxesLabel -> {"Convergent index", "q^2|x - p/q|"},
      PlotLabel -> "Approximation Quality"]]
```

15.6.8 `SternBrocotTree`

Used in: Section 6.8.4

```
SternBrocotTree[depth_Integer?Positive] :=
  Module[{nodes = {{0, 1}, {1, 0}}, edges = {}, level},
    Do[
      level = {};
      Do[
        Module[{med = {nodes[[i]][[1]] + nodes[[i+1]][[1]],
                       nodes[[i]][[2]] + nodes[[i+1]][[2]]}},
          AppendTo[edges,
            Fraction @@ nodes[[i]] -> Fraction @@ med];
          AppendTo[edges,
            Fraction @@ nodes[[i+1]] -> Fraction @@ med];
          AppendTo[level, med]],
        {i, 1, Length[nodes] - 1}];
      nodes = Flatten[
        Table[{nodes[[i]], level[[i]]}, {i, 1, Length[level]}],
        1];
      AppendTo[nodes, {1, 0}],
      {depth}];
    TreeGraph[edges,
      VertexLabels -> "Name",
      ImageSize -> Large]]
```

15.6.9 `PellFundamentalSolution`

Used in: Section 6.8.5

```
PellFundamentalSolution[d_Integer?Positive] :=
  Module[{cf = ContinuedFraction[Sqrt[d], 200], convs, r},
    r = Length[Last[ContinuedFraction[Sqrt[d]]]];
    convs = Convergents[cf];
    Module[{pq = convs[[r]]},
      {Numerator[pq], Denominator[pq]}]]
```

15.7 Chapter 7 — Integer Sequences and the OEIS

15.7.1 `DifferenceTable`

Used in: Chapter 7

Returns `levels` rows of iterated differences, starting from `seq`.

```
DifferenceTable[seq_List, levels_Integer?Positive] :=
  NestList[Differences, seq, levels]
```

15.7.2 `FindSequenceRecurrence`

Used in: Chapter 7

Attempts to identify a linear recurrence of order `maxOrder` satisfied by the sequence.

```
FindSequenceRecurrence[seq_List, maxOrder_Integer: 5] :=
  Module[{n = Length[seq], result = None},
    Do[
      Module[{ord = k,
              mat = Table[seq[[i + j - 1]], {i, 1, n - k}, {j, 1, k}],
              rhs = seq[[k+1;;n]]},
        Module[{sol = LinearSolve[mat, rhs]},
          If[Head[sol] =!= LinearSolve &&
             AllTrue[sol, IntegerQ],
            result = sol;
            Return[]]]],
      {k, 1, maxOrder}];
    result]
```

15.7.3 `RecamanSequence`

Used in: Chapter 7

```
RecamanSequence[n_Integer?Positive] :=
  Module[{seq = {0}, seen = <|0 -> True|>},
    Do[
      Module[{prev = Last[seq],
              candidate = Last[seq] - i},
        If[candidate > 0 && !KeyExistsQ[seen, candidate],
          AppendTo[seq, candidate];
          seen[candidate] = True,
          AppendTo[seq, prev + i];
          seen[prev + i] = True]],
      {i, 1, n - 1}];
    seq]
```

15.7.4 `PlotRecaman`

Used in: Chapter 7

Plots Recamán's sequence as arcs on a number line.

```
PlotRecaman[n_Integer?Positive] :=
  Module[{seq = RecamanSequence[n], arcs, above = True},
    arcs = Table[
      Module[{a = seq[[i]], b = seq[[i+1]],
              mid, r, side},
        mid = (a + b) / 2.0;
        r = Abs[b - a] / 2.0;
        side = If[EvenQ[i], 1, -1];  (* alternate above/below *)
        {Black, Circle[{mid, 0}, r,
                       If[side == 1, {0, Pi}, {-Pi, 0}]]}],
      {i, 1, n - 1}];
    Graphics[
      Join[
        {{Gray, Line[{{0, 0}, {Max[seq] + 1, 0}}]}},
        arcs,
        Map[{Black, Text[#, {#, -0.3}]} &,
            Range[0, Max[seq], 5]]],
```

```
      PlotRange -> {{-1, Max[seq] + 1}, {-Max[seq]/5, Max[seq]/5}},
      ImageSize -> 800]]
```

15.7.5 `RecamanMissing`

Used in: Chapter 7, Section 7.8.3

```
RecamanMissing[n_Integer?Positive] :=
  Module[{seq = RecamanSequence[n]},
    Complement[Range[1, Max[seq]], seq]]
```

15.7.6 `LookAndSay`

Used in: Chapter 7

Returns the first `n` terms of the look-and-say sequence starting from "1".

```
LookAndSayStep[s_String] :=
  StringJoin[
    Map[Function[run,
      ToString[Length[run]] <> StringTake[run, 1]],
      StringCases[s, RegularExpression["(.)\\1*"]]]]

LookAndSay[n_Integer?Positive] :=
  NestList[LookAndSayStep, "1", n - 1]
```

15.7.7 `LookAndSayGrowthRate`

Used in: Chapter 7

```
LookAndSayGrowthRate[n_Integer?Positive] :=
  Module[{terms = LookAndSay[n],
          lengths},
    lengths = StringLength /@ terms;
    ListLinePlot[
      Partition[N[lengths[[2;;]] / lengths[[;;-2]]], 1],
      PlotLabel -> "Look-and-say length growth ratios",
      AxesLabel -> {"Step", "Length ratio"},
      PlotStyle -> Blue,
      Epilog -> {Red, Dashed,
        InfiniteLine[{{1, 1.303577}, {2, 1.303577}}]}]]
```

15.7.8 `GenerateAndSearch`

Used in: Chapter 7

Formats a sequence for pasting into the OEIS search box.

```
GenerateAndSearch[seq_List] :=
  Module[{terms = Select[seq, IntegerQ]},
    Print["Paste into oeis.org: " <>
          StringRiffle[ToString /@ terms, ", "]];
    terms]
```

15.7.9 `IteratedDifferences`

Used in: Section 7.8.1

```
IteratedDifferences[seq_List, levels_Integer?Positive] :=
  Module[{rows = NestList[Differences, seq, levels]},
    Grid[
      Map[Function[row,
        PadRight[row, Length[seq], ""]],
        rows],
      Frame -> All,
      Background -> {None, {LightBlue, {White}}}]]
```

15.7.10 `SequenceEntropy`

Used in: Section 7.8.5

Shannon entropy of k-gram distribution in a sequence.

```
SequenceEntropy[seq_List, maxK_Integer?Positive] :=
  Table[
    Module[{ngrams = Partition[seq, k, 1],
            counts, probs},
      counts = Values[Counts[ngrams]];
      probs = counts / Total[counts];
      -Total[probs * Log[2, probs]]],
    {k, 1, maxK}]
```

15.8 Chapter 8 — Cellular Automata

15.8.1 `ShowRule`

Used in: Chapter 8

Displays the neighborhood diagram for an elementary cellular automaton rule.

```
ShowRule[rule_Integer /; 0 <= rule <= 255] :=
  Module[{bits = IntegerDigits[rule, 2, 8]},
    GraphicsGrid[
      {Table[
        Graphics[{
          {If[BitGet[n, 2] == 1, Black, White],
           Rectangle[{0, 1}, {1, 2}]},
          {If[BitGet[n, 1] == 1, Black, White],
           Rectangle[{1, 1}, {2, 2}]},
          {If[BitGet[n, 0] == 1, Black, White],
           Rectangle[{2, 1}, {3, 2}]},
```

```
          {If[bits[[8 - n]] == 1, Black, White],
           Rectangle[{0.5, -0.2}, {2.5, 0.8}]},
          {Gray, Line[{{0,0.9},{3,0.9}}]}},
          PlotRange -> {{-0.2, 3.2}, {-0.3, 2.1}},
          ImageSize -> 60],
        {n, 7, 0, -1}]},
      PlotLabel -> "Rule " <> ToString[rule]]]
```

15.8.2 `CellularAutomatonPlot`

Used in: Chapter 8

Generates and plots a space-time diagram for elementary CA rule `rule`, starting from a single active cell, for `steps` generations.

```
CellularAutomatonPlot[rule_Integer, steps_Integer?Positive] :=
  Module[{init, ca},
    init = PadLeft[{1}, 2*steps + 1, 0];
    ca = CellularAutomaton[rule, {init, 0}, steps];
    ArrayPlot[ca,
      ColorRules -> {1 -> Black, 0 -> White},
      PlotLabel -> "Rule " <> ToString[rule],
      ImageSize -> 500]]
```

Note: Wolfram Language's built-in `CellularAutomaton` handles the computation; this function wraps it with a standard display.

15.8.3 `Rule30CenterColumn`

Used in: Chapter 8

Returns the center column of the Rule 30 space-time diagram.

```
Rule30CenterColumn[steps_Integer?Positive] :=
  Module[{init = PadLeft[{1}, 2*steps + 1, 0],
          ca},
    ca = CellularAutomaton[30, {init, 0}, steps];
    ca[[All, steps + 1]]]
```

15.8.4 `Rule30RandomnessTests`

Used in: Chapter 8

Runs basic randomness tests on the Rule 30 center column.

```
Rule30RandomnessTests[n_Integer?Positive] :=
  Module[{bits = Rule30CenterColumn[n], freq, runs, ac},
    freq = N[Count[bits, 1] / n, 4];
    runs = Length[Split[bits]];
    ac = N[Total[bits[[1;;-2]] * bits[[2;;]]] / n, 4];
    {
     "Fraction of 1s: " <> ToString[freq],
     "Number of runs: " <> ToString[runs],
     "Autocorrelation (lag 1): " <> ToString[ac]
    }]
```

15.8.5 `WolframEquivalenceClasses`

Used in: Chapter 8

Partitions all 256 CA rules into equivalence classes under left-right reflection and color inversion.

```
WolframEquivalenceClasses[] :=
  Module[{mirror, invert, rules = Range[0, 255], classes},
    mirror[r_] := Module[{bits = IntegerDigits[r, 2, 8]},
      FromDigits[bits[[{1,3,2,4,5,7,6,8}]], 2]];
    invert[r_] := Module[{bits = IntegerDigits[r, 2, 8]},
      FromDigits[1 - bits[[{8,7,6,5,4,3,2,1}]], 2]];
```

```
    GatherBy[rules, Sort[{#, mirror[#], invert[#],
                          invert[mirror[#]]}] &]]
```

15.8.6 `GameOfLifeSteps`

Used in: Section 8.7.4

Runs Conway's Game of Life for `steps` generations on an initial configuration `init` (a 2D binary array).

```
GliderPattern[] :=
  {{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
   {0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
   {0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
   {0,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
   Table[0, {16}],
   Table[0, {16}],
   Table[0, {16}],
   Table[0, {16}]};

GameOfLifeStep[grid_] :=
  CellularAutomaton[
    {224, 2, {1, 1}},   (* outer totalistic Game of Life rule *)
    {grid, 0}];

GameOfLifeSteps[init_, steps_Integer?Positive] :=
  ArrayPlot[#,
    ColorRules -> {1 -> Black, 0 -> White},
    ImageSize -> 200] & /@
  NestList[GameOfLifeStep, init, steps]
```

15.9 Chapter 9 — The Logistic Map

15.9.1 `LogisticOrbit`

Used in: Chapter 9

Plots the orbit of the logistic map for given `r`, starting at `x0`.

```
LogisticOrbit[r_?NumericQ, x0_?NumericQ, n_Integer?Positive] :=
  Module[{orbit},
    orbit = NestList[r * # * (1 - #) &, x0, n];
    ListLinePlot[orbit,
      PlotLabel -> "Logistic orbit: r = " <> ToString[N[r,4]],
      AxesLabel -> {"Step", "x"},
      PlotStyle -> Blue,
      PlotRange -> {0, 1}]]
```

15.9.2 `BifurcationDiagram`

Used in: Chapter 9

Plots the bifurcation diagram for the logistic map over an r-range. `rRange` is `{rMin, rMax}`, `nPoints` is the number of r values.

```
BifurcationDiagram[rRange_List, nPoints_Integer?Positive] :=
  Module[{rVals, warmup = 500, sample = 200, pts},
    rVals = Subdivide[rRange[[1]], rRange[[2]], nPoints];
    pts = Flatten[
      Table[
        Module[{x = 0.5},
          Do[x = r * x * (1 - x), {warmup}];
          Table[{r, (x = r * x * (1 - x))}, {sample}]],
        {r, rVals}],
      1];
    ListPlot[pts,
      PlotStyle -> {PointSize[0.001], Black},
      PlotRange -> {All, {0, 1}},
      AxesLabel -> {"r", "x (long-term)"},
      PlotLabel -> "Bifurcation Diagram",
      ImageSize -> 600]]
```

15.9.3 `FeigenbaumEstimate`

Used in: Chapter 9

Estimates bifurcation points and the Feigenbaum constant.

```
BifurcationSearch[period_Integer, rLow_?NumericQ, rHigh_?NumericQ,
                  tol_: 10^-8] :=
  Module[{rMid, x, check},
    check[r_] := Module[{xx = 0.5},
      Do[xx = r * xx * (1 - xx), {1000}];
      Module[{orbit = NestList[r * # * (1 - #) &, xx, 2*period]},
        Abs[orbit[[1]] - orbit[[period + 1]]] < 0.001 &&
        Abs[orbit[[1]] - orbit[[2*period + 1]]] < 0.001]];
    While[rHigh - rLow > tol,
      rMid = (rLow + rHigh) / 2;
      If[check[rMid], rLow = rMid, rHigh = rMid]];
    N[(rLow + rHigh)/2, 8]]

FeigenbaumEstimate[n_Integer?Positive] :=
  Module[{bifPoints, intervals, ratios},
    bifPoints = Table[
      BifurcationSearch[2^k, 2.8, 3.9],
      {k, 0, n - 1}];
    intervals = Differences[bifPoints];
    ratios = Most[intervals] / Rest[intervals];
    Grid[
      Prepend[
        Table[{k, bifPoints[[k+1]],
               If[k < Length[intervals], intervals[[k]], ""],
               If[k < Length[ratios], N[ratios[[k]], 6], ""]},
              {k, 1, n}],
        {"k", "r_k", "Interval", "Ratio"}],
      Frame -> All]]
```

15.9.4 `LyapunovExponent`

Used in: Chapter 9, Section 9.7.3

Computes the Lyapunov exponent for the logistic map at a given `r`.

```
LyapunovExponent[r_?NumericQ, n_Integer: 10000] :=
  Module[{x = 0.5, lyap = 0},
    Do[x = r * x * (1 - x);
       lyap += Log[Abs[r * (1 - 2*x)]],
       {n}];
    N[lyap / n, 6]]
```

15.9.5 `LyapunovPlot`

Used in: Section 9.7.3

Plots the Lyapunov exponent as a function of `r`.

```
LyapunovPlot[rRange_List, nPoints_Integer?Positive] :=
  Module[{rVals, lyaps},
    rVals = Subdivide[rRange[[1]], rRange[[2]], nPoints];
    lyaps = LyapunovExponent /@ rVals;
    ListLinePlot[
      Transpose[{rVals, lyaps}],
      PlotLabel -> "Lyapunov Exponent",
      AxesLabel -> {"r", "\[Lambda]"},
      PlotStyle -> Blue,
      Epilog -> {Red, Dashed,
        InfiniteLine[{{3.0, 0}, {4.0, 0}}]},
      PlotRange -> {All, {-3, 1}}]]
```

15.9.6 `ChaosDemo`

Used in: Chapter 9

Demonstrates sensitive dependence on initial conditions.

```
ChaosDemo[r_?NumericQ, x0_?NumericQ,
          epsilon_?NumericQ, n_Integer?Positive] :=
  Module[{orbit1, orbit2},
    orbit1 = NestList[r * # * (1 - #) &, x0, n];
```

```
    orbit2 = NestList[r * # * (1 - #) &, x0 + epsilon, n];
    ListLinePlot[{orbit1, orbit2},
      PlotLabel -> "Chaos: r=" <> ToString[N[r,4]] <>
                   ", \[Epsilon]=" <> ToString[epsilon],
      AxesLabel -> {"Step", "x"},
      PlotLegends -> {"x0", "x0 + \[Epsilon]"},
      PlotStyle -> {Blue, Red}]]
```

15.9.7 `ZoomBifurcation`

Used in: Chapter 9

Zoomed bifurcation diagram for a narrow r-range.

```
ZoomBifurcation[rRange_List, nPoints_Integer?Positive] :=
  BifurcationDiagram[rRange, nPoints]
```

15.10 Chapter 10 — Fractal Geometry

15.10.1 `SierpinskiIFS`

Used in: Chapter 10

Generates the Sierpiński triangle via iterated function system to the given iteration depth.

```
SierpinskiTransforms[] := {
  AffineTransform[{{0.5, 0}, {0, 0.5}}, {0, 0}],
  AffineTransform[{{0.5, 0}, {0, 0.5}}, {0.5, 0}],
  AffineTransform[{{0.5, 0}, {0, 0.5}}, {0.25, 0.5}]}

SierpinskiIFS[n_Integer?Positive] :=
  Module[{pts = {{0,0},{1,0},{0.5,0.866}}},
    Do[
```

```
      pts = Flatten[
        Map[Function[p,
          Map[Through[SierpinskiTransforms[][#]][p] &,
              Range[3]]],
          pts], 1],
        {n}];
    Graphics[{PointSize[0.002], Blue,
              Point[pts]},
             PlotLabel -> "Sierpinski Triangle (level " <>
                          ToString[n] <> ")"]]
```

15.10.2 `ChaosGame`

Used in: Chapter 10

Generates a fractal via the chaos game algorithm.

```
ChaosGame[transforms_List, n_Integer?Positive,
          probs_List: {}] :=
  Module[{k = Length[transforms],
          p = If[probs == {}, ConstantArray[1/Length[transforms],
                                            Length[transforms]], probs],
          pt = {0.5, 0.5}, pts = {}},
    Do[
      Module[{i = RandomChoice[p -> Range[k]]},
        pt = transforms[[i]][pt];
        AppendTo[pts, pt]],
      {n}];
    Graphics[{PointSize[0.001], Blue, Point[pts]},
             ImageSize -> 400]]
```

15.10.3 `BarnsleyFern`

Used in: Chapter 10

```
BarnsleyFern[n_Integer?Positive] :=
  Module[{transforms = {
    AffineTransform[{{0, 0}, {0, 0.16}}, {0, 0}],
    AffineTransform[{{0.85, 0.04}, {-0.04, 0.85}}, {0, 1.6}],
    AffineTransform[{{0.2, -0.26}, {0.23, 0.22}}, {0, 1.6}],
    AffineTransform[{{-0.15, 0.28}, {0.26, 0.24}}, {0, 0.44}]},
    probs = {0.01, 0.85, 0.07, 0.07}},
  ChaosGame[transforms, n, probs]]
```

15.10.4 `KochSnowflake`

Used in: Chapter 10

Generates the Koch snowflake at iteration level `n`.

```
KochCurvePoints[pts_List] :=
  Flatten[
    Map[Function[{p1, p2},
      Module[{d = p2 - p1,
              a = p1 + (p2 - p1)/3,
              b = p1 + 2*(p2 - p1)/3,
              tip},
        tip = a + RotationMatrix[Pi/3] . (b - a);
        {p1, a, tip, b}]],
      Partition[pts, 2, 1]],
    1]

KochSnowflake[n_Integer?Positive] :=
  Module[{pts = {{0,0},{1,0},{0.5, Sqrt[3]/2},{0,0}}},
    Do[pts = Append[KochCurvePoints[pts], Last[pts]], {n}];
    Graphics[Line[pts], PlotLabel -> "Koch Snowflake (level " <>
                                      ToString[n] <> ")"]]
```

15.10.5 `KochPerimeterArea`

Used in: Chapter 10

```
KochPerimeterArea[n_Integer?Positive] :=
  Module[{perimeter = 3.0, area = Sqrt[3]/4},
    Grid[
      Prepend[
        Table[
          perimeter = perimeter * 4/3;
          area = area + 3 * (1/9)^k * (Sqrt[3]/4) * 4^(k-1);
          {k, N[perimeter, 6], N[area, 6]},
          {k, 1, n}],
        {"Iteration", "Perimeter", "Area"}],
      Frame -> All]]
```

15.10.6 `MandelbrotPlot`

Used in: Chapter 10

Plots the Mandelbrot set over the given coordinate ranges.

```
MandelbrotPlot[xRange_List, yRange_List,
               maxIter_Integer, res_Integer] :=
  Module[{xs, ys, escape},
    xs = Subdivide[xRange[[1]], xRange[[2]], res];
    ys = Subdivide[yRange[[1]], yRange[[2]], res];
    escape = Table[
      Module[{c = x + I*y, z = 0, i = 0},
        While[i < maxIter && Abs[z] <= 2,
          z = z^2 + c; i++];
        i],
      {y, Reverse[ys]}, {x, xs}];
    ArrayPlot[escape,
      ColorFunction -> "Rainbow",
      PlotLabel -> "Mandelbrot Set",
      ImageSize -> 400]]
```

15.10.7 `JuliaSet`

Used in: Section 10.6.2

```
JuliaSet[cRe_?NumericQ, cIm_?NumericQ,
         res_Integer: 400, maxIter_Integer: 200] :=
  Module[{c = cRe + I*cIm, xs, ys, escape},
    xs = Subdivide[-1.5, 1.5, res];
    ys = Subdivide[-1.5, 1.5, res];
    escape = Table[
      Module[{z = x + I*y, i = 0},
        While[i < maxIter && Abs[z] <= 2,
          z = z^2 + c; i++];
        i],
      {y, Reverse[ys]}, {x, xs}];
    ArrayPlot[escape,
      ColorFunction -> "Rainbow",
      PlotLabel -> "Julia Set: c = " <>
                   ToString[cRe] <> " + " <> ToString[cIm] <> "i",
      ImageSize -> 400]]
```

15.10.8 `FractalDimension`

Used in: Chapter 10

Box-counting dimension estimate for a binary Graphics object (rasterized to a grid).

```
FractalDimension[g_Graphics, res_Integer: 512] :=
  Module[{raster, scales, counts},
    raster = ImageData[
      Rasterize[g, "Image", ImageSize -> res],
      "Bit"];
    raster = 1 - raster;  (* invert: fractal = 1 *)
    scales = Table[2^k, {k, 1, Floor[Log2[res]] - 1}];
    counts = Map[Function[s,
      Module[{blocks = Partition[raster, {s, s}, {s, s}, {}, 0]},
        Count[Flatten[Map[Max, blocks, {2}]], 1]]],
      scales];
    Module[{pairs = Transpose[{Log[N[res/scales]], Log[N[counts]]}],
            fit},
```

```
      fit = LinearModelFit[pairs, x, x];
      N[fit["BestFitParameters"][[2]], 4]]
```

15.11 Chapter 11 — Tilings and Symmetry

15.11.1 `PenroseTiling`

Used in: Chapter 11

Generates a Penrose rhombus tiling to subdivision level `n` using the substitution rule.

```
(* Each tile represented as {type, {v1, v2, v3}} where type in {thick, thin} *)
(* Initial configuration: 10 thick rhombuses arranged in a ring *)

PenroseSubstituteThick[{v1_, v2_, v3_}] :=
  Module[{phi = GoldenRatio,
          p = v1 + (v2 - v1)/phi,
          q = v2 + (v3 - v2)/phi},
    {{"thick", {q, v1, p}},
     {"thick", {p, v2, q}},
     {"thin",  {q, v3, p}}}]

PenroseSubstituteThin[{v1_, v2_, v3_}] :=
  Module[{phi = GoldenRatio,
          p = v1 + (v2 - v1)/phi},
    {{"thin",  {p, v3, v1}},
     {"thick", {p, v2, v3}}}]

PenroseSubstituteAll[tiles_List] :=
  Flatten[
    Map[Function[t,
      If[t[[1]] == "thick",
         PenroseSubstituteThick[t[[2]]],
         PenroseSubstituteThin[t[[2]]]]],
      tiles], 1]
```

```
PenroseInitial[] :=
  Table[
    {"thick",
     {
       {0, 0},
       {Cos[2*Pi*k/5], Sin[2*Pi*k/5]},
       {Cos[2*Pi*(k+1)/5], Sin[2*Pi*(k+1)/5]}
     }},
    {k, 0, 4}]

PenroseTiling[n_Integer?Positive] :=
  Module[{tiles = PenroseInitial[]},
    Do[tiles = PenroseSubstituteAll[tiles], {n}];
    Graphics[
      Map[Function[t,
        {If[t[[1]] == "thick",
            Directive[EdgeForm[Black], RGBColor[0.9,0.7,0.2]],
            Directive[EdgeForm[Black], RGBColor[0.3,0.6,0.9]]],
         Polygon[t[[2]]]}],
        tiles],
      PlotLabel -> "Penrose Tiling (level " <> ToString[n] <> ")",
      ImageSize -> 500]]
```

15.11.2 `PenroseRhombusRatio`

Used in: Chapter 11, Section 11.7.2

```
PenroseRhombusRatio[n_Integer?Positive] :=
  Module[{tiles = PenroseInitial[], counts},
    Table[
      tiles = PenroseSubstituteAll[tiles];
      counts = Counts[tiles[[All, 1]]];
      {k,
       N[Lookup[counts, "thick", 0] /
         Lookup[counts, "thin", 1], 6]},
      {k, 1, n}]]
```

15.11.3 `SearchForPeriod`

Used in: Chapter 11

Searches for a translational period in a Penrose tiling. Returns a message if none is found within the search radius.

```
SearchForPeriod[g_Graphics, maxShift_Integer: 30] :=
  Module[{tiles = Cases[g, Polygon[pts_] :> Round[pts, 0.01], Infinity],
          centers, shifts, found = False},
    centers = Mean /@ tiles;
    Do[
      Module[{shifted = Map[# + {dx, dy} &, centers],
              matched},
        matched = Count[shifted,
          s_ /; MemberQ[centers, s, {1}, 1, SameTest ->
            (Norm[#1 - #2] < 0.05 &)]];
        If[matched == Length[centers],
           found = True;
           Print["Period found: {", dx, ", ", dy, "}"];
           Return[]]],
      {dx, -maxShift, maxShift, 0.5},
      {dy, -maxShift, maxShift, 0.5}];
    If[!found, "No translational period found within search radius " <>
               ToString[maxShift]]]
```

15.12 Chapter 12 — When the Computer Misleads You

15.12.1 `PolyaCheck`

Used in: Chapter 12

Tests Pólya's conjecture for all n from 2 to `max`.

```
LiouvilleLambda[n_Integer?Positive] :=
  (-1)^Total[FactorInteger[n][[All, 2]]]

PolyaCheck[max_Integer?Positive] :=
  Module[{cumSum = 0},
    AllTrue[Range[2, max], Function[n,
      cumSum += LiouvilleLambda[n];
      cumSum <= 0]]]
```

15.12.2 `PrimePolynomialRun`

Used in: Chapter 12, Section 12.7.1

Finds how long a polynomial produces primes for consecutive n = 0, 1, 2, …

```
PrimePolynomialRun[poly_] :=
  Module[{n = 0},
    While[PrimeQ[poly /. Slot[1] -> n] ||
          PrimeQ[poly /. x -> n], n++];
    {"First composite at n = " <> ToString[n],
     "Value: " <> ToString[poly /. x -> n]}]
```

15.12.3 `FloatingPointDemo`

Used in: Chapter 12

Demonstrates floating-point precision issues.

```
FloatingPointDemo[] :=
  {
   "Catastrophic cancellation:",
   "  (1.0 + 10^-16) - 1.0 = " <> ToString[(1.0 + 10^-16) - 1.0],
   "  Exact: (1 + 10^-16) - 1 = " <> ToString[(1 + 10^-16) - 1],
   "",
   "Accumulated rounding in sum of 1/n^2:",
   "  Floating: " <> ToString[N[Sum[1/n^2, {n, 1, 10^6}]]],
```

```
    "  Exact N: " <> ToString[N[Sum[1/n^2, {n, 1, 10^6}], 20]],
    "  Pi^2/6 = " <> ToString[N[Pi^2/6, 20]]
  }
```

15.12.4 `EulerProductConvergence`

Used in: Section 12.7.5

```
EulerProductConvergence[s_?NumericQ, maxPrimes_Integer] :=
  Module[{primes = Table[Prime[k], {k, 1, maxPrimes}],
          target = N[Pi^2/6, 15],
          partialProducts},
    partialProducts = Table[
      N[Product[1/(1 - Prime[k]^(-s)), {k, 1, p}], 15],
      {p, 1, maxPrimes}];
    ListLinePlot[
      Abs[partialProducts - target],
      PlotLabel -> "Euler Product Convergence Error (s=" <>
                   ToString[s] <> ")",
      AxesLabel -> {"Number of prime factors", "Error"},
      PlotStyle -> Blue,
      ScalingFunctions -> "Log"]]
```

15.13 Chapter 13 — Communication

15.13.1 `ConjectureReport`

Used in: Chapter 13

Generates a structured computational evidence report for a conjecture.

```
ConjectureReport[conjecture_String, testRange_List,
                 testFunction_, sampleExtremes_: False] :=
  Module[{lo = testRange[[1]], hi = testRange[[2]],
          allTrue, maxViolation, minViolation,
          start = AbsoluteTime[]},
    allTrue = AllTrue[Range[lo, hi], testFunction];
    Module[{elapsed = AbsoluteTime[] - start},
      {
       "Conjecture: " <> conjecture,
       "Test range: n = " <> ToString[lo] <> " to " <> ToString[hi],
       "Result: " <> If[allTrue, "True for all tested values",
                                 "FALSE -- counterexample exists"],
       "Computation time: " <> ToString[N[elapsed, 3]] <> " seconds"
      }]]
```

16 Appendix C — Guide to the OEIS

The **Online Encyclopedia of Integer Sequences** (oeis.org) is the indispensable reference for integer sequence research. This appendix explains how to use it effectively.

16.1 C.1 Searching the OEIS

Go to oeis.org and enter your sequence terms separated by commas in the search box:

```
1, 1, 2, 3, 5, 8, 13, 21
```

Tips for better searches:

- **Enter at least 6 terms.** Short sequences match too many entries.
- **Try without the first term.** Some sequences are indexed differently.
- **Try with signs.** If your sequence involves negative values, include them.
- **Try a subsequence.** If your full sequence gives no results, try just the first 8–10 terms.
- **Search for a formula.** You can search for words like "Fibonacci" or "prime" in the description box.

16.2 C.2 Reading an OEIS Entry

Each OEIS entry (identified by an A-number, e.g., A000045) contains:

Field	Content
Name	Brief description of the sequence
Data	First several terms
Offset	Index of the first term (0 or 1)

Field	Content
Comments	Mathematical context and observations
References	Books and papers
Links	URLs to related resources
Formula	Explicit, recursive, or generating function formulas
Example	Worked examples
Mathematica	Wolfram Language code
Python	Python code
Crossrefs	Related OEIS sequences
Keyword	Tags (e.g., “core”, “easy”, “hard”, “more”)

The **offset** field is important: A000045 (Fibonacci) has offset 0, meaning the first listed term is $F_0 = 0$. Always check the offset before using formulas.

16.3 C.3 Key Sequences Referenced in This Book

OEIS ID	Sequence	Chapter
A000040	Prime numbers	Chapter 1
A001223	Prime gaps	Chapter 1
A006877	Collatz delay records	Chapter 2
A006584	Collatz stopping times	Chapter 2
A000045	Fibonacci numbers	Chapter 3
A001175	Pisano periods	Chapter 3
A000796	Digits of π	Chapter 4
A007318	Pascal’s triangle	Chapter 5
A001333	CF convergents of $\sqrt{2}$	Chapter 6
A005132	Recamán’s sequence	Chapter 7
A005150	Look-and-say sequence	Chapter 7
A000108	Catalan numbers	Chapter 7
A002386	Maximal prime gaps	Section 1.7.1

16.4 C.4 Submitting a New Sequence

If your sequence is not in the OEIS and you believe it is new:

1. **Double-check your computation.** Verify at least 15 terms independently.
2. **Search for subsequences and transformations.** Try the sequence starting from the second term; try differences; try the sequence mod 2 or mod 3.
3. **Create an account** at oeis.org.
4. **Submit** using the "Submit a new sequence" link.
5. **Include:**
    - At least 15 terms
    - A precise definition
    - Your name and any observations
    - Wolfram Language or Python code to generate the sequence
6. **Wait** for review. An editor will check and may accept the sequence.

The OEIS has A-numbers beyond A380000 as of 2025 — there is room for well-documented new sequences.

16.5 C.5 OEIS in Wolfram Language

Wolfram Language has a built-in connection to the OEIS:

```
(* Search for a sequence by terms *)
WolframAlpha["1, 1, 2, 3, 5, 8, 13", {{"OEISResults", 1}, "Content"}]

(* Or use the OEIS directly in a browser and copy the A-number *)
(* Then look up in WL: *)
OEIS["A000045"]   (* if you have a package that supports this *)
```

For direct programmatic access, the OEIS provides a simple API:

```
(* Fetch OEIS entry A000045 as JSON *)
data = Import["https://oeis.org/search?q=id:A000045&fmt=json", "JSON"];
data[["results", 1, "data"]]   (* first several terms *)
```

16.6 C.6 Generating Sequences for OEIS Search

The helper function `GenerateAndSearch` (see Section 15.7.8) formats a computed sequence for pasting into the OEIS search box.

For sequences you compute in this book, a good habit is to always run `GenerateAndSearch` on any novel sequence you find — even if you suspect it is known. You may be surprised.

17 Appendix D — Further Reading

These are the books and resources that most directly connect to the themes of this book. They are organized by topic and annotated with the level of mathematical background needed.

17.1 D.1 Experimental Mathematics

Borwein, J. and Bailey, D. *Mathematics by Experiment: Plausible Reasoning in the 21st Century.* A K Peters, 2004.

The full technical version of the philosophy behind this book. Covers Ramanujan's identities, the BBP formula for π, integer relation algorithms, and much more. Requires calculus and some complex analysis for the later chapters. The source of the quote used in our preface.

Borwein, J., Bailey, D., and Girgensohn, R. *Experimentation in Mathematics: Computational Paths to Discovery.* A K Peters, 2004.

More advanced companion to the above. Focused on specific research problems solved through computation.

Martin, C. *Keep It Real.* 2011. *(Available in this project's reading list — see project files.)*

A practitioner's guide to doing and communicating experimental mathematics research. Particularly useful for Chapter 13.

17.2 D.2 Number Theory

Hardy, G. H. and Wright, E. M. *An Introduction to the Theory of Numbers.* Oxford University Press, 6th ed. 2008.

The classic reference for elementary and analytic number theory. Covers primes, Fibonacci numbers, continued fractions, and much more with full proofs. Requires only high school algebra for the first several chapters; calculus for the later ones.

Sierpiński, W. *Elementary Theory of Numbers.* PWN, 1987.

Accessible treatment of divisibility, primes, and Diophantine equations. Particularly good on Pell equations (Chapter 6) and Goldbach-type problems.

Guy, R. K. *Unsolved Problems in Number Theory.* 3rd ed. Springer, 2004.

A catalog of open problems. The Collatz conjecture, Goldbach's conjecture, and many others in this book appear here with references to partial results. Essential for understanding which problems are "known hard."

17.3 D.3 Chaos and Dynamical Systems

Strogatz, S. *Nonlinear Dynamics and Chaos.* 2nd ed. Westview Press, 2015.

The best introductory textbook on the logistic map, bifurcations, Lyapunov exponents, and chaos. Requires calculus. Chapter 9 of this book follows Strogatz's approach.

Gleick, J. *Chaos: Making a New Science.* Viking, 1987.

A narrative history of chaos theory, covering Feigenbaum's constant, the Lorenz attractor, and fractal geometry. No mathematics required. Excellent motivation for the technical material.

17.4 D.4 Fractal Geometry

Mandelbrot, B. *The Fractal Geometry of Nature.* W. H. Freeman, 1982.

The original text introducing fractal geometry. More philosophical and visual than mathematical; accessible to anyone. Chapter 10 draws directly on Mandelbrot's ideas.

Barnsley, M. *Fractals Everywhere.* 2nd ed. Academic Press, 1993.

Rigorous treatment of iterated function systems, the chaos game, and fractal dimension. Requires calculus and some real analysis.

Falconer, K. *Fractal Geometry: Mathematical Foundations and Applications.* 3rd ed. Wiley, 2014.

The standard graduate textbook on fractal geometry. Covers Hausdorff dimension, self-similar sets, and the Mandelbrot set with full proofs. Requires real analysis.

17.5 D.5 Cellular Automata

Wolfram, S. *A New Kind of Science.* Wolfram Media, 2002.

Stephen Wolfram's comprehensive exploration of cellular automata, computation, and the nature of complexity. Freely available at wolframscience.com. Chapter 8 follows Wolfram's classification. The book is long and occasionally controversial — read it critically.

17.6 D.6 Sequences and the OEIS

Sloane, N. J. A. *The On-Line Encyclopedia of Integer Sequences.* oeis.org. Continuously updated.

The definitive reference for integer sequences. See Appendix C for guidance on using it.

Havil, J. *Gamma: Exploring Euler's Constant.* Princeton University Press, 2003.

A beautiful exploration of the Euler-Mascheroni constant $\gamma \approx 0.5772$, which connects prime gaps, the harmonic series, and the Riemann zeta function. Accessible with only high school algebra.

17.7 D.7 Mathematical Writing and Communication

Halmos, P. *How to Write Mathematics.* American Mathematical Society, 1973.

A short, classic essay on writing mathematical prose clearly. Essential reading before writing up your research (Chapter 13).

Knuth, D., Larrabee, T., and Roberts, P. *Mathematical Writing.* Mathematical Association of America, 1989.

Based on a Stanford course. Covers notation, clarity, and the conventions of professional mathematical writing.

17.8 D.8 Tilings and Symmetry

Grünbaum, B. and Shephard, G. *Tilings and Patterns.* W. H. Freeman, 1987.

The definitive mathematical reference on tilings. Covers periodic tilings, symmetry groups, and aperiodic tilings including Penrose. Requires only geometric intuition for most chapters.

Penrose, R. *The Road to Reality.* Knopf, 2004.

A comprehensive tour of modern physics and mathematics by the discoverer of Penrose tilings. Chapter 11 connects to his work on quasicrystals and aperiodic order.